

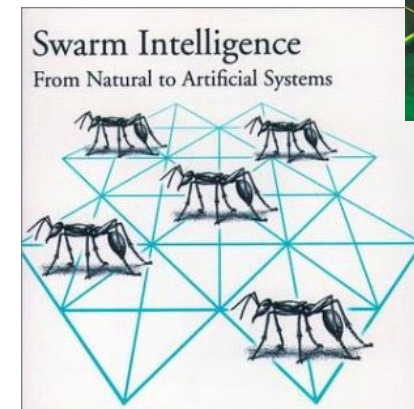
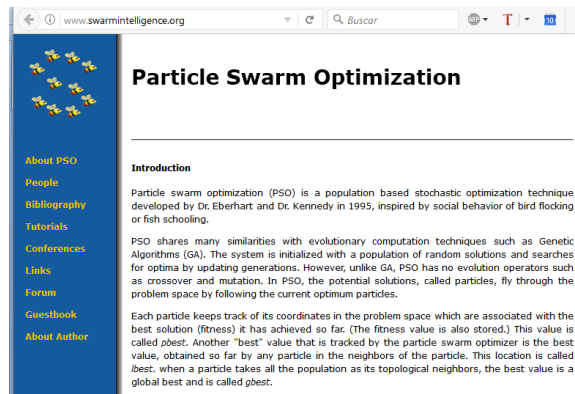
Tema 5: Inteligencia social y colectiva de enjambre

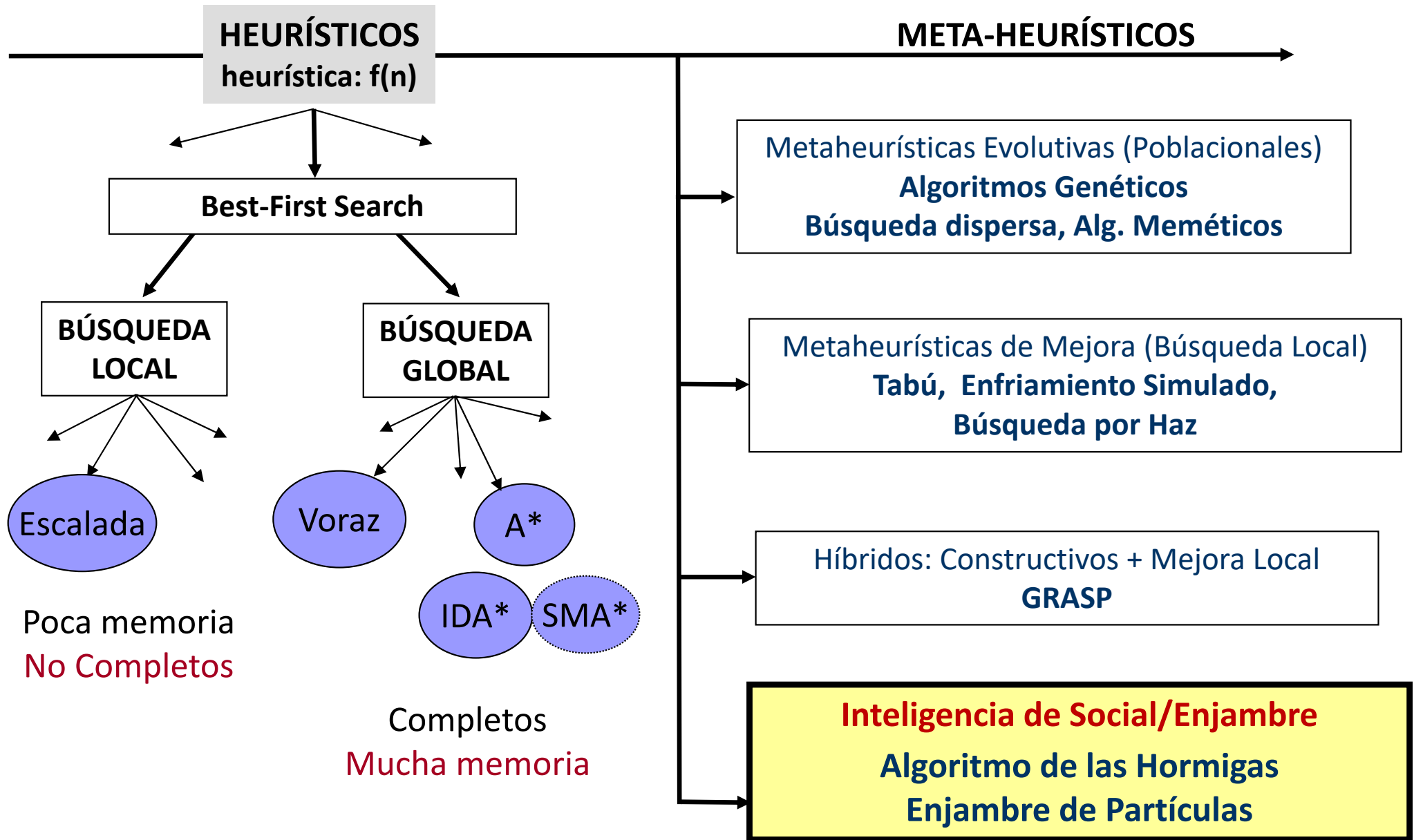
Algoritmo de las Hormigas (Ant Colony Optimization - ACO)

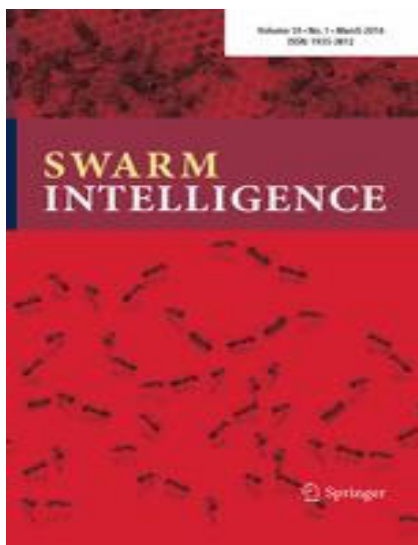
Enjambre de partículas (Particle Swarm Optimization - PSO)

Otras variantes (Inteligencia Social, Inteligencia Colectiva, Metaheurísticas Bio-Inspiradas...)

<http://www.swarmintelligence.org/>







Artificial Intelligence Review (2020) 53:5589–5635
<https://doi.org/10.1007/s10462-020-09829-2>

Performance comparison of five metaheuristic nature-inspired algorithms to find near-OGRs for WDM systems

Artif Intell Rev (2019) 52:2191–2233
<https://doi.org/10.1007/s10462-017-9605-z>

Metaheuristic research: a comprehensive survey

Artificial Intelligence Review (2020) 53:753–810
<https://doi.org/10.1007/s10462-018-09676-2>

From ants to whales: metaheuristics for all tastes

Artículos y vídeos en Poliformat

 [ACO Timetabling Tool.pdf](#)

 [Ant_Colony_Optimization_Book.pdf](#)

 [Aplicaciones Algoritmo Hormigas.pdf](#)

 [Aplicaciones Metaheuristica POO.pdf](#)

 [APPLICATION IN POWER SYSTEM PROBLEMS.pdf](#)

 [Comparison of 5 novel nature inspired metaheuristic.pdf](#)

 [From ants to whales metaheuristics for all tastes \(2020\).pdf](#)

 [Metaheuristic Review 2019.pdf](#)

 [multi-objective particle swarm optimization.pdf](#)

 [PARTICLE SWARM OPTIMIZATION.pdf](#)

 [Particle Swarm Optimization Algorithms.pdf](#)

 [PSO for engineering problems.pdf](#)

 [PSO-Survey.pdf](#)

 [PSO Viajante Multiobjetivo.pdf](#)

 [State of the art review of intelligent scheduling.pdf](#)

 [Survey PSO para timetabling.pdf](#)

Basada en el comportamiento colectivo de sistemas descentralizados y auto-organizados

- Un sistema auto-organizado mantiene una estructura (funcionalidad) independiente del exterior, o de un control global, y que proviene de las interacciones en los niveles más bajos

Múltiples interacciones, junto con realimentación positiva /negativa

Amplificación en la selección de opciones aleatorias

- En un sistema descentralizado el control está completamente distribuido entre sus elementos componentes



Estigmergia: stigma (estimulación) + ergon (trabajo)

Colaboración a través del medio físico

Interacción por el entorno:

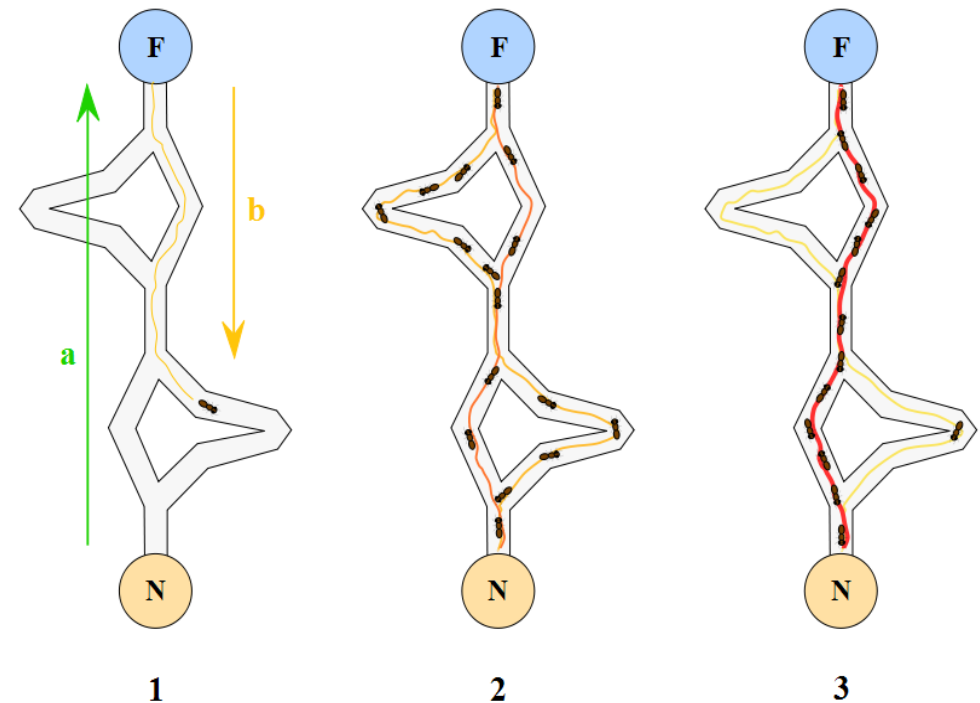
- Los agentes actúan como respuesta al entorno y a otros agentes, mediante reglas de conducta muy simples.
- Los agentes modifican (indirectamente) el entorno: interaccionan (colaboran) entre ellos a través del entorno (medio físico).
- El entorno actúa como una memoria de trabajo.
- La interacción puede ser realizada por cualquier individuo.
- Las reglas simples de la respuesta al entorno pueden crear diferentes patrones de conducta.

Inteligencia de enjambre: basada en la observación de la naturaleza (**bio-inspirada**)

- Se basa en una población (sociedad) de agentes individuales, auto-organizados y sin control centralizado.
- Los individuos interactúan entre sí y con el entorno, de forma localizada y siguiendo reglas muy simples.
- Hay un cierto seguimiento al líder, a los buenos exploradores, o a la costumbre social.
- A diferencia de las metaheurísticas poblacionales, los individuos no son eliminados (no hay reemplazo), no hay control centralizado y los individuos suelen estar en constante movimiento.

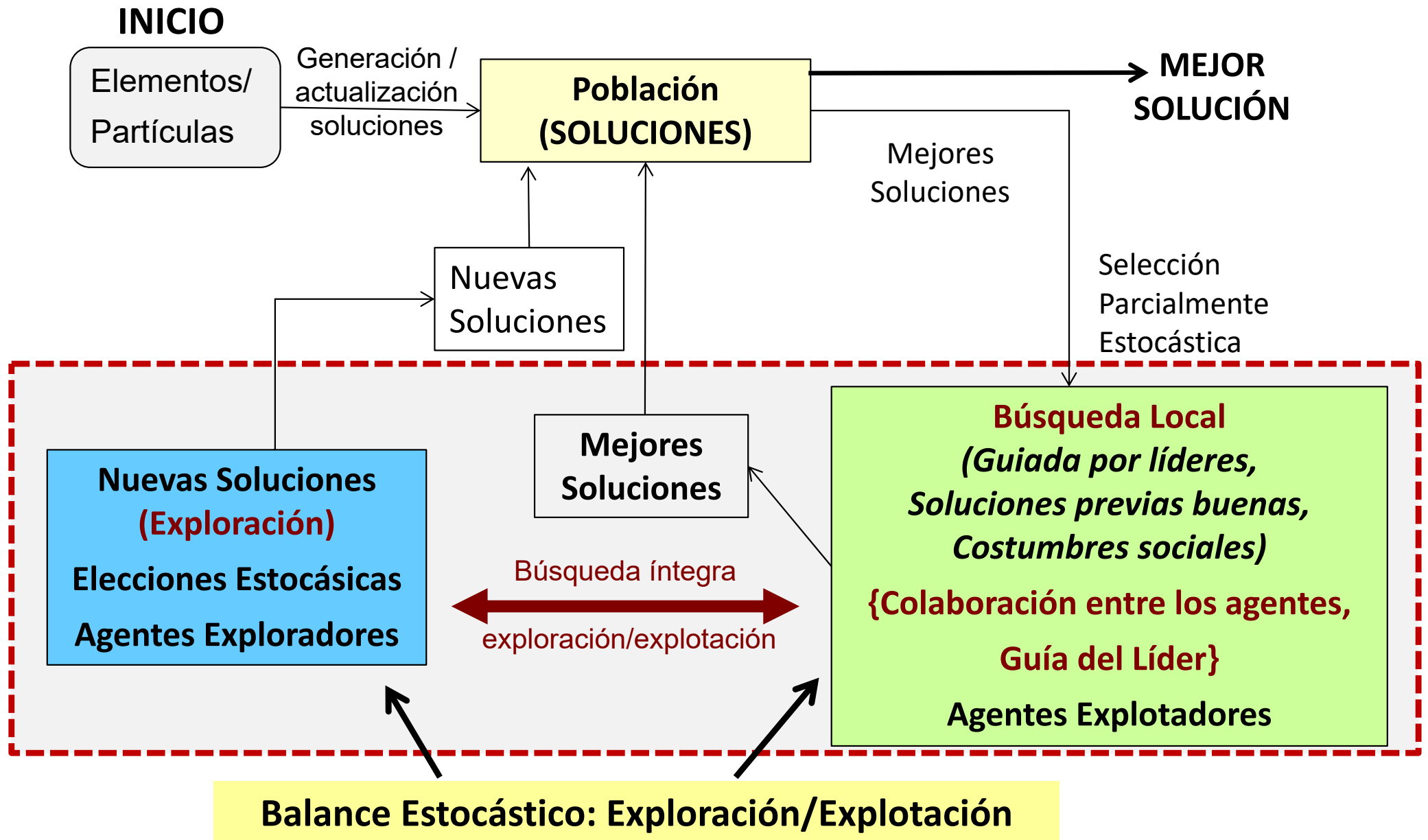
Resultado:

- La **interacción** entre los agentes induce una cierta **inteligencia colectiva** (desconocida incluso para los propios individuos)
- La funcionalidad del sistema global **transciende** el repertorio de las conductas individuales
- La productividad del sistema (colonia) es mayor que el esfuerzo de sus miembros
- La respuesta conjunta del grupo es **robusta, flexible y adaptativa** con respecto a los cambios en el entorno (**sistemas auto-inmunes**)



Esquema general

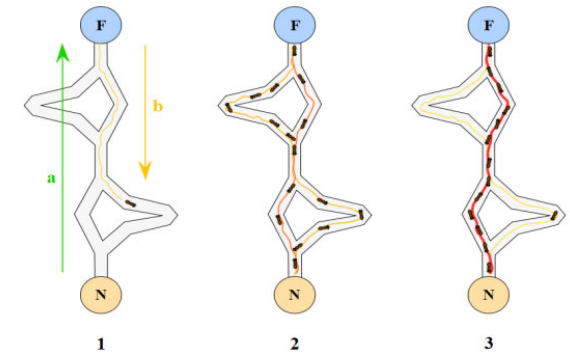
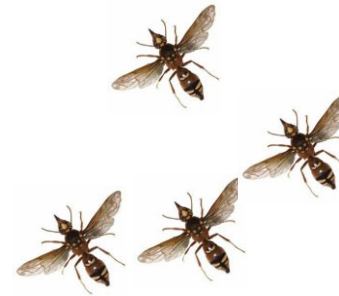
Los elementos/partículas generan/actualizan las soluciones



Variantes principales

- Algoritmo de las Hormigas
(ACO, Ant Colony Optimization)

- Enjambre de partículas
(PSO, Particle Swarm Optimization)
Bandada de Pájaros, Bancos de Peces, etc.



Otras metaheurísticas (enjambre/bioinspiradas):

Algoritmo de las abejas,
Algoritmo de los murciélagos (Bat algorithm),
Algoritmo de las luciérnagas (glowworm swarm optimization, GSO),
Caída inteligente de gotas de agua (Intelligent Water Drops, IWD),
Etc.

Algoritmo de las Hormigas

(ACO, Ant Colony Optimization) M. Dorigo, 1992

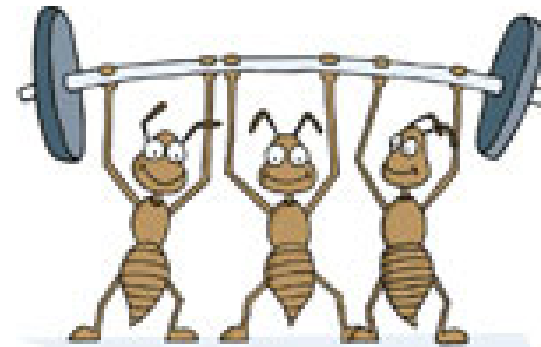
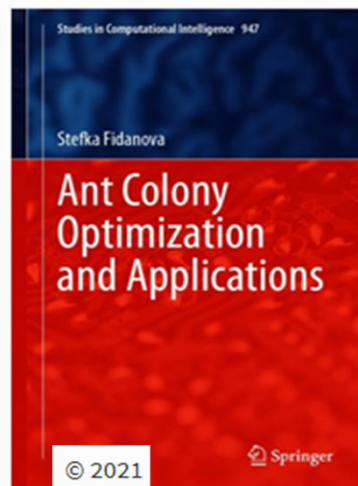


http://www.scholarpedia.org/article/Ant_colony_optimization

Google Académico

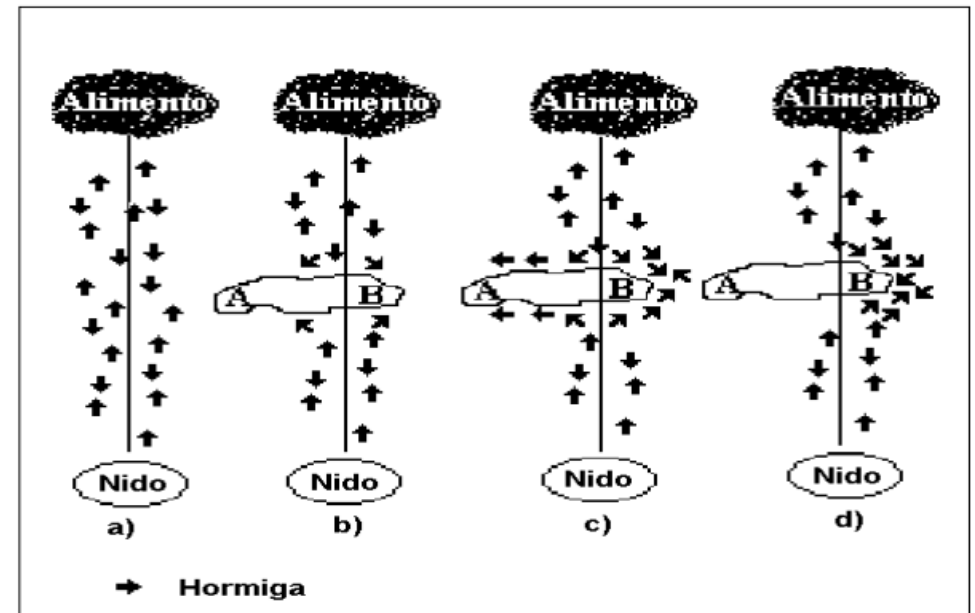
<http://www.midaco-solver.com/>

Artículos y vídeos en Poliformat



Basado en la observación de la naturaleza: Hormigas en la búsqueda de comida.

- Las hormigas recorren diversos caminos buscando el alimento. Eventualmente, encuentran (si existen) varias sendas (soluciones).
- Las hormigas dejan una sustancia (feromona), marcando el rastro que recorren.
 - La feromona se evapora con el tiempo.
 - La feromona en cada punto de un camino dependerá del número (caudal) de hormigas que han dejado su rastro.
- Las hormigas, en la búsqueda del camino, siguen el rastro de las feromonas.
- El camino más corto tendrá el mayor número de paso de hormigas: más caudal $\Rightarrow$ mayor feromona
- Al final, todas las hormigas siguen el camino "óptimo".
- Aplicable para obtener sendas optimizadas en grafos (y problemas equivalentes).



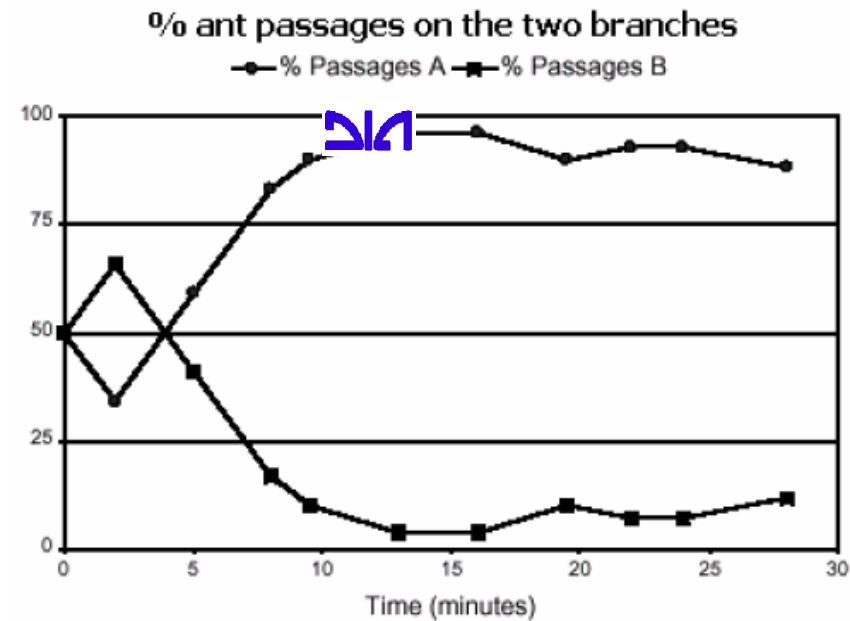
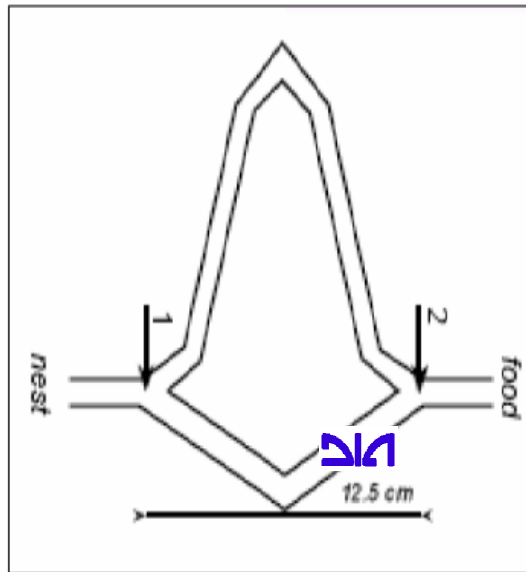
Al ser interrumpido el camino directo, la mitad de las hormigas se dirigen por A y la otra mitad por B.

- B es la menor distancia para rodear el obstáculo, luego pasan mayor cantidad de hormigas y depositan más feromonas.

Las hormigas siguen el rastro de las feromonas y elegirán rodear el obstáculo por B, lo que a su vez depositará más feromonas en ese trayecto.

Por realimentación positiva, el camino B será seleccionado como el camino más corto.

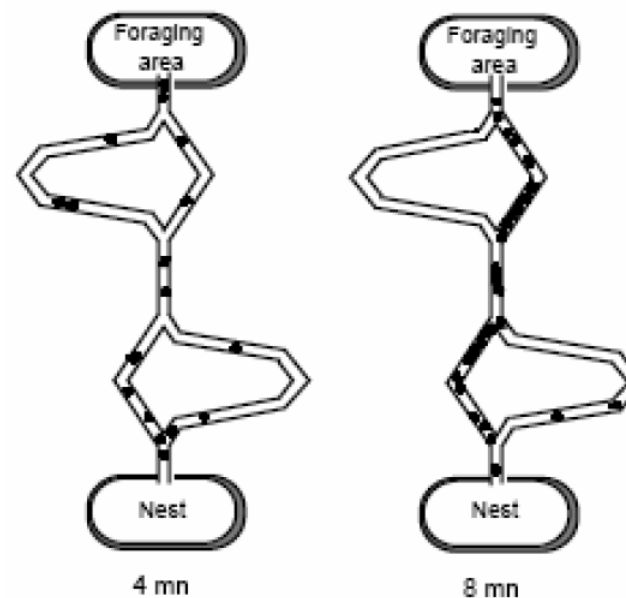
Antecedentes... Estudio sobre las Hormigas 'biológicas' (Goss et al., 1989, Deneubourg et al., 1990).



Self-organized shortcuts in the Argentine ant. Goss et al., 1989

Estudiaron el porcentaje de hormigas que tomaban cada camino.

Inicialmente los caminos lo elegían de manera arbitraria, pero casi todas acababan siguiendo el mismo camino.



A partir de sus experimentos, construyeron un **modelo estocástico sencillo** para describir la dinámica de las hormigas:

$P_{i,a}$: La probabilidad de que una hormiga llegue a la intersección i y seleccione la rama a en un instante t es:

$$p_{i,a} = \frac{[k + \tau_{i,a}]^{\alpha}}{[k + \tau_{i,a}]^{\alpha} + [k + \tau_{i,a'}]^{\alpha}}$$

donde

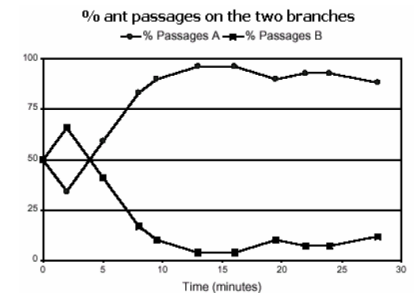
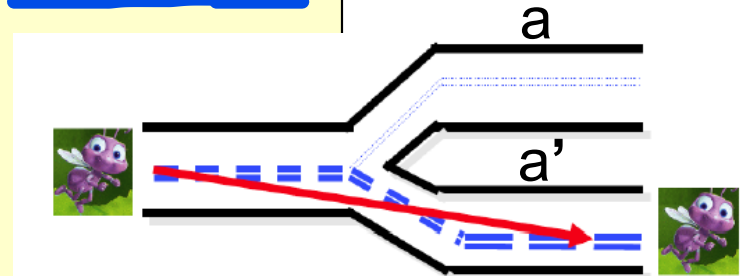
$\tau_{i,a}$ es la concentración de feromona en la rama a de la intersección i ,

$\tau_{i,a'}$ es la concentración de feromona en la otra rama de la intersección i ,

$\alpha = 2$ (obtenido de diversos experimentos),

k es el tiempo que se tarda en atravesar la rama a .

La concentración de feromona depende del caudal de hormigas



- La **metaheurística ACO** reproduce el comportamiento de las hormigas reales en una colonia artificial de hormigas para resolver problemas complejos de camino mínimo
- Cada **hormiga artificial** (agente) es un mecanismo probabilístico de **construcción** de soluciones al problema que **combina**:
 - a) Rastros de **feromona (artificiales) τ** que cambian con el tiempo para reflejar la **experiencia** adquirida por los agentes previos en la resolución del problema (explotación).
 - b) Información **heurística η** sobre el problema.

Aplicación al Problema del Viajante de comercio

- El objetivo es encontrar la senda más corta que recorre una serie de ciudades.
- Cada hormiga hace una senda alternativa de las posibles:
“cada hormiga debe visitar todas las ciudades, y cada ciudad exactamente una vez”
- En cada iteración, cada hormiga decide pasar de una ciudad a otra, en base a:
 - 1 a) Exploración Propia: Una ciudad lejana tiene menos posibilidades de ser elegida frente a una cercana (por **visibilidad: heurística η**),
 - 2 b) Explotación/Colaboración: Cuanto más marcado esté el camino (más intensa es la **feromona τ**) entre dos ciudades, mayor probabilidad de que sea elegido,
- Cada hormiga, tras completar su viaje, ha depositado más feromonas en sus trayectos si su viaje ha sido más corto (carga/longitud).
- Las feromonas se evaporan con el tiempo.

Procedure Hormigas

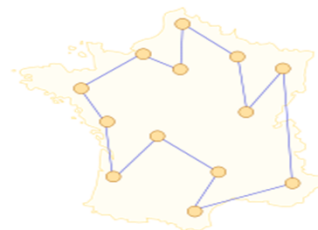
while (no-fin)

GenerarSoluciones()

Actualización-Feromonas()

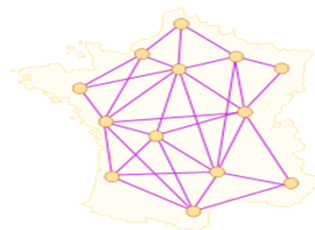
end while

end procedure



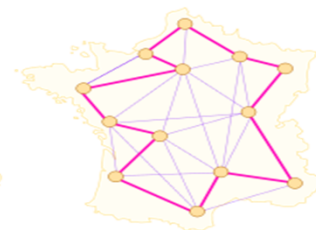
1

obtención
de una
solución
(1 hormiga)



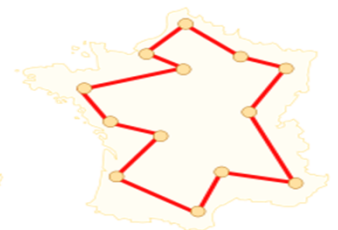
2

múltiples
hormigas
buscando



3

el mejor camino
se va definiendo
(arcos de mayor
feromona)



4

se obtiene el
mejor
camino

Algoritmo de las Hormigas. Generación de soluciones

Procedure Hormigas

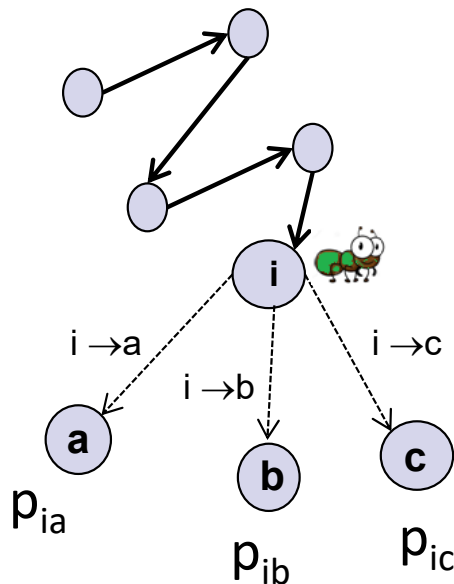
while (no-fin)

GenerarSoluciones()

Actualización-Feromonas()

end while

end procedure



GenerarSoluciones()

;aplicación al viajante

- Cada hormiga obtendrá un camino: *Soluciones* $\equiv$ *Caminos*
- En cada iteración, las hormigas eligen un trayecto $i \rightarrow j$.
(si N ciudades, entonces N-1 iteraciones)

Para cada arco i ($i=1, N-1$) ; N: ciudades

Para cada Hormiga h ($h=1, H$) ; H: Hormigas

Hormiga h selecciona siguiente trayecto (ciudad) en su senda

- Criterio de Selección de Trayecto

Una hormiga irá de $i \rightarrow j$, con probabilidad

$$p_{i,j} = \frac{(\tau_{i,j}^{\alpha})(\eta_{i,j}^{\beta})}{\sum (\tau_{i,j}^{\alpha})(\eta_{i,j}^{\beta})}$$

← Valores del arco $i \rightarrow j$
← Sumatorio para todos los nodos del grafo

Donde,

τ_{ij} es la feromona de arco $i \rightarrow j$, con una influencia α

η_{ij} es la visibilidad del arco (típicamente, $1/\text{distancia}$), con influencia β

Algoritmo de las Hormigas. Actualización de las feromonas

Procedure Hormigas

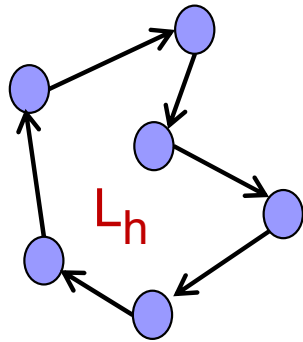
while (no-fin)

GenerarSoluciones()

Actualización-Feromonas()

end while

end procedure



- Con un buen diseño, los caminos de todas las hormigas acaban convergiendo.
- El proceso acaba tras un número dado de iteraciones, o cuando se cumple un criterio de terminación

Actualización-Feromonas()

Las feromonas en los arcos se actualizan tras la obtención de un camino (solución) por todas las hormigas.

La feromona de cada arco $i \rightarrow j$ (τ_{ij}) se actualiza de la forma:

$$\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$$

Donde,

ρ Es la tasa de evaporación, que se aplica a la feromona previa τ'_{ij}

$\Delta\tau_{ij}$ Es la feromona que se deposita tras los caminos encontrados:

$\Delta\tau_{ij} = \sum_{h=1,H} \Delta\tau^h_{ij}$ Es la suma de lo que depositan todas las hormigas

siendo $\Delta\tau^h_{ij}$ la feromona dejada por hormiga h en $i \rightarrow j$:

$= 1/L_h$ si la hormiga h atraviesa el arco

$= 0$ en otro caso

L_h es el coste (distancia) de todo el camino obtenido por la hormiga h (estamos **minimizando**, así que a mayor coste, $1/L_h$ será menor)

Algoritmo de las Hormigas. Implementación

Necesitamos 3 matrices

Matriz de feromonas: Feromonas del arco $i \rightarrow j$

Las feromonas de cada celda (i,j) se actualizan tras cada iteración:

$$\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$$

Matriz de Visibilidades:

Distancia del arco $i \rightarrow j$

Matriz de Probabilidades:

Probabilidades del arco $i \rightarrow j$

Las probabilidades de cada celda (i,j) se actualizan tras cada iteración

$$p_{i,j} = \frac{(\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}{\sum (\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}$$

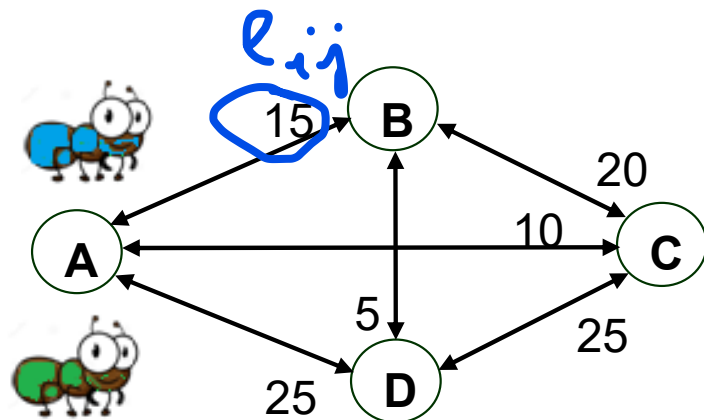
	1	2	3	4	5	...	n-1	n
1								
2								
3								
4								
.....								
n-1								
n								

Parámetros: Hormigas,
 ρ (tasa evaporación),
 α, β (pesos), τ_{INICIAL} (feromona)

$\uparrow \rho, \beta, \tau_{\text{INICIAL}} \Rightarrow$ Exploración

$\downarrow \rho, \uparrow \alpha \Rightarrow$ Explotación ($\approx$ feromona colaborativa)

Ejemplo. Problema del viajante de comercio



Parámetros:

Hormigas $\{h_i\}=2$, $\rho=0.5$, $\alpha=0.4$, $\beta=0.6$, $\tau_{\text{INICIAL}}=0.1$ (>0)

Matriz de Visibilidades: $\eta_{ij}=1/l_{ij}$

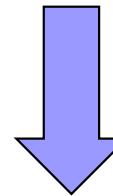
	A	B	C	D
A	0.0000	0.0667	0.1000	0.0400
B	0.0667	0.0000	0.0500	0.2000
C	0.1000	0.0500	0.0000	0.0400
D	0.0400	0.2000	0.0400	0.0000

Denominador:

$$\sum (\tau_{i,j}^{\alpha})(\eta_{i,j}^{\beta})$$

Numerador: $(\tau_{i,j}^{\alpha})(\eta_{i,j}^{\beta})$

	A	B	C	D	Denominador (Suma)
A	0.0000	0.0784	0.1000	0.0577	0.2361
B	0.0784	0.0000	0.0660	0.1516	0.2960
C	0.1000	0.0660	0.0000	0.0577	0.2237
D	0.0577	0.1516	0.0577	0.0000	0.2670



Matriz de Probabilidades: $p_{i,j} = \frac{(\tau_{i,j}^{\alpha})(\eta_{i,j}^{\beta})}{\sum (\tau_{i,j}^{\alpha})(\eta_{i,j}^{\beta})}$

	A	B	C	D
A	0.0000	0.3321	0.4235	0.2444
B	0.2649	0.0000	0.2229	0.5121
C	0.4471	0.2949	0.0000	0.2580
D	0.2161	0.5677	0.2161	0.0000

Matriz de Feromonas: $\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$

	A	B	C	D
A	0.1	0.1	0.1	0.1
B	0.1	0.1	0.1	0.1
C	0.1	0.1	0.1	0.1
D	0.1	0.1	0.1	0.1

Ejemplo. Problema del viajante de comercio. Iteración 1. Tramo 1

Parámetros:

Hormigas $\{h_i\}=2$, $\rho=0.5$, $\alpha=0.4$, $\beta=0.6$, $\tau_{\text{INICIAL}}=0.1$ (>0)

Matriz de Visibilidades: $(\eta_{ij}=1/l_{ij})$

	A	B	C	D
A	0.0000	0.0667	0.1000	0.0400
B	0.0667	0.0000	0.0500	0.2000
C	0.1000	0.0500	0.0000	0.0400
D	0.0400	0.2000	0.0400	0.0000

Matriz de Feromonas: $\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$

	A	B	C	D
A	0.1	0.1	0.1	0.1
B	0.1	0.1	0.1	0.1
C	0.1	0.1	0.1	0.1
D	0.1	0.1	0.1	0.1

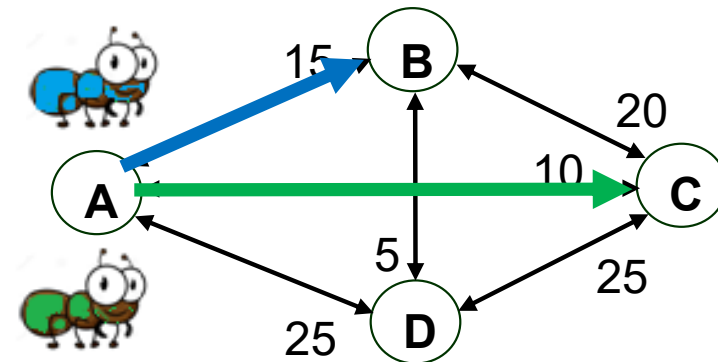
Matriz de Probabilidades: $p_{i,j} = \frac{(\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}{\sum (\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}$

	A	B	C	D
A	0.0000	0.3321	0.4235	0.2444
B	0.2649	0.0000	0.2229	0.5121
C	0.4471	0.2949	0.0000	0.2580
D	0.2161	0.5677	0.2161	0.0000

Selección aleatoria:

h1, en A elige C ($p=0.4235$) {A, C}

h2, en A elige B ($p=0.3321$) {A, B}



Ejemplo. Problema del viajante de comercio. Iteración 1. Tramo 2

Parámetros:

Hormigas $\{h_i\}=2$, $\rho=0.5$, $\alpha=0.4$, $\beta=0.6$, $\tau_{\text{INICIAL}}=0.1$ (>0)

Matriz de Visibilidades: ($\eta_{ij}=1/l_{ij}$)

	A	B	C	D
A	0.0000	0.0667	0.1000	0.0400
B	0.0667	0.0000	0.0500	0.2000
C	0.1000	0.0500	0.0000	0.0400
D	0.0400	0.2000	0.0400	0.0000

Matriz de Feromonas: $\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$

	A	B	C	D
A	0.1	0.1	0.1	0.1
B	0.1	0.1	0.1	0.1
C	0.1	0.1	0.1	0.1
D	0.1	0.1	0.1	0.1

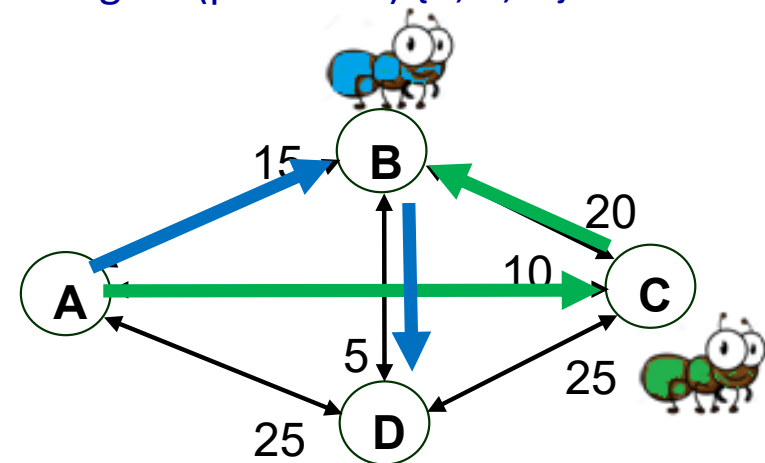
Matriz de Probabilidades: $p_{i,j} = \frac{(\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}{\sum (\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}$

	A	B	C	D
A	0.0000	0.3321	0.4235	0.2444
B	0.2649	0.0000	0.2229	0.5121
C	0.4471	0.2949	0.0000	0.2580
D	0.2161	0.5677	0.2161	0.0000

Selección aleatoria:

h1, en C elige B ($p=0.2949$) {A, C, **B**}

h2, en B elige D ($p=0.5121$) {A, B, **D**}



Ejemplo. Problema del viajante de comercio. Iteración 1. Tramo 3

Parámetros:

Hormigas $\{h_i\}=2$, $\rho=0.5$, $\alpha=0.4$, $\beta=0.6$, $\tau_{\text{INICIAL}}=0.1$ (>0)

Matriz de Visibilidades: $(\eta_{ij}=1/l_{ij})$

	A	B	C	D
A	0.0000	0.0667	0.1000	0.0400
B	0.0667	0.0000	0.0500	0.2000
C	0.1000	0.0500	0.0000	0.0400
D	0.0400	0.2000	0.0400	0.0000

Matriz de Feromonas: $\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$

	A	B	C	D
A	0.1	0.1	0.1	0.1
B	0.1	0.1	0.1	0.1
C	0.1	0.1	0.1	0.1
D	0.1	0.1	0.1	0.1

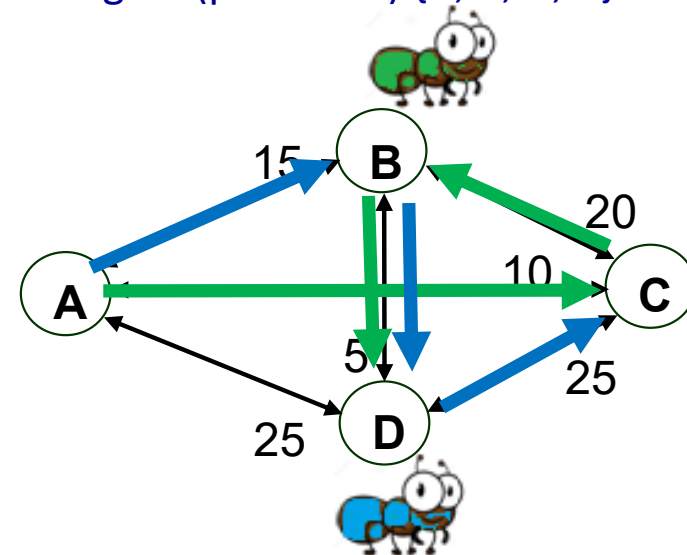
Matriz de Probabilidades: $p_{i,j} = \frac{(\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}{\sum (\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}$

	A	B	C	D
A	0.0000	0.3321	0.4235	0.2444
B	0.2649	0.0000	0.2229	0.5121
C	0.4471	0.2949	0.0000	0.2580
D	0.2161	0.5677	0.2161	0.0000

Selección aleatoria:

h1, en B elige D ($p=0.5121$) {A, C, B, **D**}

h2, en D elige C ($p=0.2161$) {A, B, D, **C**}



Ejemplo. Problema del viajante de comercio. Iteración 1. Tramo 4

Parámetros:

Hormigas $\{h_i\}=2$, $\rho=0.5$, $\alpha=0.4$, $\beta=0.6$, $\tau_{\text{INICIAL}}=0.1$ (>0)

Matriz de Visibilidades: ($\eta_{ij}=1/l_{ij}$)

	A	B	C	D
A	0.0000	0.0667	0.1000	0.0400
B	0.0667	0.0000	0.0500	0.2000
C	0.1000	0.0500	0.0000	0.0400
D	0.0400	0.2000	0.0400	0.0000

Matriz de Feromonas: $\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$

	A	B	C	D
A	0.1	0.1	0.1	0.1
B	0.1	0.1	0.1	0.1
C	0.1	0.1	0.1	0.1
D	0.1	0.1	0.1	0.1

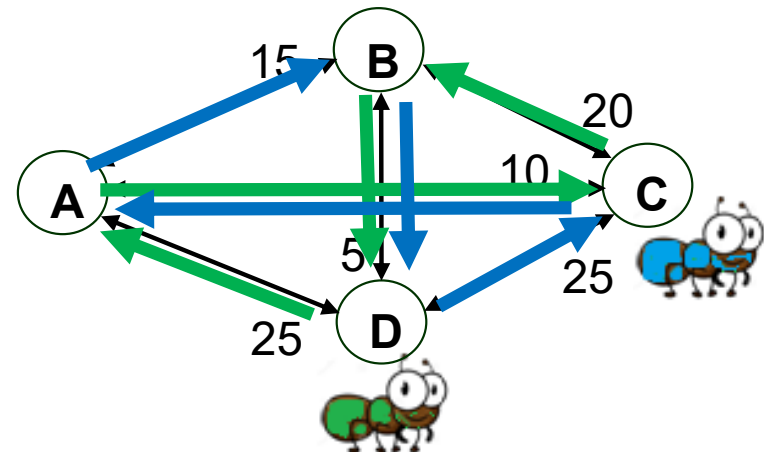
Matriz de Probabilidades: $p_{i,j} = \frac{(\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}{\sum (\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}$

	A	B	C	D
A	0.0000	0.3321	0.4235	0.2444
B	0.2649	0.0000	0.2229	0.5121
C	0.4471	0.2949	0.0000	0.2580
D	0.2161	0.5677	0.2161	0.0000

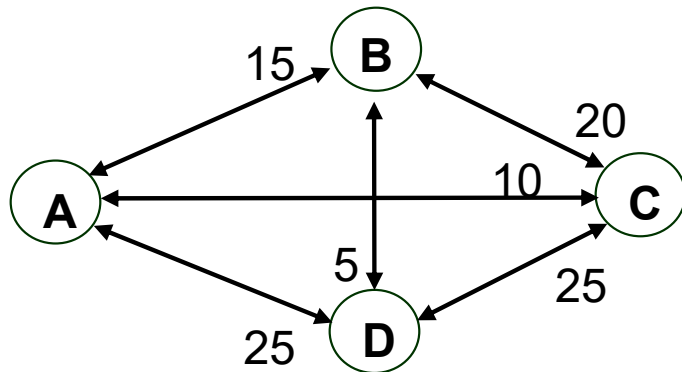
Selección aleatoria:

h1, en D elige A ($p=0.2161$) {A, C, B, D, **A**} (coste 60)

h2, en C elige A ($p=0.4471$) {A, B, D, C, **A**} (coste 55)



Ejemplo. Problema del viajante de comercio. Iteración 1. Actualización de feromonas



Parámetros:

Hormigas $\{h_i\}=2$, $\rho=0.5$, $\alpha=0.4$, $\beta=0.6$, $\tau_{\text{INICIAL}}=0.1 (>0)$

h1, con L1 = 60, {A, C, B, D, A}

En cada tramo (arco) que pasa deja 1/60 de feromonas

h2, con L2 = 55, {A, B, D, C, A}

En cada tramo (arco) que pasa deja 1/55 de feromonas

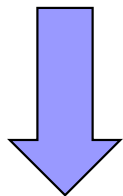
La feromona de cada arco $i \rightarrow j$ (τ_{ij}) se actualiza de la forma: $\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$

Matriz de Feromonas (actual)

	A	B	C	D
A	0.1	0.1	0.1	0.1
B	0.1	0.1	0.1	0.1
C	0.1	0.1	0.1	0.1
D	0.1	0.1	0.1	0.1

Alternativamente, resultado si $\rho=0.9$

	A	B	C	D
A	0.01	0.02818182	0.02666667	0.01
B	0.01	0.01	0.01	0.04484848
C	0.02818182	0.02666667	0.01	0.01
D	0.02666667	0.01	0.02818182	0.01



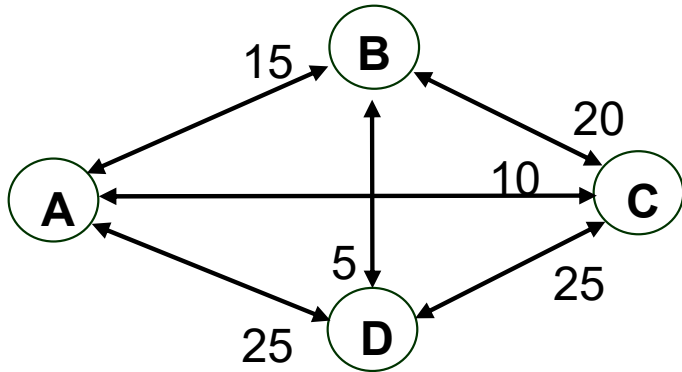
Matriz de Feromonas
tras la evaporación

	A	B	C	D
A	0.05	0.05	0.05	0.05
B	0.05	0.05	0.05	0.05
C	0.05	0.05	0.05	0.05
D	0.05	0.05	0.05	0.05

Matriz de Feromonas **tras añadir
las nuevas feromonas** (actualizada)

	A	B	C	D
A	0.05	0.06818182	0.06666667	0.05
B	0.05	0.05	0.05	0.08484848
C	0.06818182	0.06666667	0.05	0.05
D	0.06666667	0.05	0.06818182	0.05

Ejemplo. Problema del viajante de comercio. Iteración 1. Actualización de probabilidades



Parámetros:

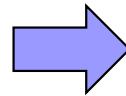
Hormigas $\{h_i\}=2$, $\rho=0.5$, $\alpha=0.4$, $\beta=0.6$, $\tau_{\text{INICIAL}}=0.1 (>0)$

Tras actualizar las feromonas, hay que actualizar la matriz de probabilidades para la siguiente iteración

$$\text{Matriz de Probabilidades: } p_{i,j} = \frac{(\tau_{i,j}^{\alpha})(\eta_{i,j}^{\beta})}{\sum (\tau_{i,j}^{\alpha})(\eta_{i,j}^{\beta})}$$

Matriz de Probabilidades (actual)

	A	B	C	D
A	0.0000	0.3321	0.4235	0.2444
B	0.2649	0.0000	0.2229	0.5121
C	0.4471	0.2949	0.0000	0.2580
D	0.2161	0.5677	0.2161	0.0000



Matriz de Probabilidades (actualizada)

	A	B	C	D
A	0.0000	0.3432	0.4337	0.2231
B	0.2364	0.0000	0.1989	0.5647
C	0.4622	0.3022	0.0000	0.2356
D	0.2299	0.5382	0.2320	0.0000

El tramo B→A no se ha recorrido: su probabilidad disminuye

El tramo B→D se ha recorrido por las dos hormigas: su probabilidad aumenta

Ejemplo. Problema del viajante de comercio. Iteración 2

Parámetros:

Hormigas $\{h_i\}=2$, $\rho=0.5$, $\alpha=0.4$, $\beta=0.6$

Matriz de Visibilidades: $(\eta_{ij}=1/l_{ij})$

	A	B	C	D
A	0.0000	0.0667	0.1000	0.0400
B	0.0667	0.0000	0.0500	0.2000
C	0.1000	0.0500	0.0000	0.0400
D	0.0400	0.2000	0.0400	0.0000

Matriz de Feromonas: $\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$

	A	B	C	D
A	0.05	0.06818182	0.06666667	0.05
B	0.05	0.05	0.05	0.08484848
C	0.06818182	0.06666667	0.05	0.05
D	0.06666667	0.05	0.06818182	0.05

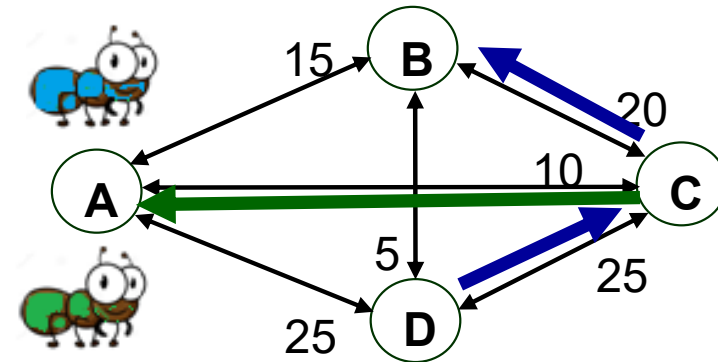
Matriz de Probabilidades: $p_{i,j} = \frac{(\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}{\sum (\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}$

	A	B	C	D
A	0.0000	0.3432	0.4337	0.2231
B	0.2364	0.0000	0.1989	0.5647
C	0.4622	0.3022	0.0000	0.2356
D	0.2299	0.5382	0.2320	0.0000

Selección aleatoria:

h1, realiza camino {A, D, B, C, A} (coste 60)

h2, realiza camino {A, D, C, B, A} (coste 85)



Se vuelven a recorrer estos tres tramos, aumentando la cantidad de feromonas

El proceso continúa, tratando de encontrar el camino óptimo (coste 55). Ej: {A, C, D, B, A} o {A, B, D, C, A} (son inversos)

Algoritmo general ACO

- Inicialización Matriz de Visibilidad, Matriz de Feromonas y Matriz de Probabilidades

- Repetir hasta fin

Construcción Soluciones:

Para cada paso i ($i=1, N$) {
 Para cada Hormiga- h ($h=1, \text{Hormigas}$)

 { Hormiga- h selecciona siguiente arco en su senda:

 Una hormiga irá de $i \rightarrow j$, con probabilidad $p_{i,j} = \frac{(\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}{\sum (\tau_{i,j}^\alpha)(\eta_{i,j}^\beta)}$

 Donde,

τ_{ij} es la feromona de arco $i \rightarrow j$, con una influencia α

η_{ij} es la visibilidad del arco, con influencia β }

}

Dependiendo del problema, se pueden intercambiar los bucles (si el tamaño de la solución no es necesariamente fijo y puede depender de la hormiga)

Actualización Matriz de Feromonas:

La feromona de cada arco $i \rightarrow j$ (τ_{ij}) se actualiza de la forma: $\tau_{ij} = (1-\rho) \tau'_{ij} + \Delta\tau_{ij}$

Donde, ρ Es la tasa de evaporación, que se aplica a la feromona previa τ'_{ij}

$\Delta\tau_{ij}$ Feromona depositada por todas las hormigas en los arcos que recorren:

$$\Delta\tau_{ij} = \sum_{h=1, H} \Delta\tau^h_{ij}$$

siendo $\Delta\tau^h_{ij}$ la feromona dejada por hormiga h en $i \rightarrow j$:

= $1/L_h$ si la hormiga h atraviesa el arco, y ha realizado una senda de coste L_h

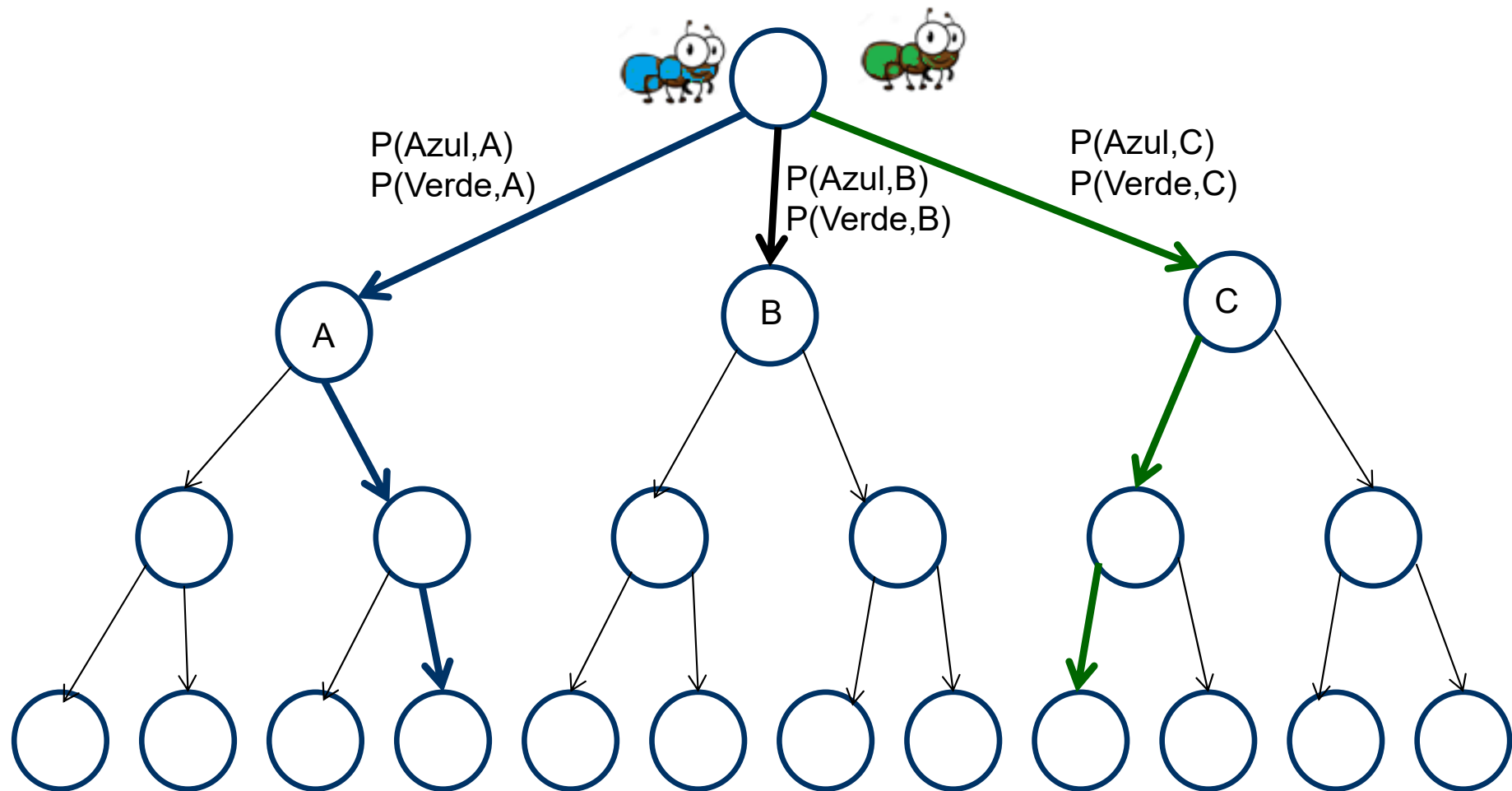
= 0 en otro caso

Actualización Matriz de Probabilidades

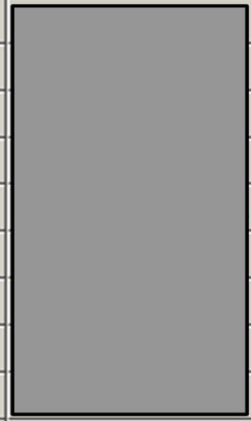
En general...

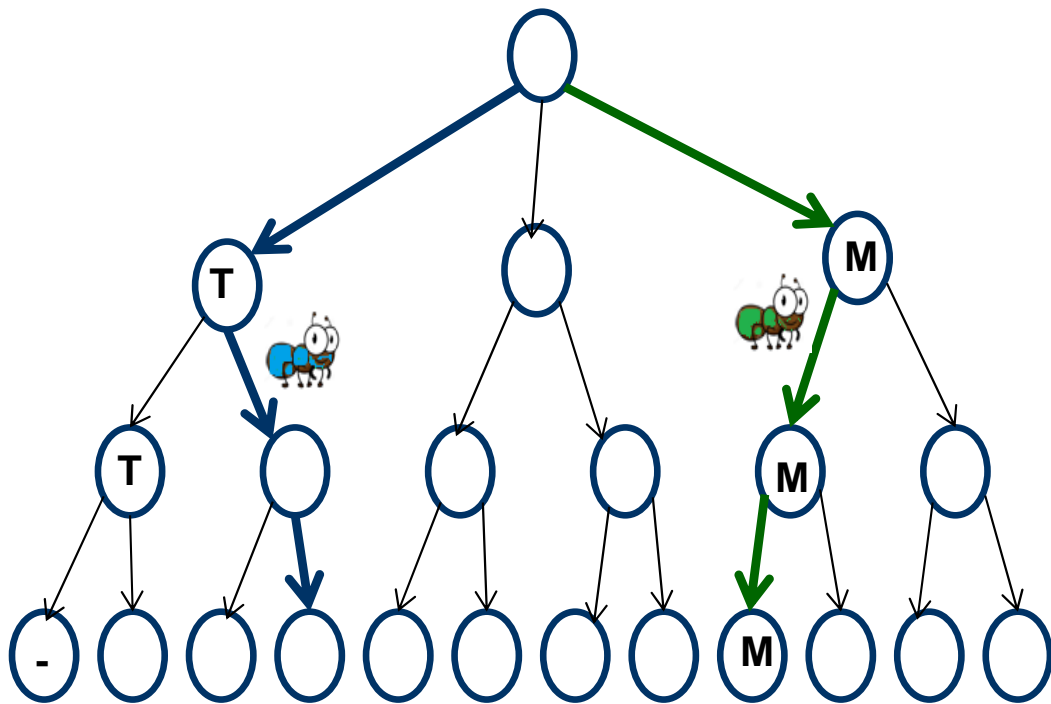
Los problemas que se pueden resolver con los algoritmos ACO son los que se pueden representar como un grafo ponderado $G = (N,A)$, donde A es el conjunto de aristas que conectan los nodos N (espacio de búsqueda con arcos de peso no uniforme)

Cada hormiga debe tomar una decisión aleatoria desde cada nodo



Otros ejemplos. Distribución y asignación de horarios

JUNIO																																		
Cod.	Nombre	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	1	2	
		L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J	
 1		N	N	-	-	-	-	-	T	T	-	-	M	M	M	M	-	-	-	-	-	-	N	N	N	N	-	-	-	-	T	T		
 2		-	M	M	M	M	-	-	-	-	-	-	N	N	N	N	-	-	-	-	-	T	T	T	T	-	-	N	N	N	N	V	V	
 3		M	M	M	M	M	-	M	M	M	M	M	M	M	-	-	-	-	-	M	M	M	M	M	-	-	M	M	M	M	M	M	V	V
 4		M	M	M	M	M	-	-	-	N	N	-	-	-	T	T	T	T	T	T	-	-	-	-	N	N	N	N	N	N	-	-	-	M
 5		-	-	-	-	-	-	-	T	T	T	T	T	-	-	-	-	-	-	M	M	M	M	M	-	-	-	M	M	M	M	M	-	
 6		N	N	-	-	-	-	-	T	T	T	T	T	-	M	N	N	N	N	-	-	-	-	-	M	M	M	M	M	-	-	T	T	
 7		-	-	-	-	M	N	N	N	-	-	-	M	M	M	M	M	M	-	-	-	T	T	T	T	-	-	-	-	T	T	V	V	
 8		-	-	-	-	M	N	N	N	N	-	-	T	T	T	T	T	T	-	-	N	N	N	N	-	-	-	-	-	T	T	T	T	
 9		-	-	-	-	-	-	-	T	T	-	-	M	M	M	M	M	M	-	-	N	N	N	N	-	-	-	-	-	M	M	M	M	



An ant colony based timetabling tool

Thatchai Thepphakorn ^a, Pupong Pongcharoen ^{a,*}, Chris Hicks ^b

Int. J. Production Economics 149 (2014) 131–144

Using Ant Colony Optimization to Solve Train Timetabling Problem of Mass Rapid Transit

Jau-Ming Su¹ Jen-Yu Huang²

¹Department of Transportation Technology and Logistic Management, Chung-Hua University

²Division of Traffic Engineering Institute of Civil Engineer, National Taiwan University



International Journal of Production Economics

Volume 149, March 2014, Pages 131-144

Ant Colony System-based Applications to Electrical Distribution System Optimization

Gianfranco Chicco
Politecnico di Torino
Italy

An ant colony based timetabling tool

Thatchai Thepphakorn^a, Pupong Pongcharoen^a  , Chris Hicks^b

 **Show more**

<https://doi.org/10.1016/j.ijpe.2013.04.026>

Get ri



International Journal of Innovative Research in Computer and Communication Engineering

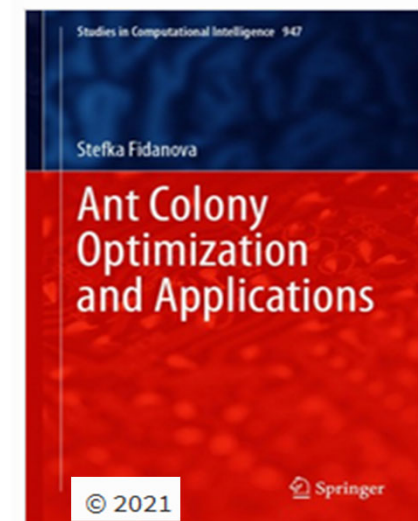
(An ISO 3297: 2007 Certified Organization)

Website: www.ijircce.com

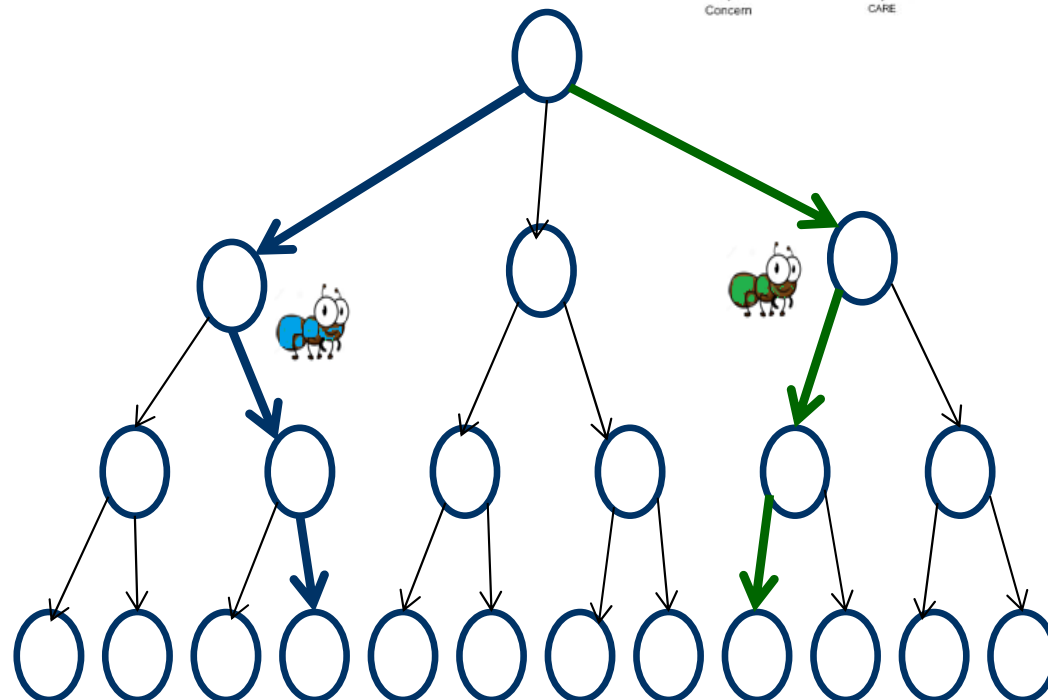
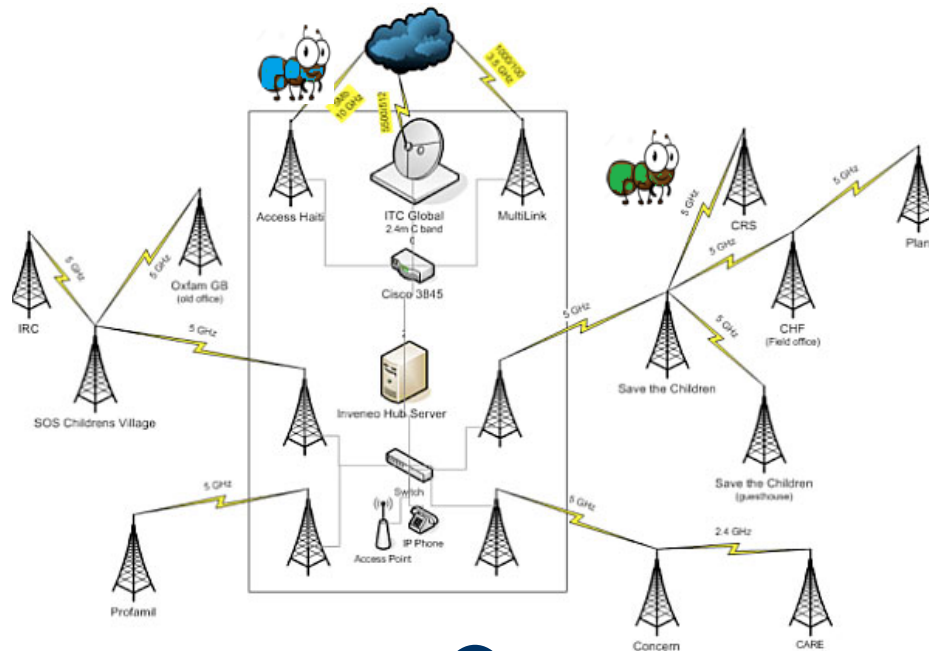
Vol. 5, Issue 3, March 2017

Timetable Generation Using Ant Colony Optimization Algorithm

Priyanka Gore¹, Poonam Sonawane¹, Suneha Potdar²



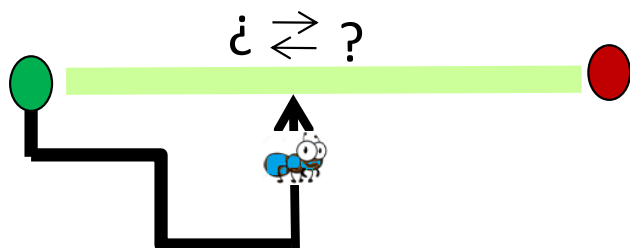
Otros ejemplos. Asignación/reparto de recursos



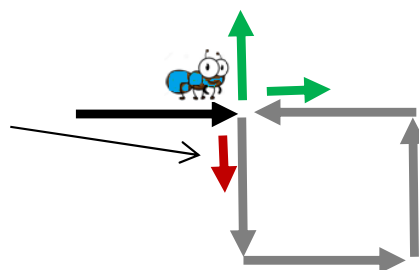
An ANTS heuristic for the frequency assignment problem. Maniezzo V, Carbonaro A. Future Gener Comput Syst 16(8): 927–935.

Otros ejemplos. Recorridos en laberintos

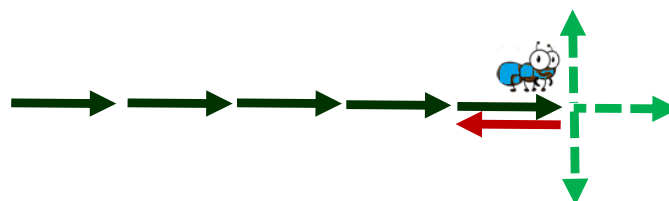
Información de feromonas: valor y dirección



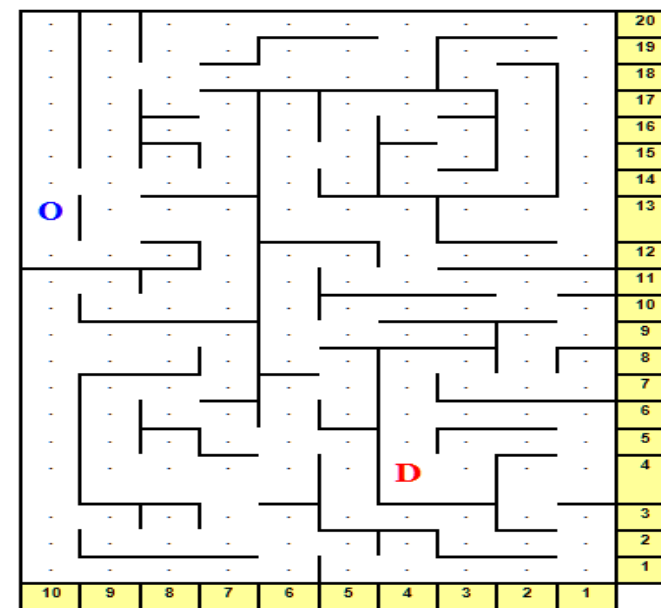
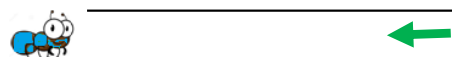
Ciclos: Olvidar ciclo, penalizar selección



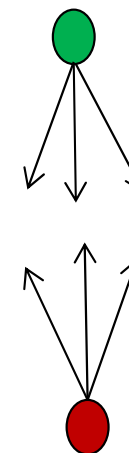
Penalizar vuelta atrás



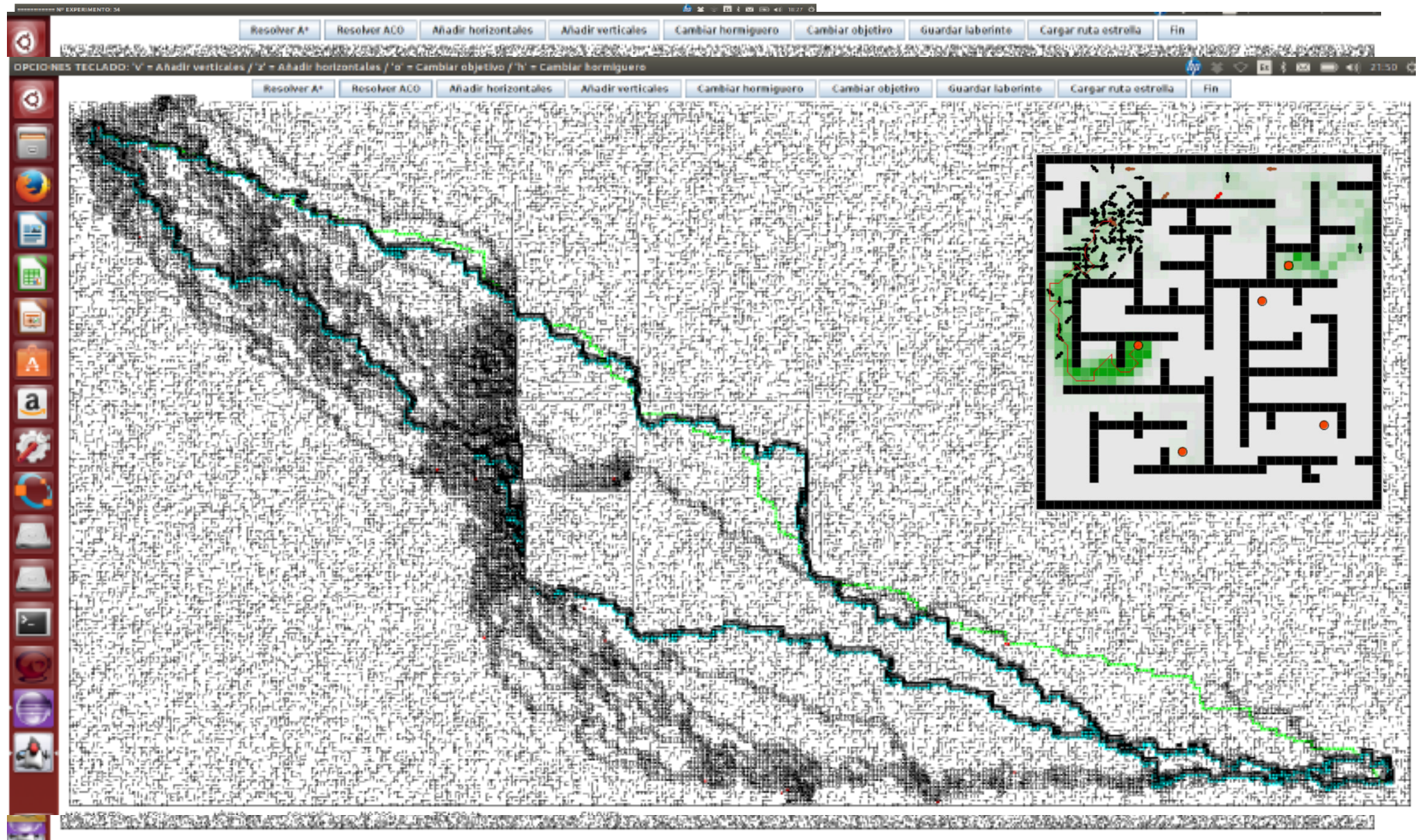
Bloqueo del camino: Permitir vuelta atrás



Búsqueda Bidireccional



Otros ejemplos. Recorridos en laberintos



Dependencia de los parámetros

↑ Feromona inicial ⇒ ↑ Exploración

Si se inicializan las feromonas a valores muy bajos, la búsqueda estará muy influenciada por los primeros recorridos de las hormigas: explotación.

Si se inicializan las feromonas a valores muy altos (todos iguales) deberán pasar muchas iteraciones hasta que la evaporación reduzca los suficiente los valores de feromona para que las feromonas introducidas por las hormigas empiecen a influir en la búsqueda: exploración

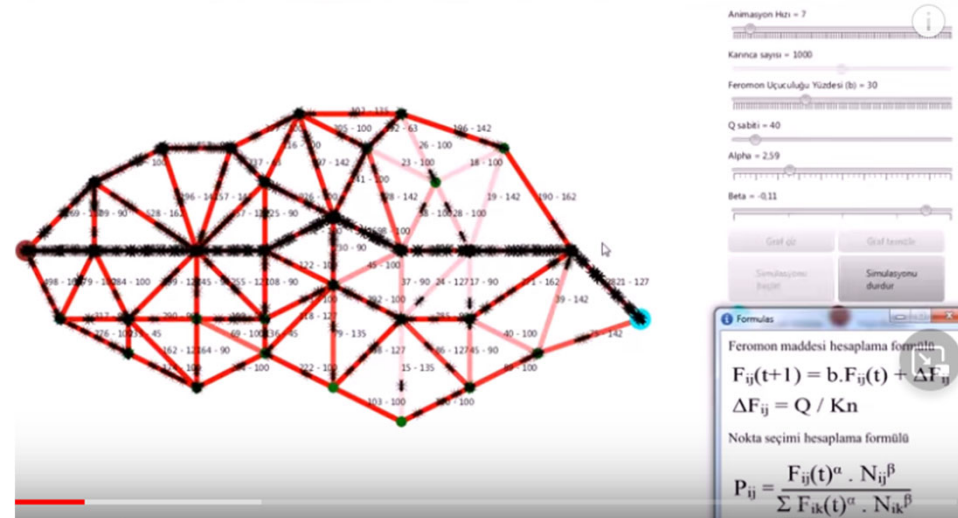
↑ Feromona depositada por hormigas ⇒ ↑ Explotación

↑H: nº de Hormigas

↑ ρ (evaporación): menos realimentación entre hormigas

α : influencia feromona (τ : retroalimentación)

β : influencia visibilidad (η : heurística del problema)



Variaciones Algoritmo de las Hormigas

Sistema Elitista

En el método original, solo las hormigas depositan feromonas ($1/L_h$). Ahora, en cada iteración, la mejor solución global también deposita feromonas (como si fuera una hormiga ficticia).

Sistema basado en el rango

Las soluciones son ordenadas de acuerdo a su calidad (longitud).

La cantidad de feromona depositada por cada hormiga depende del rango de su solución, en vez de la longitud de su senda.

Sistema de Colonias

Todas las hormigas dejan la misma feromona en los arcos que recorren, independientemente de la bondad de la solución.

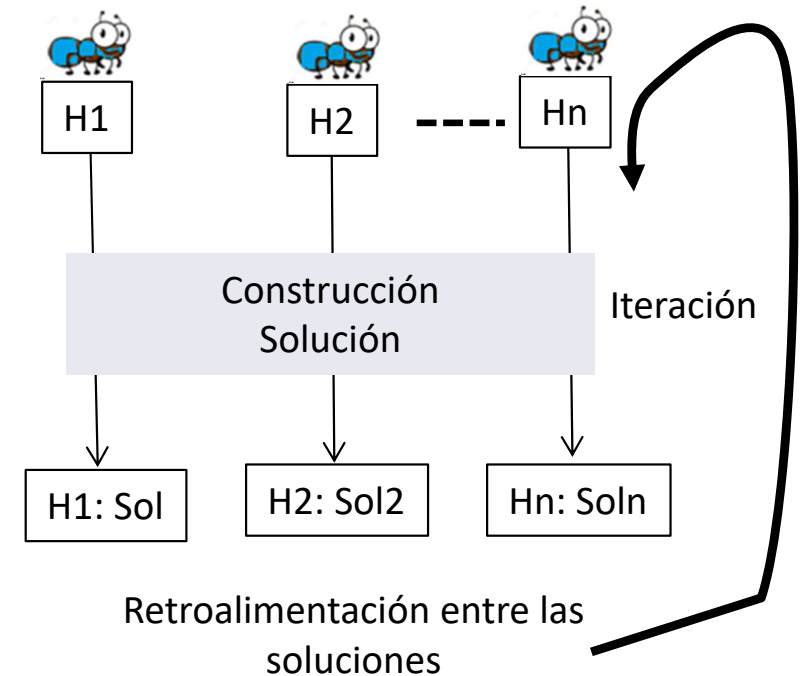
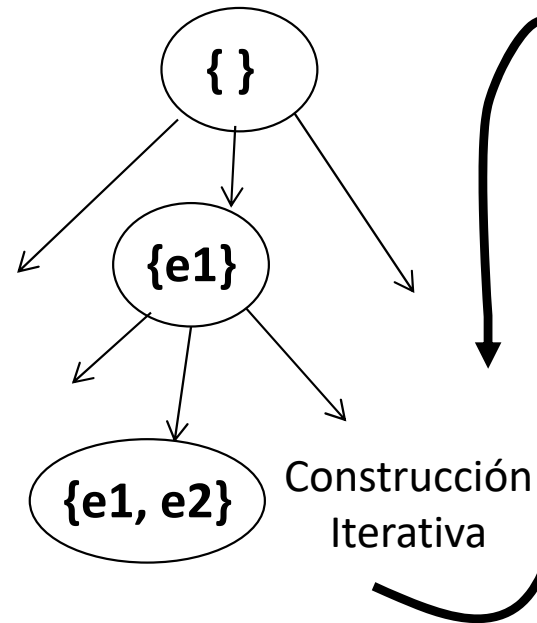
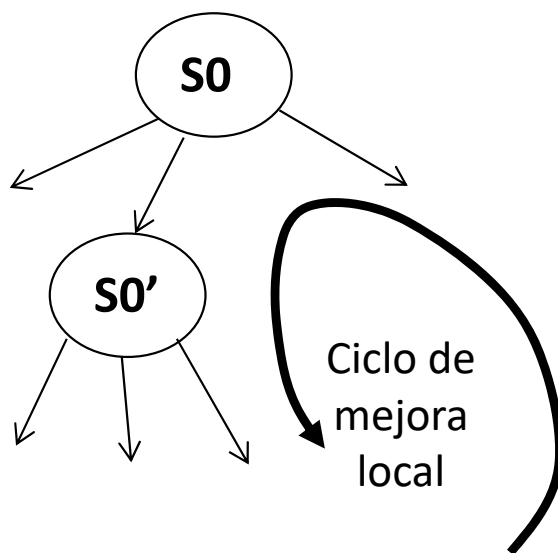
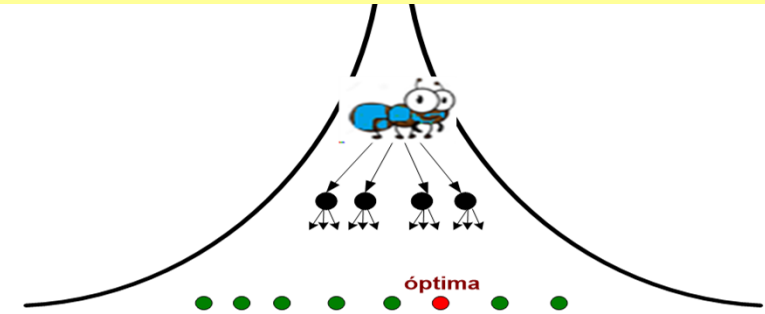
Pero ahora las hormigas no están sincronizadas y las que acaban antes (por haber encontrado mejor camino) vuelven a comenzar una nueva búsqueda. Así, los mejores caminos tendrán más probabilidad de ser repetidos. *Es la típica forma en la naturaleza.*

Acciones daemon

Cuando una hormiga encuentra una solución, se añade un proceso de mejora (típicamente búsqueda local) para tratar de optimizar esa solución.

Conclusiones y comparativa

Algoritmo Hormigas *versus* Metaheurísticas Mejora y Constructivas



Búsqueda Local en la vecindad

- Enfriamiento Simulado, Tabú, Búsqueda en Haz
- Función de Evaluación $f(n)$
- No hay realimentación entre soluciones, salvo la propia solución actual

Constructivo (GRASP)

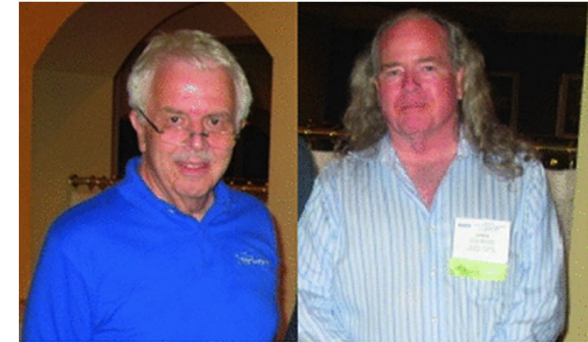
- Construcción de Solución (Múltiples soluciones independientes)
- Mejora Local posterior
- Función heurística $f(n)$

Algoritmo de las Hormigas

- Construcción de las Soluciones
- Paso a paso, por múltiples hormigas
- Retro-alimentación entre las soluciones obtenidas

Enjambre de Partículas

(PSO, Particle Swarm Optimization)



Russel Eberhart and James Kennedy

Kennedy & Eberhart, 1995



Particle swarm optimization. *James Kennedy and Russell Eberhart.*

Proceedings of IEEE International Conference on Neural Networks, pg 1942–1948, 1995. IEEE Press.

<http://www.swarmintelligence.org/>

<http://www.particleswarm.info/>

Bibliografía (Artículos y vídeos en Poliformat)



Contents lists available at [ScienceDirect](#)

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Short communication

Salp Swarm Optimization: A critical review

Chapter 11 Different Applications of PSO

Applied Intelligence (2019) 49:335–351
<https://doi.org/10.1007/s10489-018-1258-3>

Swarm Optimization. A critical review (2022)
State of the art review of intelligent scheduling (2020)
Bee-inspired metaheuristics (2021)
Comparison of 5 novel nature inspired metaheuristic (2020)
Different Applications of PSO (2021)
From ants to whales, metaheuristics for all tastes (2020)
LIBRO Ant Colony Optimization and Applications (2021)
Metaheuristic Review 2019
Multi-objective particle swarm optimization (2020)
Particle Swarm Optimisation A Historical Review (2020)
Particle swarm optimization (2021)
Particle Swarm Optimization A Comprehensive Survey
Particle Swarm Optimization Algorithms (2017)
Application of particle swarm in power systems (2017)
PSO para muchos objetivos
PSO Viajante Multiobjetivo
PSO for optimizing high-dimensional problems (2019)

Enhanced particle swarm optimization with multi-swarm
and multi-velocity for optimizing high-dimensional problems

Particle Swarm Optimization: A Comprehensive Survey

Metaheurística bioinspirada, basada en la interacción social de las bandadas de pájaros, bancos de peces, etc.: Enjambres de Partículas

- En un grupo (enjambre) se establecen diferentes jerarquías, de acuerdo a las características de cada individuo.
- Particularmente, existe un líder, reconocido y seguido por los demás individuos del grupo.
 - El líder tiene habilidades que le permiten ser guía para buscar alimento.
 - Los individuos de menor jerarquía confían en la capacidad del líder para dirigirlos.

En el proceso de búsqueda de alimento (búsqueda solución):

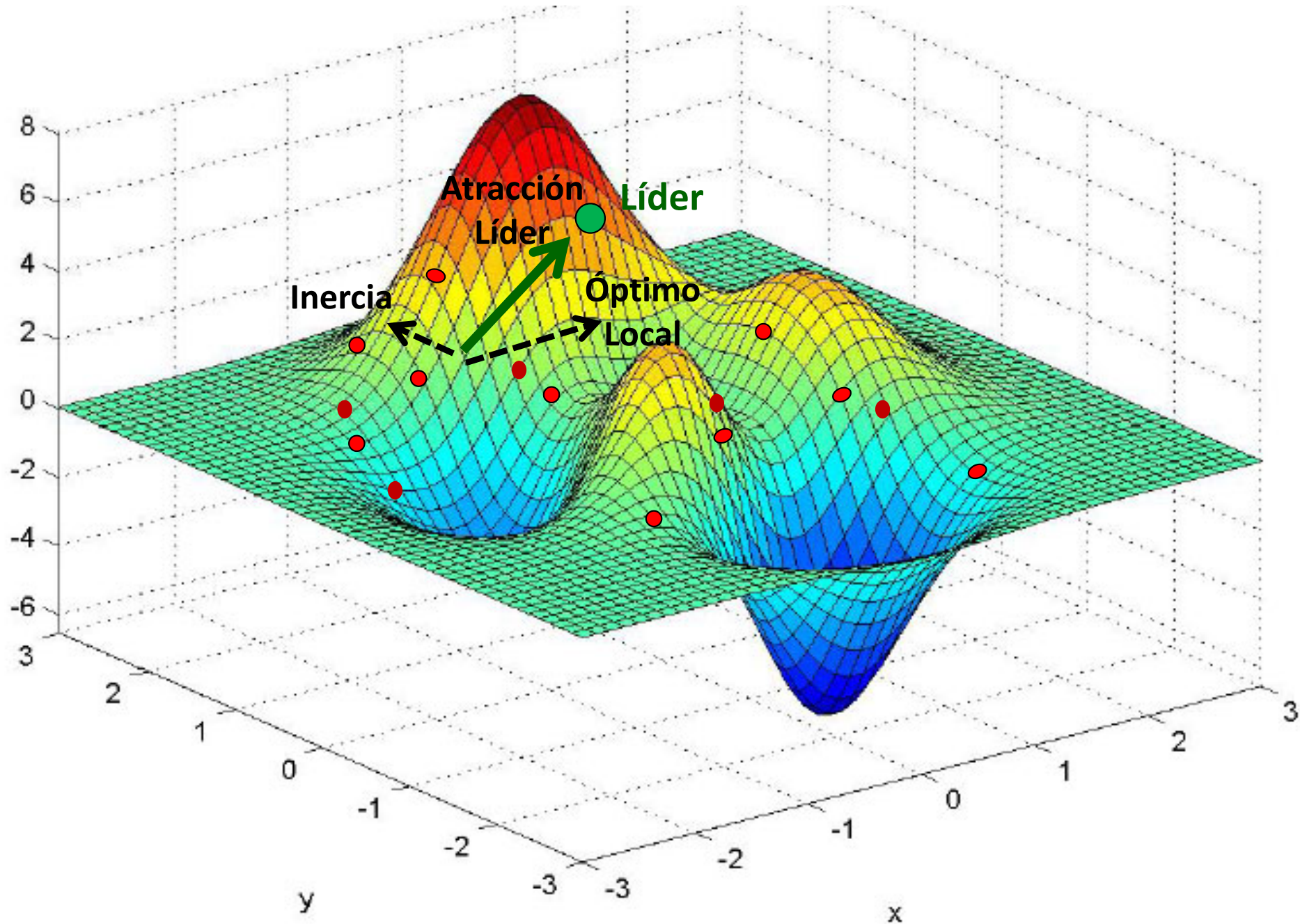
- El líder guía al grupo por donde sospecha encontrarán alimento.
- Todos los individuos siguen al líder, pero tratando también a su vez de localizar alimento y evitar colisiones.
- Eventualmente, comunican al grupo nuevas ubicaciones de alimento (existe una comunicación entre el conjunto de partículas del enjambre).



El grupo no es estático. Puede surgir un individuo con mejores capacidades que el actual, sustituyéndolo en el mando del grupo: nuevo líder.

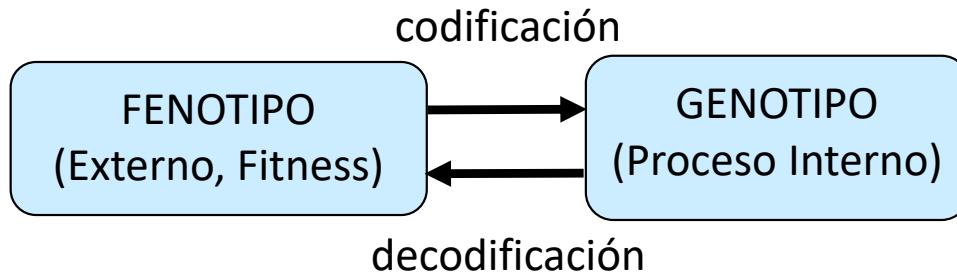
- El proceso de **selección** se basa en comparar el valor de aptitud (valor de la función objetivo) de cada miembro del grupo con el valor de aptitud del actual líder: el miembro del grupo con mejor evaluación será el nuevo líder si supera al líder actual.

Idea Básica: cada **partícula** sobrevuela el espacio de soluciones **n-dimensional** en busca de soluciones óptimas



Modelado del problema y Proceso

Cada partícula (individuo) del enjambre (o nube de partículas) representa una **solución** y realiza un proceso de **búsqueda** de mejores soluciones.



- La búsqueda de cada partícula_i en el espacio de soluciones se realiza en base al “conocimiento propio”, a la influencia del líder y al movimiento de la partícula:
 - Solución propia: Pbest_i
 - Solución aportada por el líder: Gbest
 - Movimiento de la partícula: V_i
- Se plantean unas reglas matemáticas para el movimiento de cada partícula.
- El objetivo final es que el enjambre de partículas converja hacia las mejores soluciones

MODELADO

Partícula_i $\cong$ Solución_i
 $\Rightarrow$

Conjunto de decisiones (o valores)
sobre cada variable del problema:

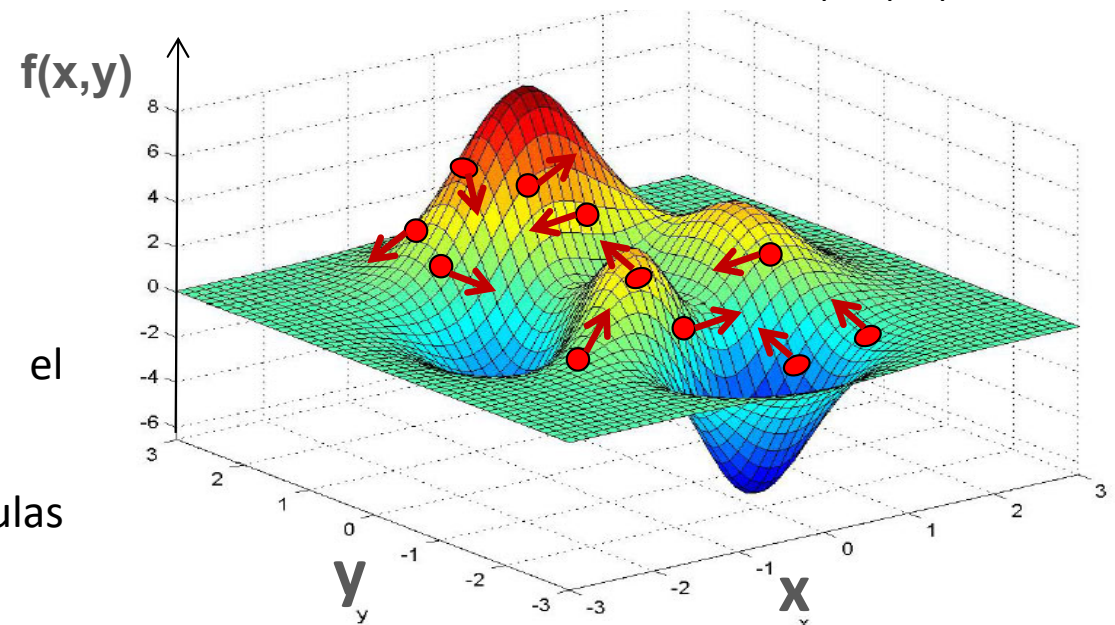
$$\text{Partícula}_i = [x_{i1}, x_{i2}, x_{i3}, \dots, x_{in}]$$

Típicamente vector de Reales o Enteros



Espacio de Búsqueda $\cong$
Espacio de Decisión n-dimensional

Ejemplo 2-dimensional: Partícula_i: (x_i, y_i)



Esquema Básico Algoritmo PSO

Crear enjambre inicial de partículas (soluciones en el espacio de búsqueda) y velocidades iniciales (típicamente de manera aleatoria).

Soluciones : { (Posición_i, Velocidad_i) }

Mientras no_fin (nº evaluaciones, optimalidad, etc.)

1. Evaluar cada partícula_i: Evaluación de Soluciones

- Pbest_i: Mejor solución encontrada por la partícula_i
- V_i: Velocidad de la partícula_i (incluye dirección)
- Gbest: Mejor solución encontrada por el enjambre hasta el momento (líder).

Búsqueda nuevas soluciones

2. Obtener función de vuelo de las partículas:

Obtener nueva velocidad y dirección de cada partícula (V'_i) en base a posición del líder (Gbest) y al propio proceso de búsqueda local (V_i, Pbest_i).

3. Actualizar la posición de cada partícula (nueva solución).

Aplicar función de vuelo a la posición actual y obtener nueva posición ($X'_i = X_i \otimes V'_i$)

Esta función depende del dominio; ej. $X'_i = X_i + \alpha V'_i$

Para cada partícula_i:

Posición_i: Codificación de una solución S_i
Genotipo (n-dimensional)

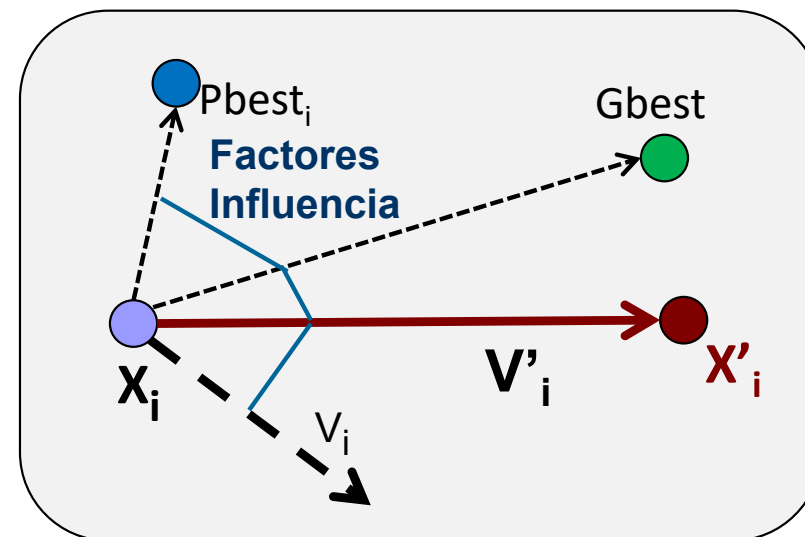
[X_{i1}, X_{i2}, X_{i3}, ... X_{in}]

Velocidad_i: Velocidad de la partícula,
hacia una nueva solución

$S_i \rightarrow S'_i$

Pbest_i: Mejor solución de la partícula_i
[p_{i1}, p_{i2}, ... p_{in}]

Gbest: Mejor solución del enjambre



Cada partícula explora el espacio de soluciones a partir de su mejor posición y de la mejor posición global

Ecuación del Movimiento: V_i

Cada Partícula $_i$ se caracteriza por (espacio n -dimensional):

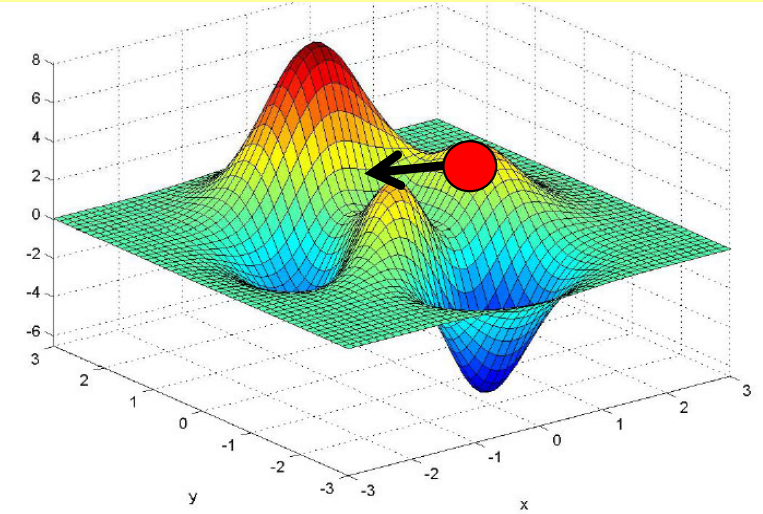
Posición $\mathbf{X}_i = [x_{i1}, x_{i2}, \dots, x_{in}]$

Velocidad $\mathbf{V}_i = [v_{i1}, v_{i2}, \dots, v_{in}]$

Mejor posición personal (en su evolución): $\mathbf{P}_i = [p_{i1}, p_{i2}, \dots, p_{in}]$

Además, la mejor posición global alcanzada por todas las partículas:

$\mathbf{Gbest} = [g_1, g_2, \dots, g_n]$



En cada iteración del proceso de búsqueda, cada Partícula $_i$ actualiza su velocidad de vuelo $\mathbf{V}'_i$, donde cada k -dimensión $[v'_{i1}, v'_{i2}, v'_{ik}, \dots, v'_{in}]$ se obtiene como:

$$v'_{ik} = \mathbf{w} * v_{ik}$$

$$+ c_1 (p_{ik} - x_{ik}) * \text{random}(0, 1)$$

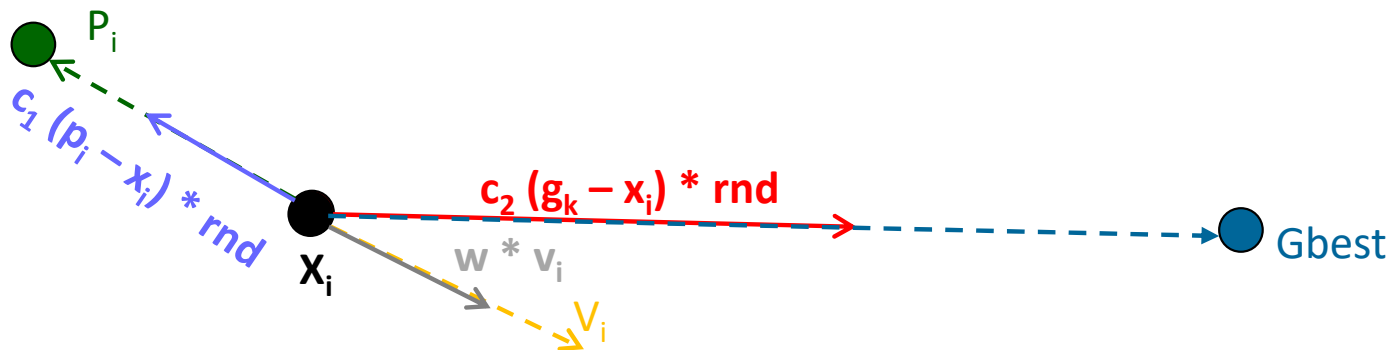
$$+ c_2 (g_k - x_{ik}) * \text{random}(0, 1)$$

$w \in [0, 1]$: Inercia o impacto de la velocidad previa V_i

c_1 : Parámetro cognitivo (experiencia individual)

c_2 : Parámetro social (influencia enjambre, cooperación)

En muchos casos, $w + c_1 + c_2 = 1$



Ecuación del Movimiento: V_i

Cada Partícula $_i$ se caracteriza por (espacio n -dimensional):

Posición $\mathbf{X}_i = [x_{i1}, x_{i2}, \dots, x_{in}]$

Velocidad $\mathbf{V}_i = [v_{i1}, v_{i2}, \dots, v_{in}]$

Mejor posición personal (en su evolución): $\mathbf{P}_i = [p_{i1}, p_{i2}, \dots, p_{in}]$

Además, la mejor posición global alcanzada por todas las partículas:

$\mathbf{Gbest} = [g_1, g_2, \dots, g_n]$

En cada iteración del proceso de búsqueda, cada Partícula $_i$ actualiza su velocidad de vuelo $\mathbf{V}'_i$, donde cada k -dimensión $[v'_{i1}, v'_{i2}, v'_{ik}, \dots, v'_{in}]$ se obtiene como:

$$\mathbf{V}'_{ik} = \mathbf{W} * \mathbf{V}_{ik}$$

$$+ \mathbf{c}_1 (\mathbf{p}_{ik} - \mathbf{x}_{ik}) * \text{random}(0, 1)$$

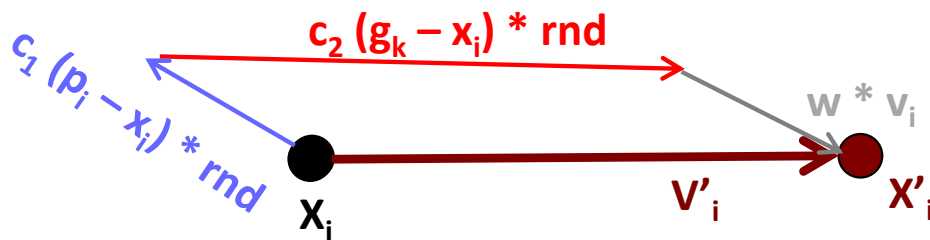
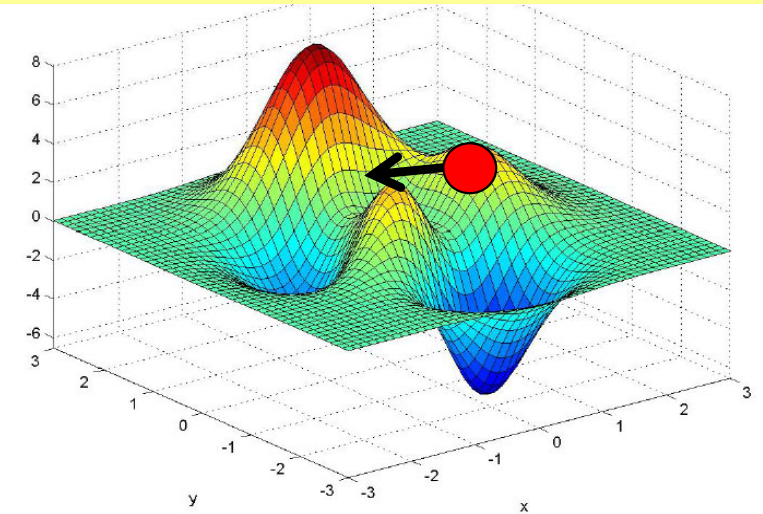
$$+ \mathbf{c}_2 (\mathbf{g}_k - \mathbf{x}_{ik}) * \text{random}(0, 1)$$

$w \in [0, 1]$: Inercia o impacto de la velocidad previa $\mathbf{V}_i$

c_1 : Parámetro cognitivo (experiencia individual)

c_2 : Parámetro social (influencia enjambre, cooperación)

En muchos casos, $w + c_1 + c_2 = 1$

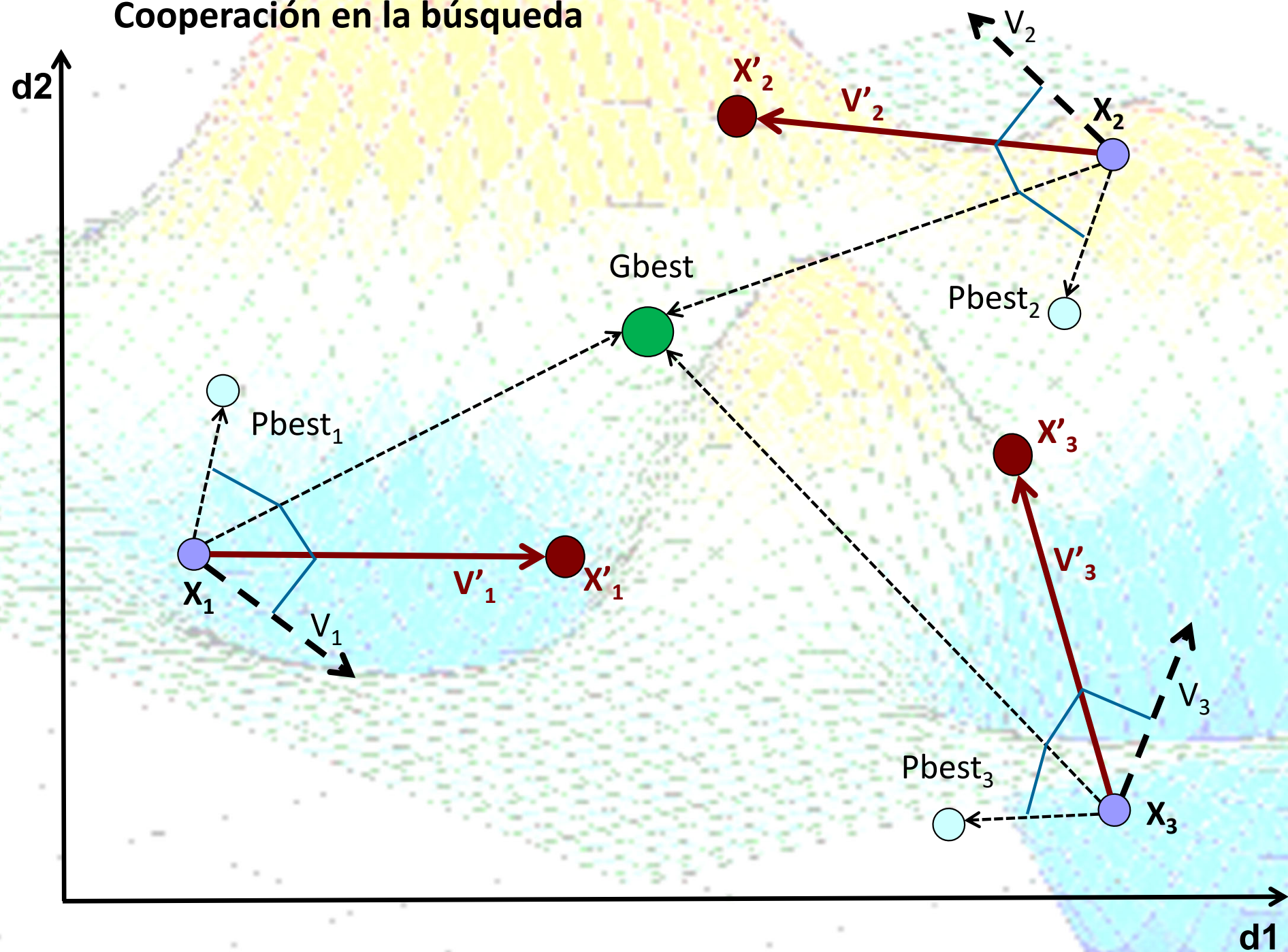


La nueva velocidad representa también la dirección, por lo que se obtiene la nueva posición $\mathbf{x}'_i$.

Suma de 3 vectores:



Cooperación en la búsqueda



Algoritmo PSO Básico. Minimización

Algorithm 1: classical PSO

input : Objective function $f : S \rightarrow \mathbb{R}$ to be minimized

output: $G \in \mathbb{R}^D$

Inicialización

// Initialization

1 **for** $n = 1 \rightarrow N$ **do** Para cada partícula

2 Initialize position $X^n \in \mathbb{R}^D$ randomly; Posición y Velocidad inicial

3 Initialize velocity $V^n \in \mathbb{R}^D$;

4 Initialize local attractor $L^n := X^n$; Mejor Posición Local inicial

5 Initialize $G := \operatorname{argmin}_{\{L^1, \dots, L^N\}} f$; Mejor Posición Global inicial (entre todas las partículas)

// Movement

Bucle de optimización

6 **repeat**

7 **for** $n = 1 \rightarrow N$ **do** Para cada partícula

8 **for** $d = 1 \rightarrow D$ **do** Para cada dimensión

9 $V^{n,d} := \chi \cdot V^{n,d} + c_1 \cdot \operatorname{rand}() \cdot (L^{n,d} - X^{n,d}) + c_2 \cdot \operatorname{rand}() \cdot (G^d - X^{n,d})$; Actualiza Velocidad

10 $X^{n,d} := X^{n,d} + V^{n,d}$; Actualiza Posición

11 **if** $f(X^n) \leq f(L^n)$ **then** $L^n := X^n$; Actualiza Óptimo Local

12 **if** $f(X^n) \leq f(G)$ **then** $G := X^n$; Actualiza Óptimo Global

13 **until** Termination criterion holds;

14 **return** G ;

Parámetros PSO

Como en toda metaheurística, es importante el ajuste de los parámetros (diferentes variantes propuestas en PSO)

nº partículas (tamaño del enjambre)

Dependiente de la irregularidad de la hiper-superficie del fitness. Cuanto más irregular, serán necesarias más partículas

$$\mathbf{v}'_{ik} = \mathbf{w} * \mathbf{v}_{ik} + \mathbf{c}_1 (\mathbf{p}_{ik} - \mathbf{x}_{ik}) * \text{RND}(0, 1) + \mathbf{c}_2 (\mathbf{g}_k - \mathbf{x}_{ik}) * \text{RND}(0, 1)$$

w (inercia)

- Factor de **exploración** en búsqueda local
- Usualmente tiende a incrementarse en la búsqueda

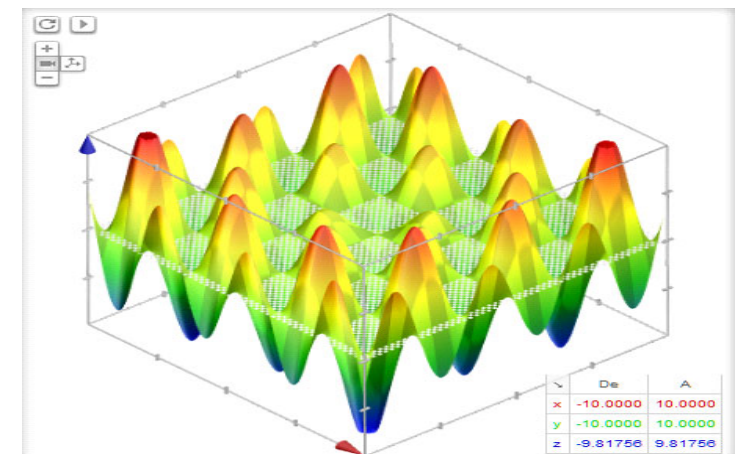
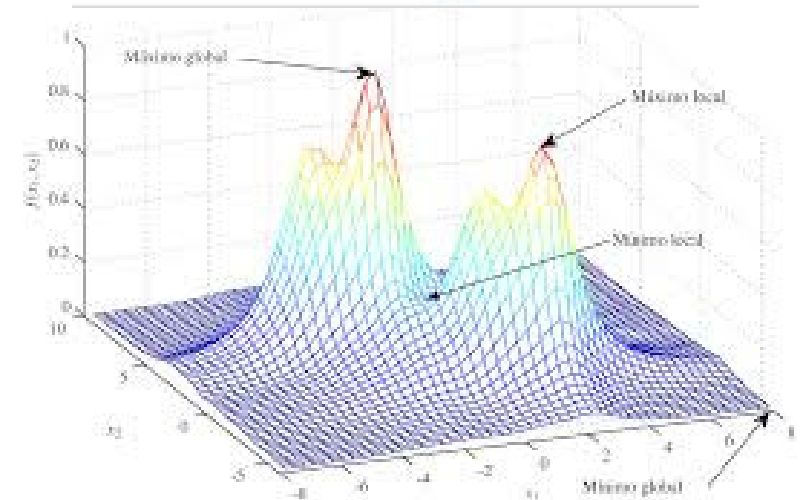
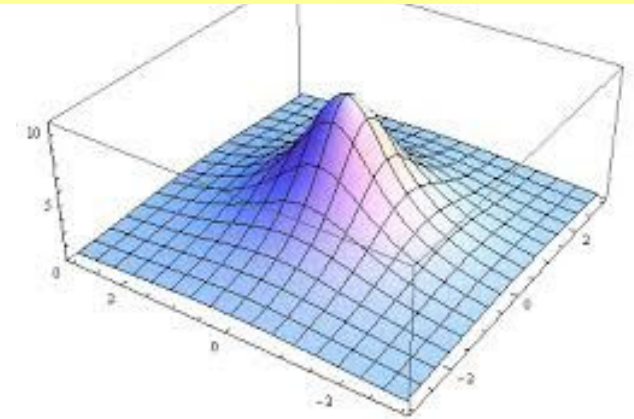
c₁: factor cognitivo personal (memoria)

- Controla el movimiento hacia su propio óptimo (explotación local de cada partícula)

c₂: factor cognitivo social (cooperación)

- Factor de explotación global. Controla el movimiento hacia el mejor óptimo conseguido por el enjambre
- Un valor alto puede producir caer en óptimos locales de los que no se puede salir

RND(0, 1): le da más peso al factor personal o social



Ejemplo 1. Una dimensión

Variable: $x \in [-1, 8]$

Objetivo: maximizar $f(x)$ según figura

Parámetros:

Nº partículas: 2

$w=0.9$ (muchacha inercia para exploración)

$\alpha=0.2$

$c_1=1$ (explotación local)

$c_2=1$ (explotación global)

Función de vuelo V_{i+1} y actualización de la posición X_{i+1} :

$$V(i+1) = w * V(i) + c_1(\underline{Pbest_i} - X(i)) + c_2(\underline{Gbest} - X(i))$$

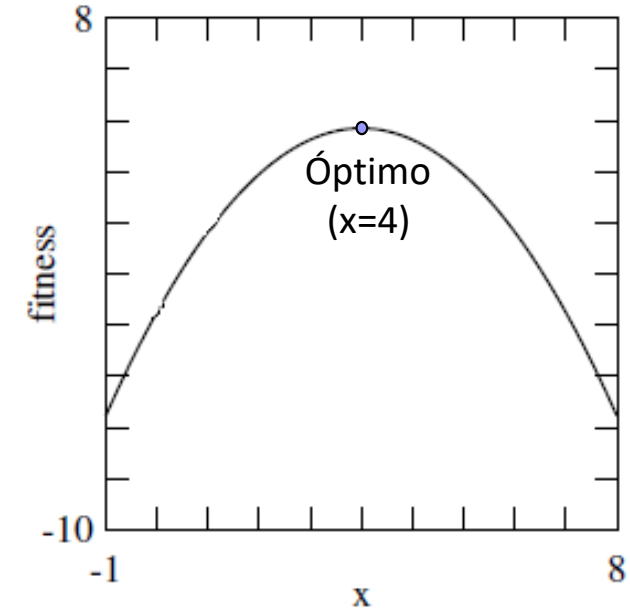
$$\underline{X(i+1)} = X(i) + \alpha * V(i+1)$$

Donde:

$Pbest_i$ = mejor solución local para partícula- i hasta el momento

$Gbest$ = mejor solución global hasta el momento (líder del enjambre)

Nota: para todos los cálculos, $RND(0, 1) = 1$



An Introduction to Metaheuristics for
Optimization, Chopard, Tomassini,
Springer (2018)

Ejemplo 1. Una dimensión

Iteración 0 (t=0). Inicialización al azar

Partícula₁:

$V_1(0) = 0$ // parado

$X_1(0) = 0$

$Pbest_1 = 0$

Partícula₂:

$V_2(0) = 0$ // parado

$X_2(0) = 1$

$Pbest_2 = 1$

$Gbest = 1$

Iteración 1 (t=1)

Partícula₁:

$pbest$

$gbest$

$V_1(1) = 0.9 * 0 + (0 - 0) + (1 - 0) = 1$

$X_1(1) = X_1(0) + 0.2 V_1(1) = 0 + 0.2 * 1 = 0.2$

$Pbest_1 = 0.2$

✗

Partícula₂:

$V_2(1) = 0.9 * 0 + (1 - 1) + (1 - 1) = 0$

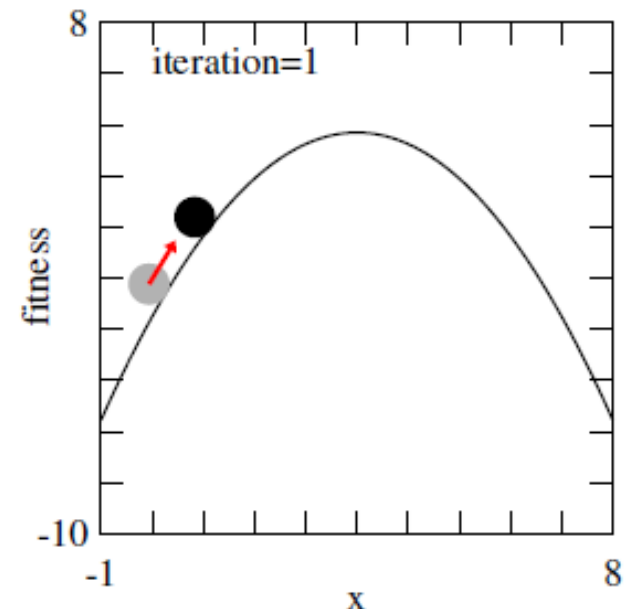
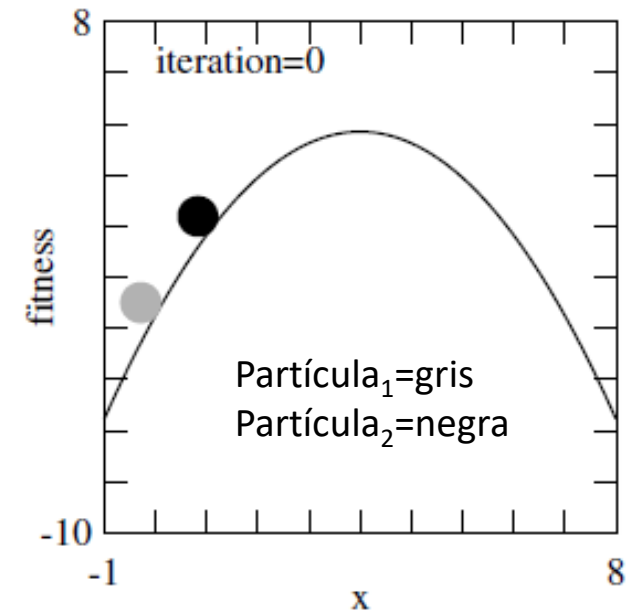
$X_2(1) = X_2(0) + 0.2 V_2(1) = 1 + 0.2 * 0 = 1$

$Pbest_2 = 1$

$Gbest = 1$

Ejemplo que ilustra las partículas moviéndose sobre la curva de la función de evaluación.

Realmente solo se mueven en el eje-X



Ejemplo 1. Una dimensión

Iteración 2 (t=2)

Partícula₁:

$$V_1(2) = 0.9 * 1 + (0.2 - 0.2) + (1 - 0.2) = 0.9 + 0.8 = 1.7$$

$$X_1(2) = X_1(1) + 0.2 V_1(2) = 0.2 + 0.2 * 1.7 = 0.54$$

$$Pbest_1 = 0.54$$

Partícula₂:

$$V_2(2) = 0.9 * 0 + (1 - 1) + (1 - 1) = 0$$

$$X_2(2) = X_2(1) + 0.2 V_2(2) = 1 + 0.2 * 0 = 1$$

$$Pbest_2 = 1$$

$$V(i+1) = w * V(i) + c_1(Pbest_i - X(i)) + c_2(Gbest - X(i))$$

$$X(i+1) = X(i) + 0.2V(i+1)$$

$$Gbest = 1$$

Iteración 3 (t=3)

Partícula₁:

$$V_1(3) = 0.9 * 1.7 + (0.54 - 0.54) + (1 - 0.54) = 1.07$$

$$X_1(3) = X_1(2) + 0.2 V_1(3) = 0.54 + 0.2 * 1.07 = 0.754$$

$$Pbest_1 = 0.754$$

Partícula₂:

$$V_2(3) = 0.9 * 0 + (1 - 1) + (1 - 1) = 0$$

$$X_2(3) = X_2(2) + 0.2 V_2(3) = 1 + 0.2 * 0 = 1$$

$$Pbest_2 = 1$$

$$Gbest = 1$$

Ejemplo 1. Una dimensión

Iteración 4 (t=4)

Partícula₁:

$$V_1(4) = 0.9 * 1.07 + (0.754 - 0.754) + (1 - 0.754) = 1.209$$

$$X_1(4) = X_1(3) + 0.2 V_1(4) = 0.754 + 0.2 * 1.209 = 0.9958$$

$$Pbest_1 = 0.9958$$

Partícula₂:

$$V_2(4) = 0.9 * 0 + (1 - 1) + (1 - 1) = 0$$

$$X_2(4) = X_2(3) + 0.2 V_2(4) = 1 + 0.2 * 0 = 1$$

$$Pbest_2 = 1$$

$$V(i+1) = w * V(i) + c_1(Pbest_i - X(i)) + c_2(Gbest - X(i))$$

$$X(i+1) = X(i) + 0.2V(i+1)$$

$$Gbest = 1$$

Iteración 5 (t=5)

Partícula₁:

$$V_1(5) = 0.9 * 1.209 + (0.9958 - 0.9958) + (1 - 0.9958) = 1.0923$$

$$X_1(5) = X_1(4) + 0.2 V_1(5) = 0.9958 + 0.2 * 1.0923 = 1.21426$$

$$Pbest_1 = 1.21426$$

Partícula₂:

$$V_2(5) = 0.9 * 0 + (1 - 1) + (1 - 1) = 0$$

$$X_2(5) = X_2(4) + 0.2 V_2(5) = 1 + 0.2 * 0 = 1$$

$$Pbest_2 = 1$$

$$Gbest = 1.21426 \quad // \text{ cambio de líder (el nuevo líder viene dado por Partícula}_1\text{)}$$

Ejemplo 1. Una dimensión

Iteración 6 (t=6)

Partícula₁:

$$V_1(6) = 0.9 * 1.0923 + (1.21426 - 1.21426) + (1.21426 - 1.21426) = 0.98307$$

$$X_1(6) = X_1(5) + 0.2 V_1(6) = 1.21426 + 0.2 * 0.98307 = 1.410874$$

$$Pbest_1 = 1.410874$$

Partícula₂:

$$V_2(6) = 0.9 * 0 + (1 - 1) + (1.21426 - 1) = 0.21426$$

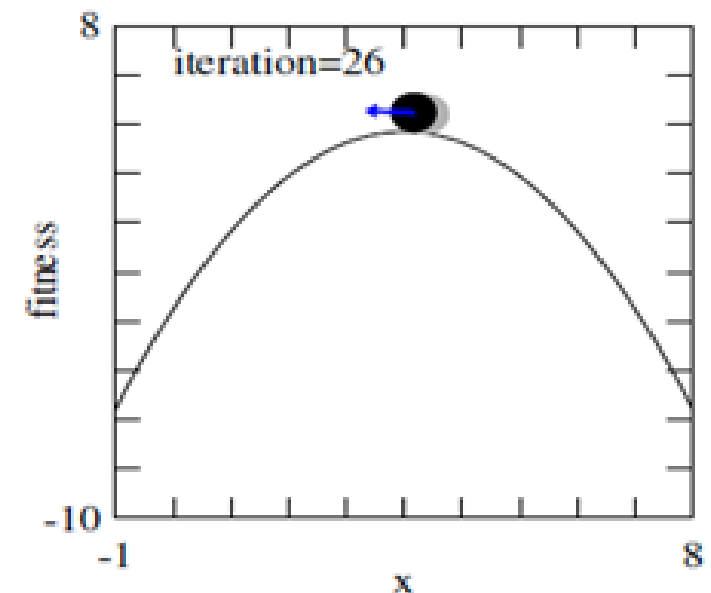
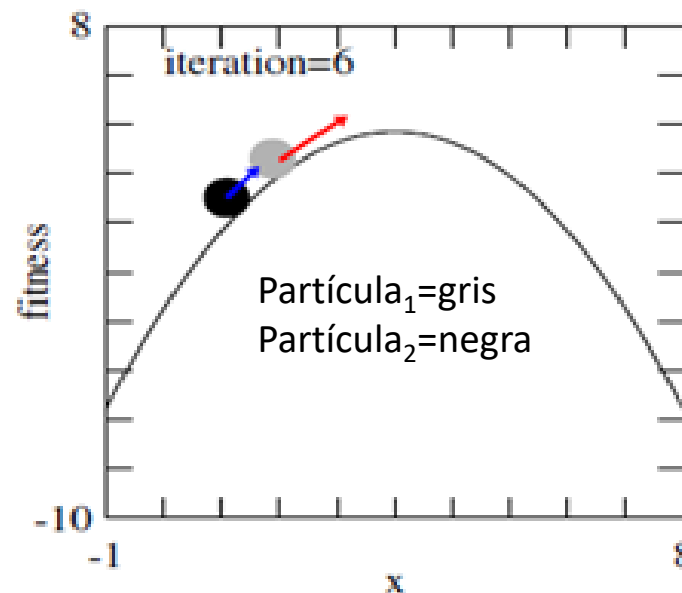
$$X_2(6) = X_2(5) + 0.2 V_2(6) = 1 + 0.2 * 0.21426 = 1.042852$$

$$Pbest_2 = 1.042852$$

$$V(i+1) = w * V(i) + c_1(Pbest_i - X(i)) + c_2(Gbest - X(i))$$
$$X(i+1) = X(i) + 0.2V(i+1)$$

$$Gbest = 1.410874$$

Debido a la inercia,
la Partícula₁
arrastrará a la
Partícula₂



Ejemplo 2. Dos dimensiones

Variable: $x_1, x_2 \in [-10, +10]$

Objetivo: maximizar $f(x,y) = 100 - (x^2 + y^2)$ según figura
(solución óptima 100)

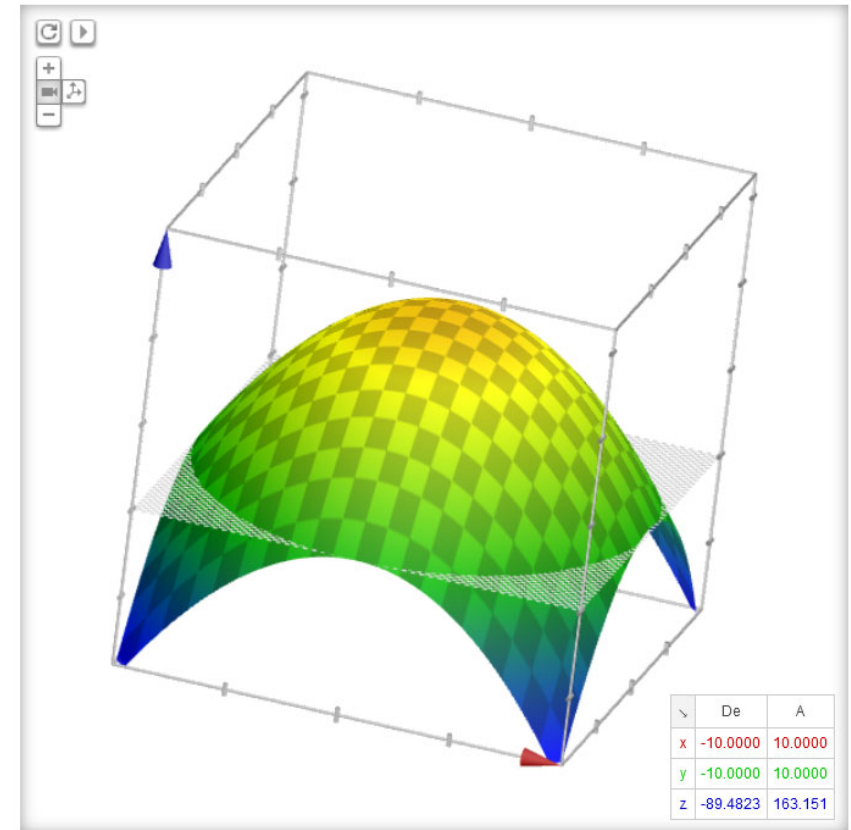
Parámetros:

Nº partículas: 2

$w = 0.9$

$c_1 = 1$

$c_2 = 1$



Función de vuelo V_{i+1} y actualización de la posición X_{i+1} :

$$v_i(t+1) = 0.9v_i(t) + [b_i(t) - x_i(t)] + [B(t) - x_i(t)]$$

$$x_i(t+1) = x_i(t) + 0.2v_i(t+1)$$

Donde:

$b_i(t)$ = mejor solución local para partícula-i encontrada hasta tiempo t

$B(t)$ = mejor solución global (líder del enjambre) encontrada hasta tiempo t

Nota: para todos los cálculos, $RND(0, 1) = 1$

Ejemplo 2. Dos dimensiones

$$v_i(t+1) = 0.9v_i(t) + [b_i(t) - x_i(t)] + [B(t) - x_i(t)]$$
$$x_i(t+1) = x_i(t) + 0.2v_i(t+1)$$

Iteración 0 (t=0)

Partícula₁:

$$V_1(0) = (0,0)$$

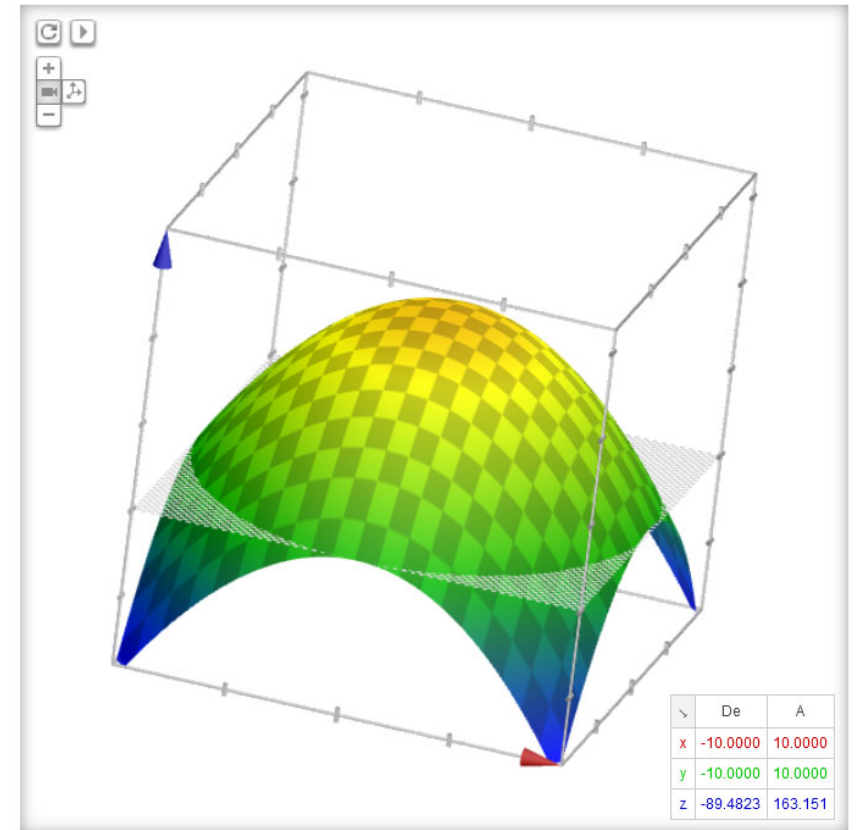
$$X_1(0) = (6,4), b_1(0) = 100 - (6^2 + 4^2) = 48$$

Partícula₂:

$$V_2(0) = (0,0)$$

$$X_2(0) = (3,8), b_2(0) = 100 - (3^2 + 8^2) = 27$$

$B(0) = (6,4)$ mejor posición global (calidad=48)



Ejemplo 2. Dos dimensiones

$$v_i(t+1) = 0.9v_i(t) + [b_i(t) - x_i(t)] + [B(t) - x_i(t)]$$
$$x_i(t+1) = x_i(t) + 0.2v_i(t+1)$$

Iteración 1 (t=1)

Partícula₁:

$$V_1(1) = 0.9 * (0,0) + [((6,4) - (6,4)) + ((6,4) - (6,4))] = (0,0)$$

$$X_1(1) = (6,4) + 0.2 * (0,0) = (6,4)$$

$$b_1(1) = 100 - (6^2 + 4^2) = 48$$

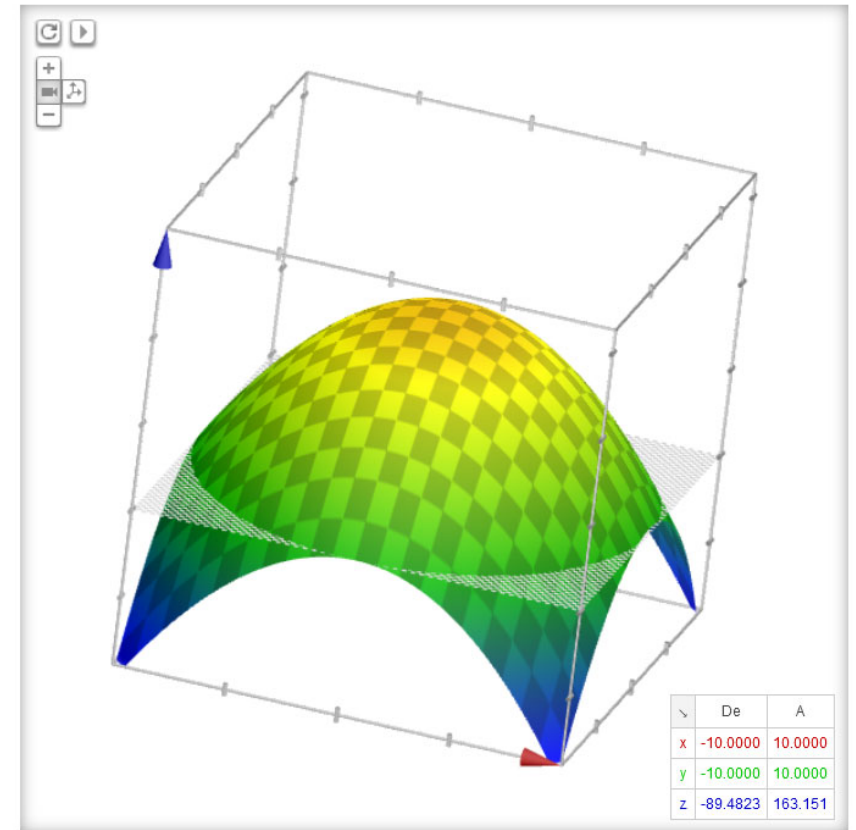
Partícula₂:

$$V_2(1) = 0.9 * (0,0) + [((3,8) - (3,8)) + ((6,4) - (3,8))] = (3,-4)$$

$$X_2(1) = (3,8) + 0.2 * (3,-4) = (3.6,7.2)$$

$$b_2(1) = 100 - (3.6^2 + 7.2^2) = 35.2$$

B(1) = (6,4) mejor posición global (calidad = 48)



Ejemplo 2. Dos dimensiones

$$v_i(t+1) = 0.9v_i(t) + [b_i(t) - x_i(t)] + [B(t) - x_i(t)]$$
$$x_i(t+1) = x_i(t) + 0.2v_i(t+1)$$

Iteración 2 (t=2)

Partícula₁:

$$V_1(2) = 0.9 * (0,0) + [((6,4) - (6,4)) + ((6,4) - (6,4))] = (0,0)$$

$$X_1(2) = (6,4) + 0.2 * (0,0) = (6,4)$$

$$b_1(2) = 100 - (6^2 + 4^2) = 48$$

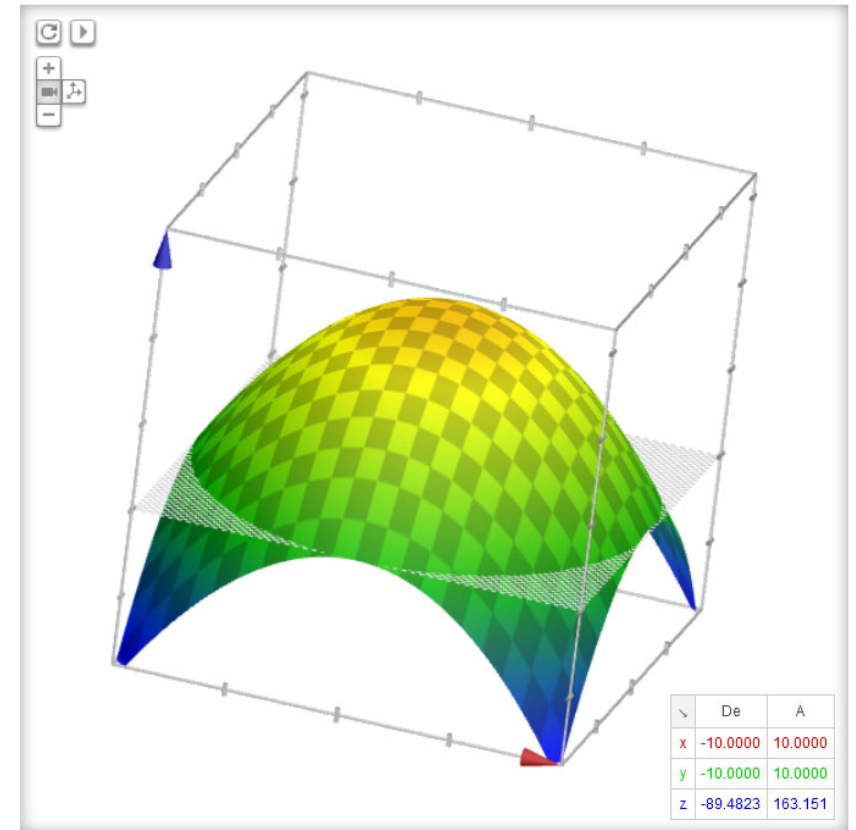
Partícula₂:

$$V_2(2) = 0.9 * (3,-4) + [((3.6,7.2) - (3.6,7.2)) + ((6,4) - (3.6,7.2))] = (5.1,-6.8)$$

$$X_2(2) = (3.6,7.2) + 0.2 * (5.1,-6.8) = (4.62,5.84)$$

$$b_2(2) = 100 - (4.62^2 + 5.84^2) = 44.55$$

B(2) = (6,4) mejor posición global (calidad = 48)



Ejemplo 2. Dos dimensiones

$$v_i(t+1) = 0.9v_i(t) + [b_i(t) - x_i(t)] + [B(t) - x_i(t)]$$
$$x_i(t+1) = x_i(t) + 0.2v_i(t+1)$$

Iteración 3 (t=3)

Partícula₁:

$$V_1(3) = 0.9 * (0,0) + [((6,4) - (6,4)) + ((6,4) - (6,4))] = (0,0)$$

$$X_1(3) = (6,4) + 0.2 * (0,0) = (6,4)$$

$$b_1(3) = 100 - (6^2 + 4^2) = 48$$

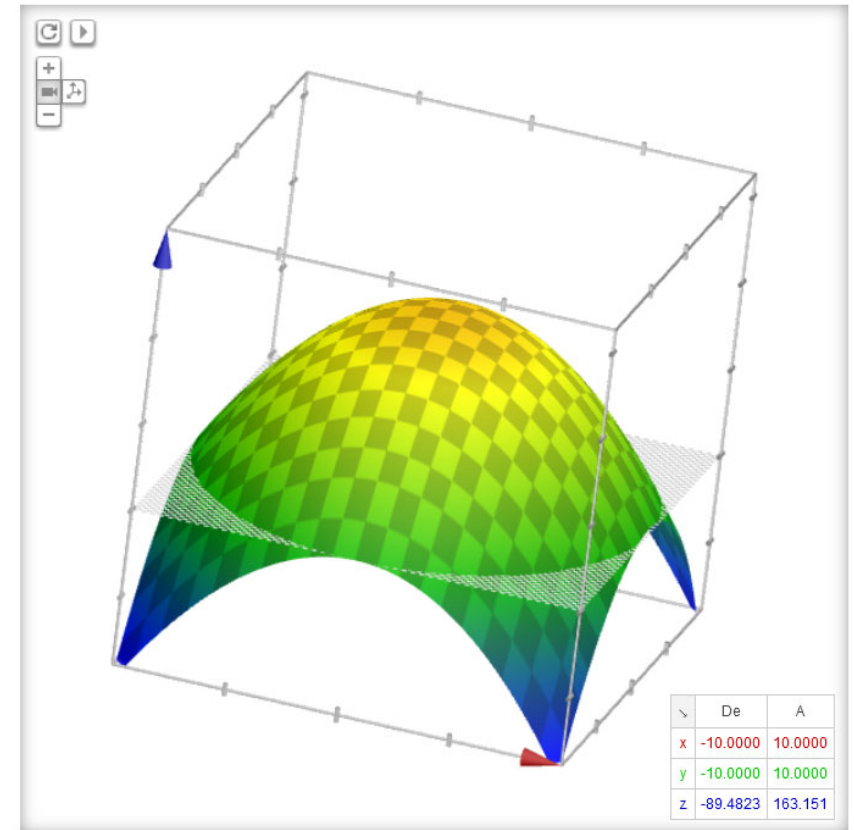
Partícula₂:

$$V_2(3) = 0.9 * (5.1, -6.8) + [((4.62, 5.84) - (4.62, 5.84)) + ((6,4) - (4.62, 5.84))] = (5.97, -7.96)$$

$$X_2(3) = (4.62, 5.84) + 0.2 * (5.97, -7.96) = (5.814, 4.248)$$

$$b_2(3) = 100 - (5.814^2 + 4.248^2) = 48.159$$

B(3) = (5.814, 4.248) mejor posición global (calidad = 48.159)

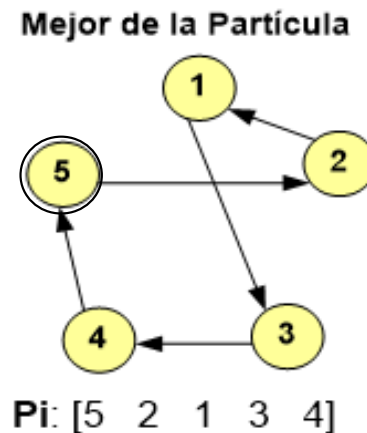
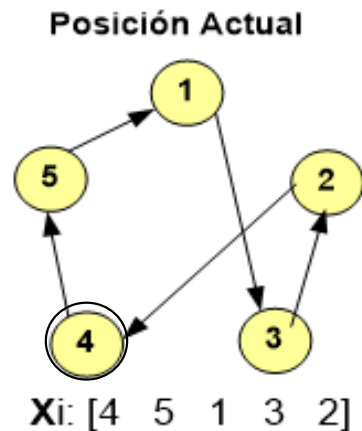


Nuevas iteraciones...

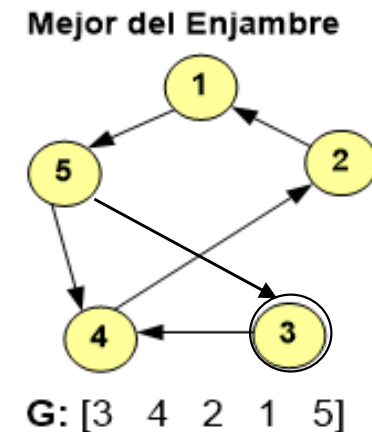
Ejemplo 3. Problema del viajante de comercio (decidiendo origen)

Particle Swarm Optimization Algorithm for the Traveling Salesman Problem. E. Goldberg, M. Goldberg, G. Souza. In: 'Travelling Salesman Problem'

Cada partícula, concretamente su posición X_i , representa un recorrido (solución en un espacio de soluciones)



$$\begin{aligned} P_i &= [5 \ 2 \ 1 \ 3 \ 4] \\ X_i &= [4 \ 5 \ 1 \ 3 \ 2] \\ P_i - X_i &= [1 \ -3 \ 0 \ 0 \ 2] \end{aligned}$$



$$\begin{aligned} G &= [3 \ 4 \ 2 \ 1 \ 5] \\ X_i &= [4 \ 5 \ 1 \ 3 \ 2] \\ G - X_i &= [-1 \ -1 \ 1 \ -2 \ 3] \end{aligned}$$

Ejemplo 3. Problema del viajante de comercio

$$P_i - X_i = [1 \quad -3 \quad 0 \quad 0 \quad 2]$$

$$G - X_i = [-1 \quad -1 \quad 1 \quad -2 \quad 3]$$

Supongamos velocidad de la Partícula_i: $V_i = [3 \quad 5 \quad 2 \quad 6 \quad -8]$

$$V'_{ik} = w * V_{ik} + c_1 * (P_{ik} - X_{ik}) * \text{RND}(0,1) + c_2 * (G_k - X_{ik}) * \text{RND}(0,1)$$

Nota: para todos los cálculos, $\text{RND}(0, 1) = 1$

$w=0.6$, $c_1=0.3$, $c_2=0.1$

La nueva velocidad V'_i se calcula (para cada dimensión) como:

$$V'_{i1} = 0.6 * 3 + 0.3 * 1 + 0.1 * (-1) = 2$$

$$V'_{i2} = 0.6 * 5 + 0.3 * (-3) + 0.1 * (-1) = 2$$

$$V'_{i3} = 0.6 * 2 + 0.3 * 0 + 0.1 * 1 = 1.3$$

$$V'_{i4} = 0.6 * 6 + 0.3 * 0 + 0.1 * (-2) = 3.4$$

$$V'_{i5} = 0.6 * (-8) + 0.3 * 2 + 0.1 * 3 = -3.9$$

Redondeando: $V'_i = [2 \quad 2 \quad 1 \quad 3 \quad -4]$

Ejemplo 3. Problema del viajante de comercio

$$X_i = [4 \ 5 \ 1 \ 3 \ 2]$$

$$V'_i = [2 \ 2 \ 1 \ 3 \ -4]$$

Se define $X'_i = X_i \otimes V'_i$ donde $\otimes$ representa el número de desplazamientos (*swap* positivo o negativo) de cada nodo en el vector

$\otimes$ = información dependiente del dominio

$$X'_i = X_i \otimes V'_i = [4 \ 5 \ 1 \ 3 \ 2] \otimes [2 \ 2 \ 1 \ 3 \ -4] = [5 \ 1 \ 3 \ 4 \ 2]$$

$$[4 \ 5 \ 1 \ 3 \ 2] \otimes 2 = [5 \ 1 \ 4 \ 3 \ 2]$$

$$[5 \ 1 \ 4 \ 3 \ 2] \otimes 2 = [1 \ 4 \ 5 \ 3 \ 2]$$

$$[1 \ 4 \ 5 \ 3 \ 2] \otimes 1 = [4 \ 1 \ 5 \ 3 \ 2]$$

$$[4 \ 1 \ 5 \ 3 \ 2] \otimes 3 = [1 \ 5 \ 3 \ 4 \ 2]$$

$$[1 \ 5 \ 3 \ 4 \ 2] \otimes -4 = [5 \ 1 \ 3 \ 4 \ 2]$$

Nueva posición $X'_i = [5 \ 1 \ 3 \ 4 \ 2]$

Variantes del Algoritmo PSO

Integrated Particle Swarm Optimization (iPSO)

- En PSO estándar, la actualización de cada partícula (X_i) se guía por dos posiciones significativas del espacio de búsqueda: P_i , G
- En PSO estándar, si la partícula actual se encuentra muy cerca de cualquiera de estas dos posiciones, sus capacidades de orientación se reducen o incluso se desvanecen (se hacen las restas $P_i - X_i$ y $G - X_i$)

iPSO incorpora una nueva partícula, llamada partícula ponderada (X^W):

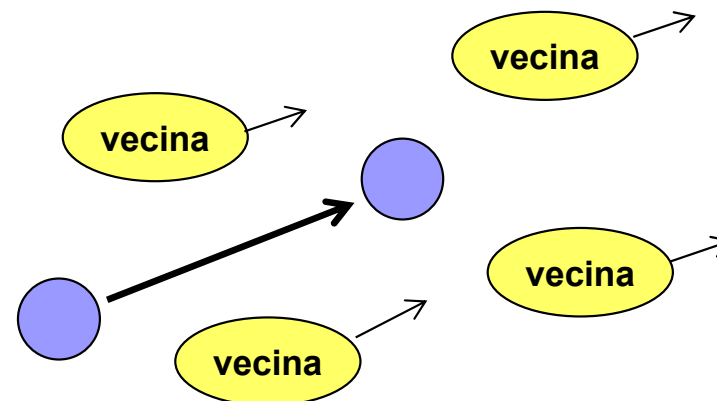
X^W es la posición ponderada media de las mejores posiciones P_i de cada una de las M partículas del enjambre:

$$X^W = \sum_{i=1, M} c_i^w * P_i \quad ; \text{ siendo } c_i^w \text{ el factor de ponderación para cada partícula } P_i$$

Esta nueva partícula se incorpora en una revisada función de cruce (para obtener la nueva posición)

(Interactive search algorithm: A new hybrid metaheuristic optimization algorithm.

Mortazavi, Toğan, Nuhoğlu. Engi.App. of Artificial Intelligence, 71, May 2018)



Variantes del Algoritmo PSO

Hybrid Gravitational Search Algorithm & Particle Swarm Optimization (PSO – GSA)

Las partículas en un PSO clásico siguen unas reglas matemáticas de movimiento:

$$X_i(t+1) = X_i(t) + V_i(t), \quad \text{donde} \quad V_i = w * V_i + C_1 (p_{ik} - x_{ik}) * \text{RND} + C_2 (g_k - x_{ik}) * \text{RND}$$

En el método PSO, combinado con una Búsqueda Gravitacional (GSA), se asume que:

- Las partículas tienen masa, tal que, a mayor masa, generan mayor fuerza gravitacional. La masa depende de la bondad (fitness) de la partícula.
- Aparece una fuerza de atracción entre partículas, que depende de sus masas (fitness) y de su distancia relativa (en el espacio de soluciones).
- La fuerza de atracción provoca una aceleración (a_i) del movimiento de cada partícula, que depende de la combinación (Σ) de las fuerzas de atracción a la que está sometida por el resto de partículas
- La nueva ecuación de velocidad es:

$$V_i(t+1) = w \times v_i(t) + C_1 \times a_i(t) + \left(\frac{C_1}{C_2} \right) \times (p_g - x_i)$$

Mejora:

El PSO clásico no considera el fitness ni la distancia entre todas las partículas en el movimiento

El método PSO-GSA incluye una afectación común del fitness y distancia entre todas las partículas

A novel hybrid gravitational search particle swarm optimization algorithm. Khanh, Ling.
Engineering Applications of Artificial Intelligence Volume 102, June 2021

Variantes del Algoritmo PSO

Modelado de movimientos de grupos, vida artificial, flocking

Flocking: Movimiento colectivo de un alto número de entidades autónomas (rebaños, bandadas, manadas, etc.)

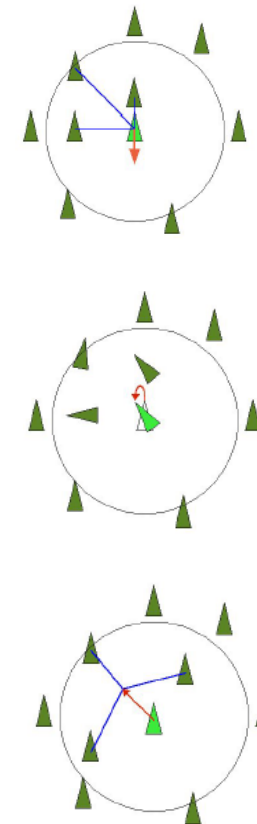
Consideración de otros factores o reglas en el movimiento de las partículas:

- *Separación*: cada partícula tiende a moverse lejos de sus vecinos si están demasiado cerca. Mantiene una distancia razonable, pero no demasiado lejos (*repulsión posición vecinos*).
- *Alineación*: cada partícula sigue la *dirección media* del movimiento de sus vecinas e igualando su velocidad. Búsqueda en el entorno local alineada con la búsqueda de sus partículas vecinas.
- *Cohesión*: las partículas siguen una dirección global y mantienen una cohesión de grupo con sus vecinas, tendiendo ir al centro del enjambre. Cada partícula tiende a moverse hacia la *posición media* de su vecindad.

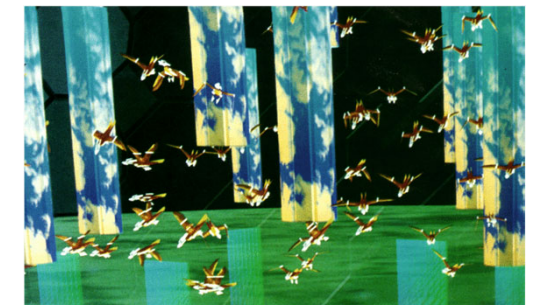
Modelo inicial flocking: Flocks, herds and schools: *A Distributed Behavioral Model*, Craig W. Reynolds. *Computer Graphics*, 21(4), July 1987.

Entorno Boids: <http://www.red3d.com/cwr/boids/>

Demo: <https://owenmcnaughton.github.io/Boids.js/>



DEMO



Variantes del Algoritmo PSO

- Extensión a problemas multi-objetivo (considerando Frontera de Pareto)
- Con o sin líder (únicamente *factor inercial y memoria*)
- Clustering: agrupación de partículas (mejores globales de cada grupo o líderes locales)
- Introducir una fuerza repulsiva entre las partículas (para evitar convergencia prematura)
- Perturbaciones momentáneas de las partículas ($\cong$ mejora local)
- **Variante APSO:** utilizar solo 'G' (mejor solución global): *Accelerated Particle Swarm Optimization (APSO, Xin-She Yang 2021)*:

$$\mathbf{v}_i^{t+1} = \mathbf{v}_i^t + \beta(\mathbf{g}^* - \mathbf{x}_i^t) + \alpha\epsilon_t \quad \text{simplificada como} \quad \mathbf{x}_i^{t+1} = (1 - \beta)\mathbf{x}_i^t + \beta\mathbf{g}^* + \alpha\epsilon_t$$

donde ϵ es una distribución gaussiana $\{0,1\}$, y típicamente $\alpha \approx 0.1 \sim 1$ and $\beta \approx 0.1 \sim 0.7$, ($\alpha \approx 1$, $\beta \approx 0.5$)

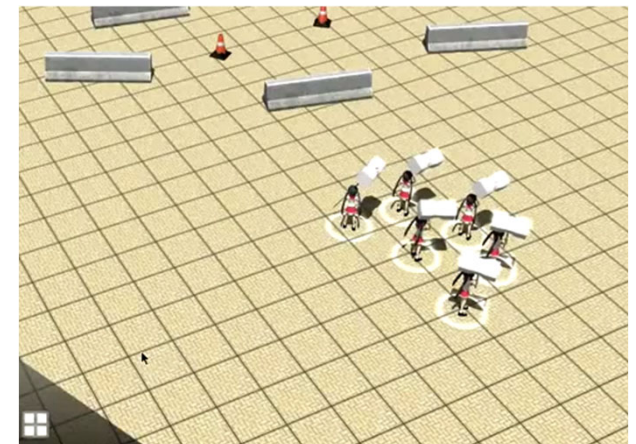
- **Parametrización:** incremento/decremento dinámico de la inercia (w), memoria local (c_1) y cooperación social (c_2)

Conclusiones de PSO

- Metaheurística de optimización basada en modelos de **conductas sociales**.
- Requiere un conjunto de soluciones iniciales (partículas) que van moviéndose en un espacio de soluciones siguiendo unas reglas matemáticas (inercia, propias, sociales...)
- Combina exploración y explotación, ajustable con los parámetros del método:

w (inercia), c_1 (factor local), c_2 (factor social)

- **Doble** concepto de convergencia: 1) hacia la mejor solución del enjambre (mejor posición global G), y 2) del enjambre hacia una zona de 'buenas soluciones' (que pueden no incluir la óptima)
- Asume muy pocas hipótesis del problema o del espacio de soluciones (posición y velocidad de cada partícula)
- Amplia utilización actual. Muchas **variantes** y aplicaciones



...otras Metaheurísticas Sociales

Bio-inspirada: El objetivo de una colonia de abejas es encontrar buenas zonas de alimento (flores) y maximizar la cantidad de néctar recogido.

El método supone 3 tipos de abejas:

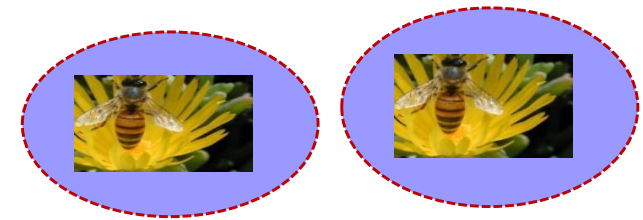
- Exploradoras: exploran áreas amplias, buscando nuevas fuentes de alimento (exploración).

Cuando encuentran nuevas fuentes de alimento, evalúan la cantidad de néctar y retienen en su memoria solo la mejor zona encontrada. Aproximadamente son el 10%.

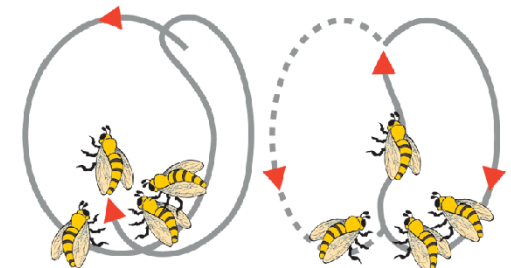
- Recolectoras/trabajadoras: viajan a una fuente concreta de alimento y recolectan néctar. También examinan y recolectan en las zonas de alimento cercanas (explotación).

Cuando las abejas exploradoras y las recolectoras regresan a la colmena, inician un baile compartiendo la información que poseen (ubicación y calidad de las fuentes encontradas).

- En espera: esperan en la entrada de la colmena a que vuelvan las exploradoras y las recolectoras. Reciben la información del baile, y pasan a ser abejas recolectoras de las mejores zonas.



recolectoras



en espera

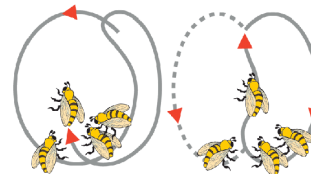
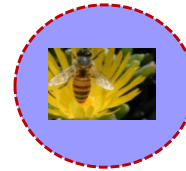
Solución $\Leftrightarrow$ Posición de la fuente de alimento

Calidad Solución $\Leftrightarrow$ Cantidad de Néctar de la fuente

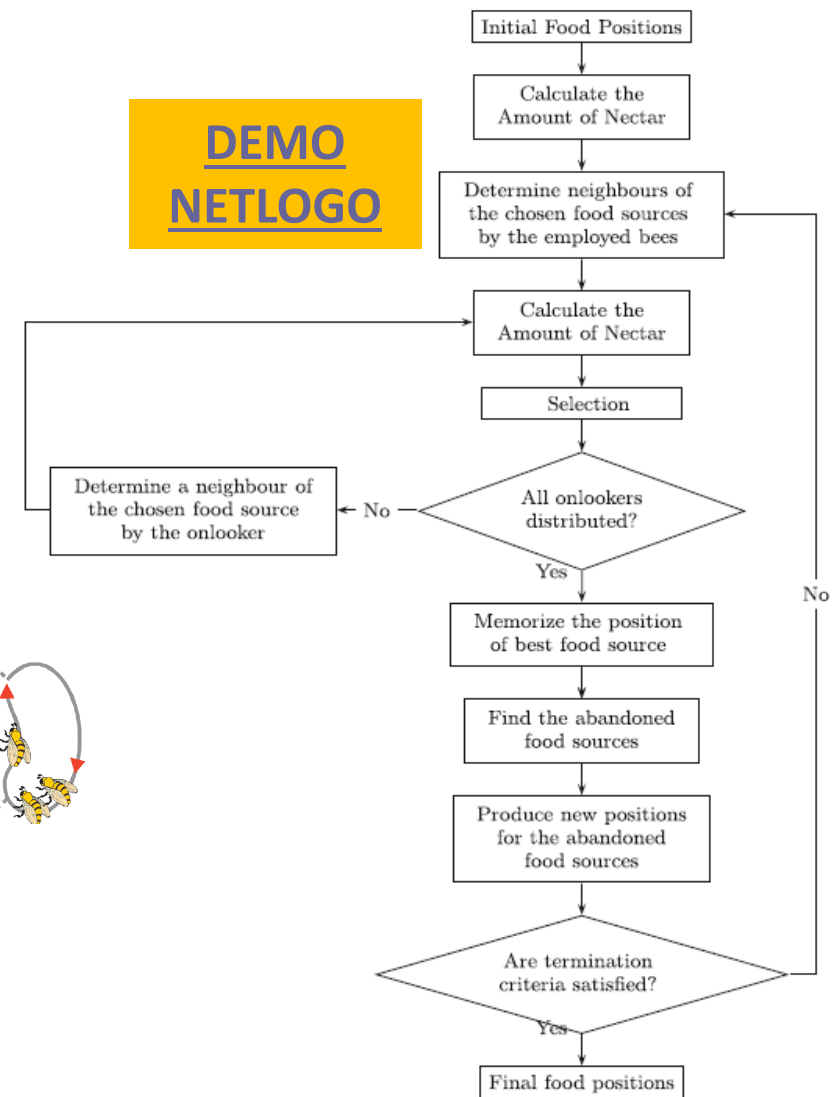
Abeja (exploradora, recolectora) $\Leftrightarrow$ Proceso de búsqueda

Esquema básico

- Nivel exploración: Obtención de nuevas soluciones.
 - Lanzar procesos exploradores en busca de soluciones.
 - Se forma una población de soluciones.
- Nivel explotación: Búsqueda Local para mejora de soluciones
 - Lanzar procesos explotadores, explotando soluciones cercanas a obtenidas.
 - Se añaden a la población de soluciones
- Evaluación de soluciones (Danza de las abejas)
 - Evaluación de soluciones (néctar obtenido por exploradoras y recolectoras).
 - Se lanzan nuevos procesos (abejas en espera) que explotan las mejores soluciones de la población.
- Ciclo, hasta condición de terminación.



DEMO NETLOGO



Parámetros:

- Número de procesos/abejas totales
- Número de procesos/abejas de cada tipo (%), o soluciones seleccionadas para ser mejoradas.

Algoritmo inspirado en murciélagos

(Bat Algorithm - BA)

A New Metaheuristic Bat-Inspired Algorithm. X. S. Yang. Nature Inspired Cooperative Strategies for Optimization, Studies in Computational Intelligence, 284, (2010)

Cada murciélago virtual vuela al azar con una velocidad v_i a la posición (solución) x_i con una frecuencia de onda e intensidad A_i .

Cuando localiza una presa, cambia su frecuencia e intensidad y la búsqueda se intensifica como una búsqueda local, que continúa hasta que se cumpla el criterio de parada.

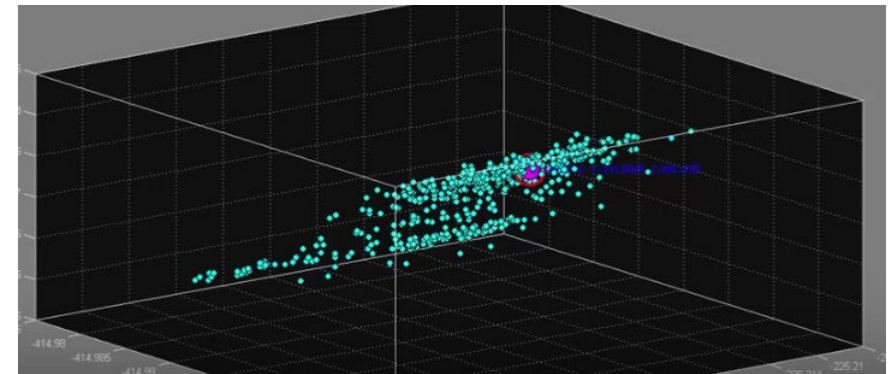
El proceso completo se repite iterativamente hacia una nueva presa.

De esta forma, se asume que, mediante el ajuste de la frecuencia, se puede controlar el balance entre exploración y explotación.

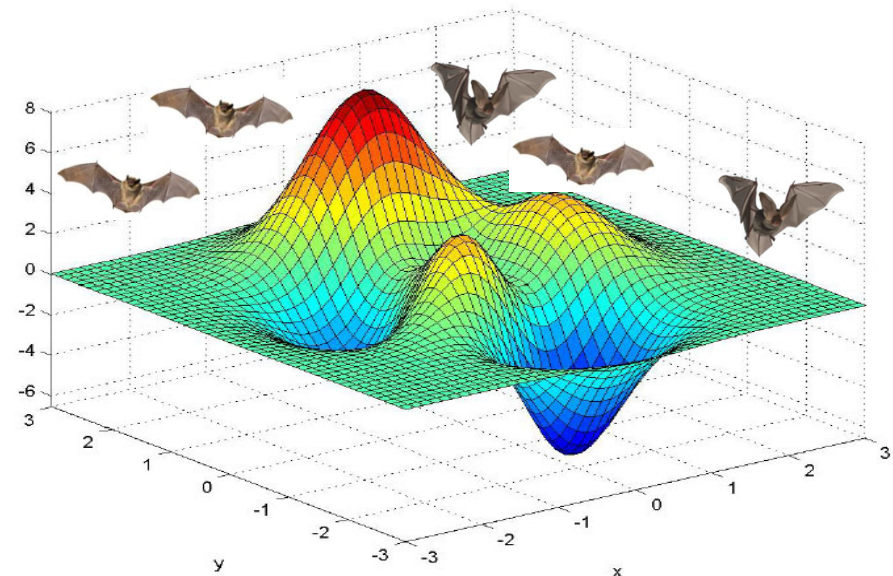
An improved discrete bat algorithm for symmetric and asymmetric Traveling Salesman Problems

Eneko Osaba^{a,b,*}, Xin-She Yang^b, Fernando Diaz^a, Pedro Lopez-García^a, Roberto Carballedo^a

^a Deusto Institute of Technology (DeustoTech), University of Deusto, Av. Universidades 24, Bilbao 48007, Spain
^b School of Science and Technology, Middlesex University, Hendon Campus, London, NW4 4BT, United Kingdom



La interacción entre partículas se basa en la frecuencia e intensidad de las vecinas



Enjambre de Luciérnagas (Glow worm swarm optimization, Krishnanand, Ghose, 2005)

Algoritmo de Luciérnagas (Firefly Algorithm , X. S. Yang 2009)

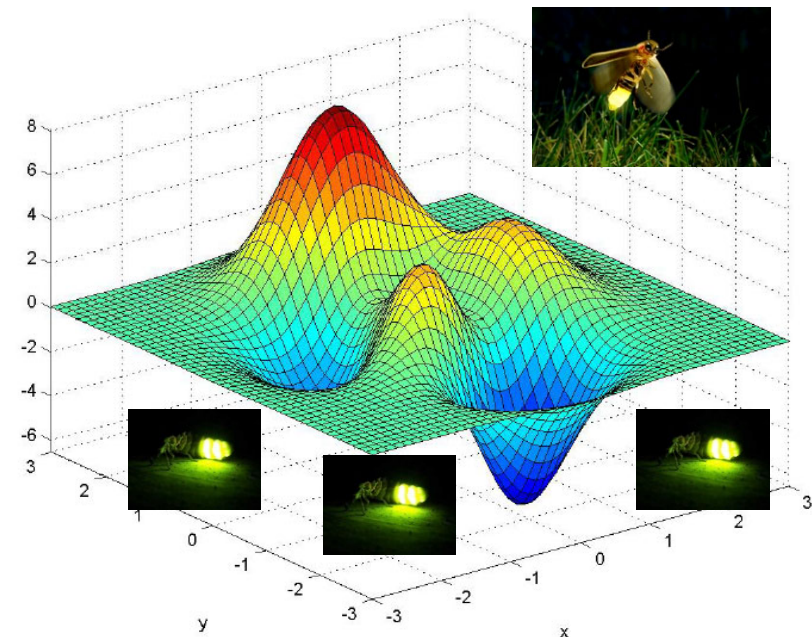


- Las luciérnagas basan su comportamiento social en la luminosidad que emiten (luciferina).
- En su cortejo nocturno, los machos vuelan en busca de pareja mientras emiten destellos de luz característicos de cada especie.
- Las hembras de la misma especie pueden responder con destellos específicos y así el apareamiento puede ocurrir.
- Con similar planteamiento, la luminosidad de una luciérnaga depende de la calidad de la solución encontrada y la distancia desde donde las otras compañeras están buscando soluciones.
- Cada luciérnaga selecciona, utilizando un mecanismo probabilístico, un vecino que tiene un valor más alto de luciferina que su propio y se mueve hacia él.

Aplicaciones: Diseño de redes ópticas, Canales de comunicación, Infraestructuras (puentes), etc.



La interacción entre partículas se basa en la luminosidad (intensidad y color)



Metaheurísticas de arañas sociales

A swarm optimization algorithm inspired in the behavior of the social-spider.

Cuevas, Cienfuegos, Zaldívar, Pérez-Cisneros. Expert Systems with Applications, 40 (16), (2013), pp. 6374-6384



Applied Soft Computing

Volume 30, May 2015, Pages 614–627



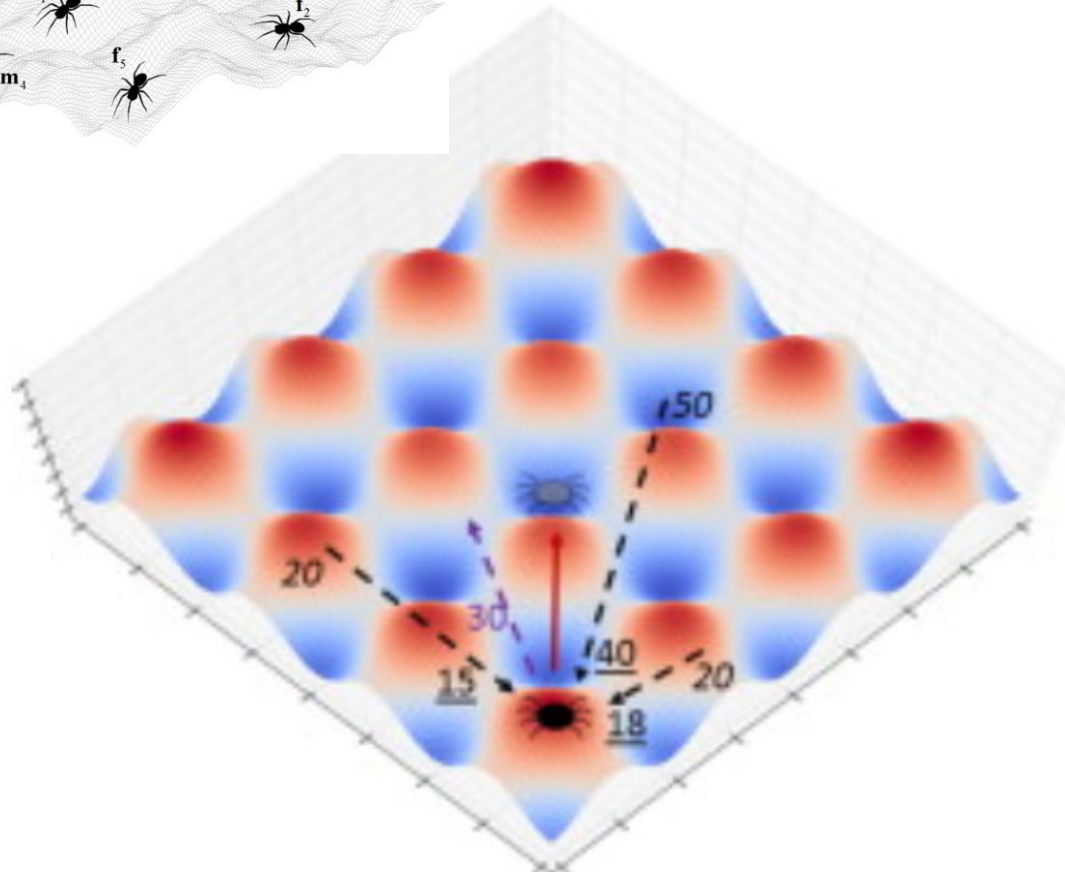
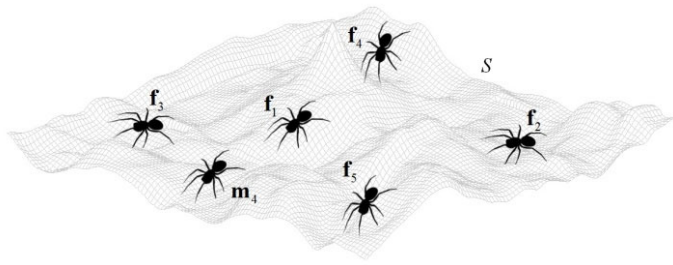
A social spider algorithm for global optimization

James J.Q. Yu , Victor O.K. Li

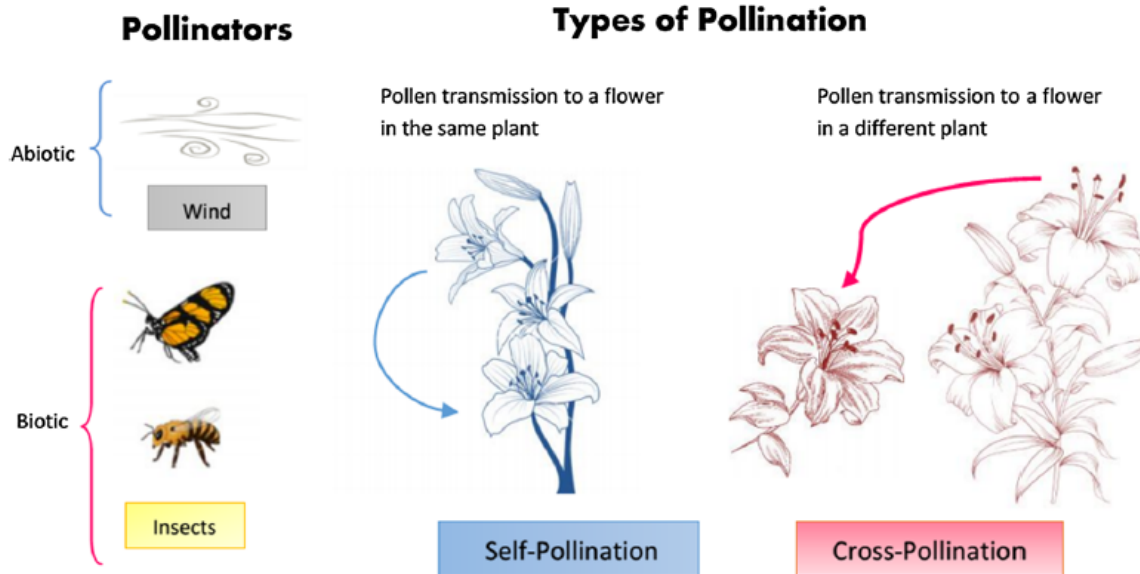
[Show more](#)

doi:10.1016/j.asoc.2015.02.014

[Get rights and content](#)



- Current position
- New position
- Received vibration
- 20 Vibration intensity at source
- 15 Attenuated vibration intensity
- Target vibration at last iteration
- 30 Target vibration intensity
- Spider Movement



Flower pollination algorithm: a comprehensive review

Mohamed Abdel-Basset¹ · Laila A. Shawky¹

Metaheurística que toma su metáfora en la proliferación de flores en las plantas

```

1: Initialize parameters with switching probability  $p \in [0,1]$  ;
2: Generate initial population of flowers randomly;
3: Evaluate initial population and find the current best solution  $gbest$ ;
4: while (stopping criterion not satisfied) do
5:   For each flower
6:     if  $rand() < p$ 
7:       Global pollination :  $x_i^{t+1} = x_i^t + L(x_i^t - gbest)$  ; // Based on Lévy step
8:     else
9:       Select two random solutions  $x_j^t$  and  $x_k^t$  ;
10:      Local pollination :  $x_i^{t+1} = x_i^t + \epsilon(x_j^t - x_k^t)$ ;
11:    end if
12:    Evaluate new solutions;
13:    Update solutions with better new ones;
14:  end for
15:  Keep the current best solution;
16: end while
    
```

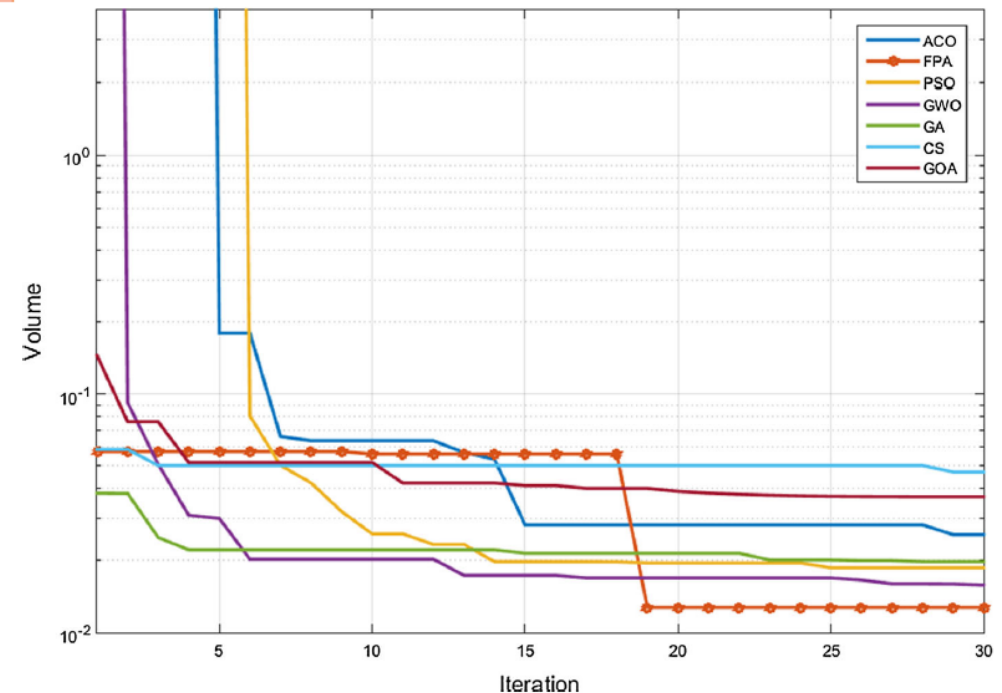
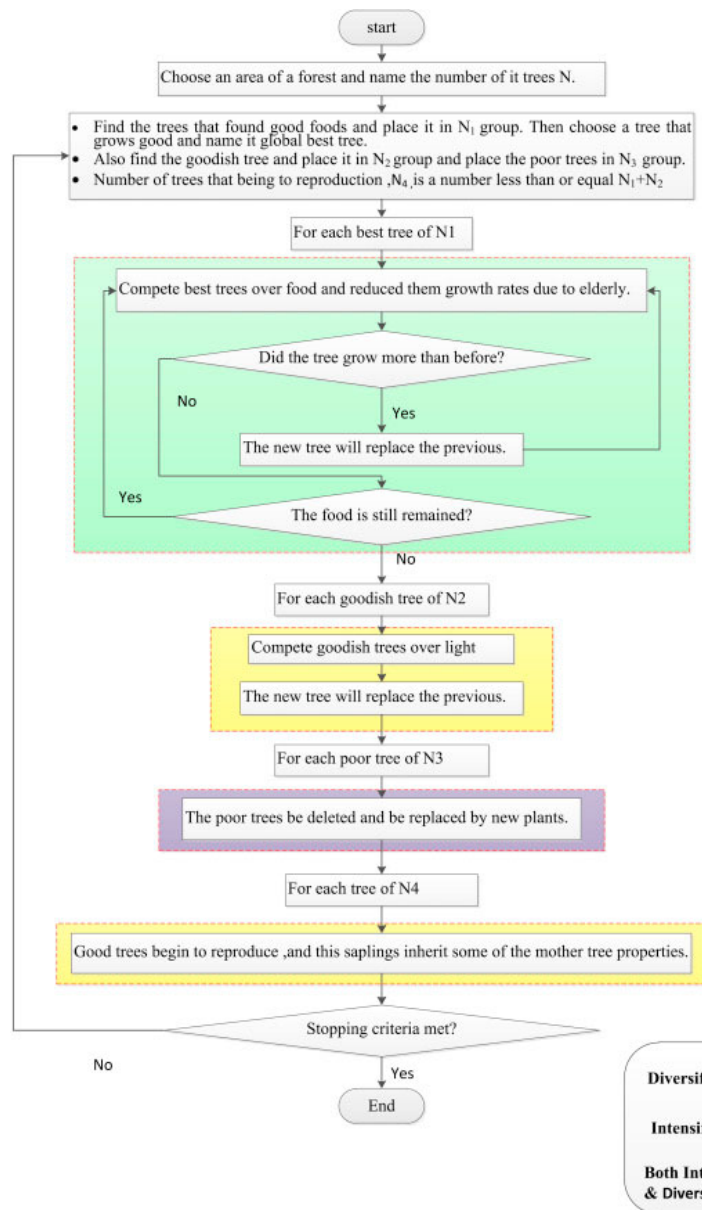


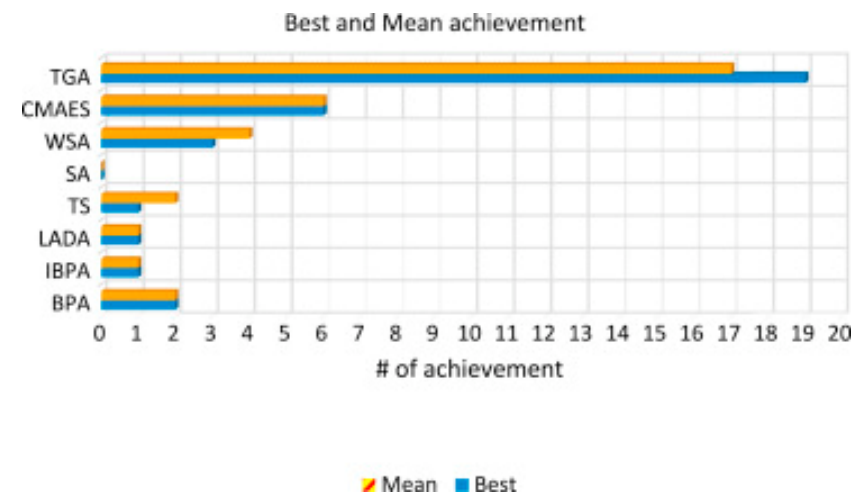
Fig. 4 Convergence of FPA and other metaheuristics

Metaheurística basada en la conducta de los árboles

Competencia para obtener luz y comida



Tree Growth Algorithm (TGA): A novel approach for solving optimization problems





Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai



Yin-Yang-pair Optimization: A novel lightweight optimization algorithm



Varun Punnathanam, Prakash Kotecha*

Department of Chemical Engineering, Indian Institute of Technology Guwahati, Guwahati 781039, Assam, India

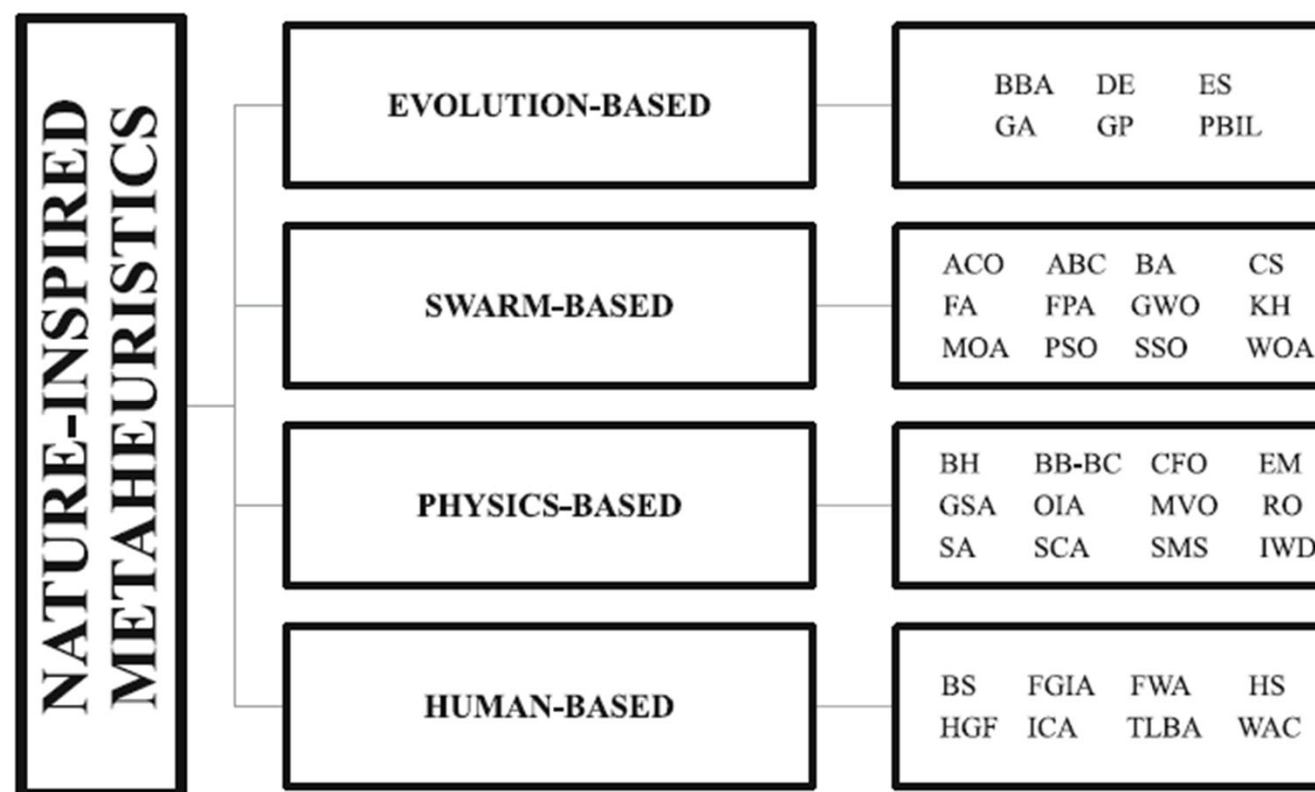
Basada en la filosofía del Yin y Yang
entre fuerzas en conflicto:
Exploración y Explotación





From ants to whales: metaheuristics for all tastes

Fernando Fausto¹ · Adolfo Reyna-Orta² · Erik Cuevas¹ · Ángel G. Andrade² · Marco Perez-Cisneros¹

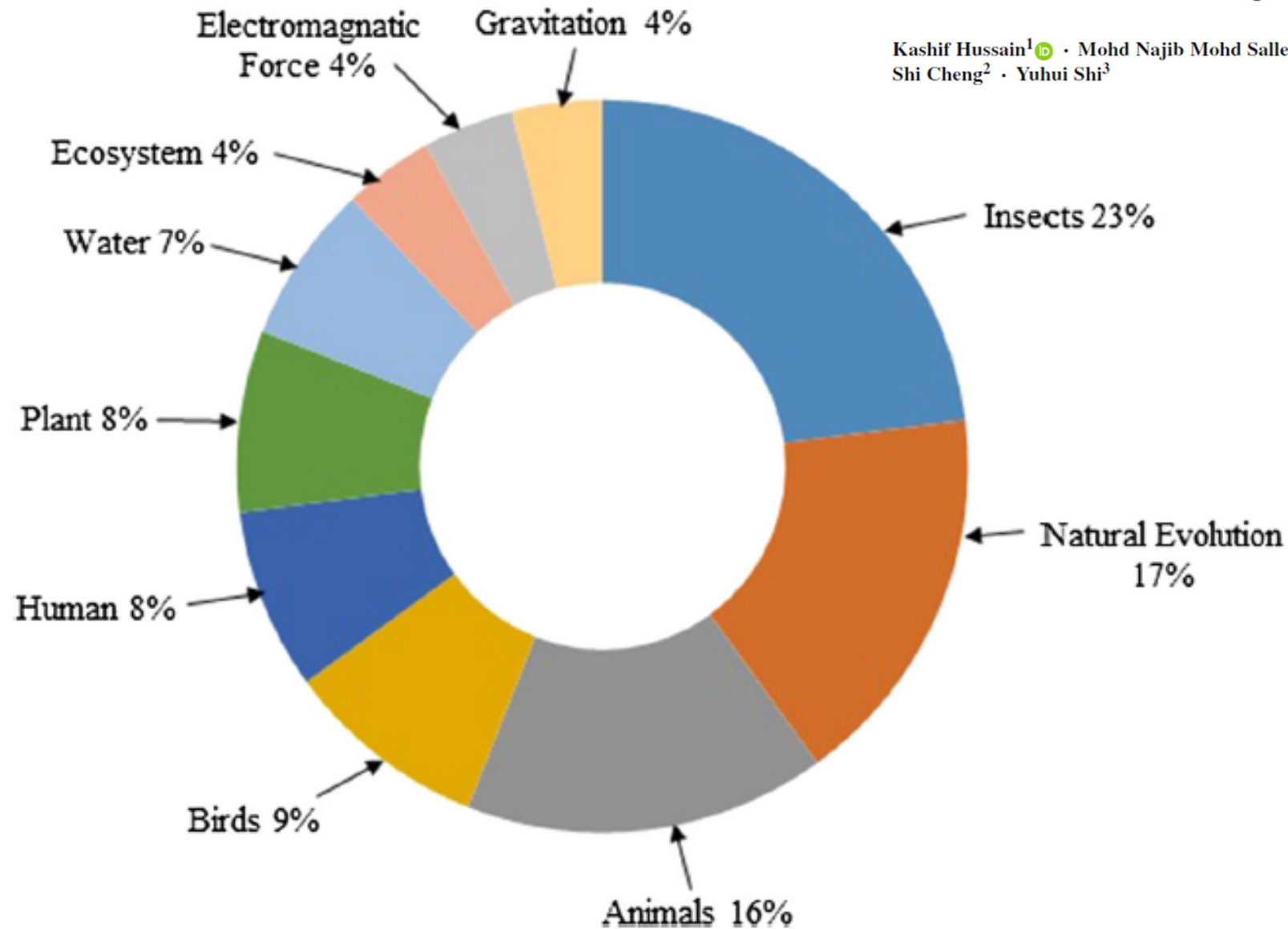


Variantes Metaheurísticas

Reference	Algorithm name	Reference	Algorithm name
Robbins and Monro (1951)	Stochastic Approximation Method	Wierstra et al. (2008)	Natural Evolution Strategies
Hooke and Jeeves (1961)	Pattern Search (PS)	Yang (2009)	Firefly Algorithm (FA)
Rastrigin (1963)	Random Search (RS)	Teodorović (2009)	Bee Colony Optimization (BCO)
Matyas (1965)	Random Optimization	He et al. (2009)	Group Search Optimizer (GSO)
Fogel et al. (1966)	Evolutionary strategy (ES)	Yang and Deb (2009)	Cuckoo Search (CS)
Hastings (1970)	Metropolis–Hastings algorithm	Rashedi et al. (2009)	Gravitational Search Algorithm (GSA)
Kernighan and Lin (1970)	Graph Partitioning Method	Husseinzadeh Kashan (2014)	League Championship Algorithm (LCA)
Holland (1975)	Genetic Algorithm (GA)	Kadioglu and Sellmann (2009)	Dialectic Search
Glover (1977)	Scatter Search	Yang (2010a)	Bat Algorithm (BA)
Kirkpatrick et al. (1983)	Simulated Annealing (SA)	Lourenço et al. (2010)	Iterated Local Search(ILS)
Glover (1986)	Tabu Search (TS)	Shah-Hosseini (2011)	Galaxy-based Search Algorithm (GbSA)
Farmer et al. (1986)	Artificial Immune System (AIS)	Tamura and Yasuda (2011b) ; Tamura and Yasuda (2011a)	Spiral Optimization (SO)
Moscato (1989)	Memetic Algorithm	Rajabioun (2011)	Cuckoo Optimization Algorithm (COA)
Koza (1992)	Genetic Programming (GP)	Rao et al. (2011)	Teaching-Learning-Based Optimization (TLBO)
Dorigo et al. (1996)	Ant Colony Optimization (ACO)	Alsheddy (2011)	Guided Local Search (GLS)
Fonseca and Fleming (1993)	Multi-Objective GA (MOGA)	Gandomi and Alavi (2012)	Krill Herd (KH) Algorithm
Battiti and Brunato (2010)	Reactive Search Optimization (RSO)	Hajiaghahi-Keshteli and Aminnayeri (2014)	Keshtel Algorithm (KA)
Srinivas and Deb (1994)	NSGA for Multi-Objective Optimization	Civicioglu (2012)	Differential Search Algorithm (DSA)
Eberhart and Kennedy (1995)	Particle Swarm Optimization (PSO)	Alatas (2012)	Artificial Chemical Reaction Optimization Algorithm (ACROA)
Hansen and Ostermeier (2001)	CMA-ES	Husseinzadeh Kashan (2013)	Optics Inspired Optimization(OIO)
Storn and Price (1997)	Differential Evolution (DE)	Husseinzadeh Kashan et al. (2015)	Grouping Evolution Strategies (GES)
Rubinstein (1997)	Cross Entropy Method (CEM)	Sadollah et al. (2013)	Mine Blast Algorithm (MBA)
Hansen et al. (1997)	Variable Neighborhood search(VNS)	Hatamlou (2013)	Black Hole (BH)
Taillard and Voss (1999)	Partial Optimization Metaheuristic Under Special Intensification Conditions (POPMUSIC)	Civicioglu (2013a)	Artificial Cooperative Search Algorithm(ACS)
Geem et al. (2001)	Harmony Search (HS)	Kaveh and Mahdavi (2015)	Colliding Bodies Optimization (CBO)
Hanseth and Aanestad (2001)	Bootstrap Algorithm (BA)	Gandomi (2014)	Interior Search Algorithm (ISA)
Larrañaga and Lozano (2002)	Estimation of Distribution Algorithms (EDA)	Salimi (2014)	Stochastic Fractal Search (SFS)
Deb et al. (2002)	NSGA-II for Multi-Objective Optimization	Cheng and Prayogo (2014)	Symbiotic Organisms Search (SOS)
Liu and Passino (2002)	Bacterial Foraging behavior Optimization (BFO)	Zheng (2015)	Water Wave Optimization (WWO)
Nakrani and Tovey (2004)	Bees Optimization	Dogan and Ölmez (2015)	Vortex Search Algorithm (VSA)
Zainal, Nurezayana and Zain, Azlan and Radzi, Nor and Udin, Amirmudin (2013)	Glowworm Swarm Optimization (GSO)	Wang et al. (2015)	Elephant Herding Optimization (EHO)
Basturk and Karaboga (2006)	Artificial Bee Colony Algorithm (ABC)	Baykasoğlu and Akpinar (2017)	Weighted Superposition Attraction (WSA)
Pham et al. (2005)	Bees Algorithms (BA)	Mirjalili (2016b)	Dragonfly algorithm
Haddad et al. (2006)	Honey-bee Mating Optimization (HMO)	Liang et al. (2016)	Virus Optimization Algorithm
Shah-Hosseini (2009)	Intelligent Water Drops (IWD)	Mirjalili (2016a)	Sine Cosine Algorithm (SCA)
Atashpaz-Gargari and Lucas (2007)	Imperialist Competitive Algorithm (ICA)	Ebrahimi and Khomehchi (2016)	Sperm Whale Algorithm (SWA)
Mucherino and Seref (2007)	Monkey Search (MS)	Mirjalili et al. (2017)	Salp Swarm Algorithm

Metaheuristic research: a comprehensive survey

Kashif Hussain¹  · Mohd Najib Mohd Salleh¹ ·
Shi Cheng² · Yuhui Shi³



Aplicaciones basadas en Inteligencia de Enjambre. Path-finding

Objetivo: encontrar el mejor camino (habitualmente, el más corto) entre dos puntos a través de un mapa con obstáculos. Problemas de laberinto, Aplicaciones para videojuegos, etc.

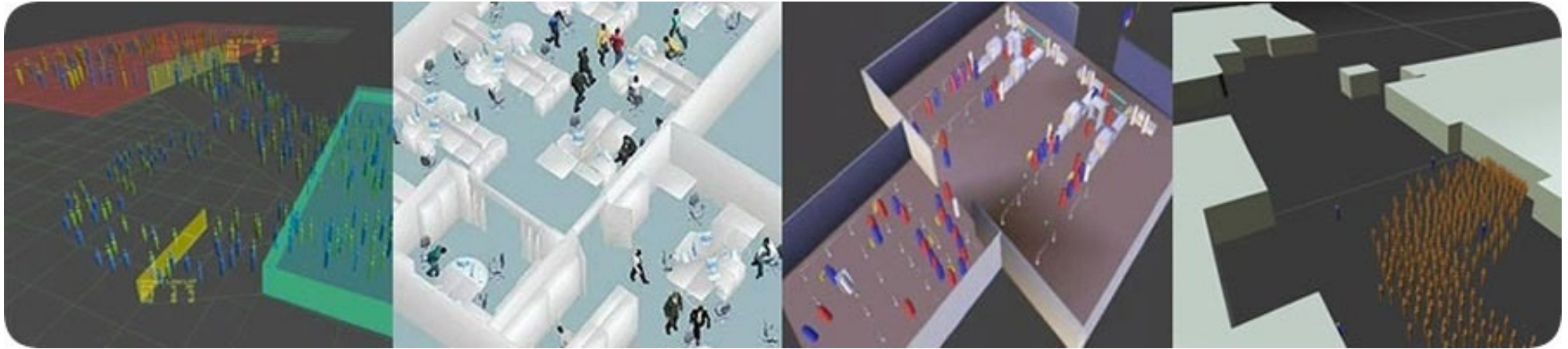
Algoritmos típicos: Dijkstra (muy costoso en laberintos complejos),
Algoritmos A (equivalente a Dijkstra, con $h(n)=0$),
A* sobre-informados, etc.

Metaheurísticas: Grasp, inteligencia de enjambre (algoritmo de las hormigas)...

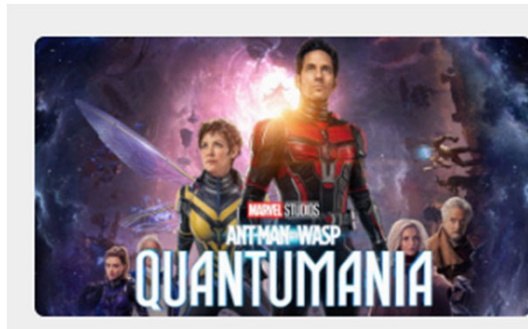


<http://www.movingai.com/benchmarks/>
Benchmarks for Grid-Based Path finding.
IEEE Trans. on Comp. Int. and AI in games (2012)

Aplicaciones basadas en Inteligencia de Enjambre. Simulación de comportamientos sociales y movimiento de individuos (swarm behaviour)



<http://massivesoftware.com>



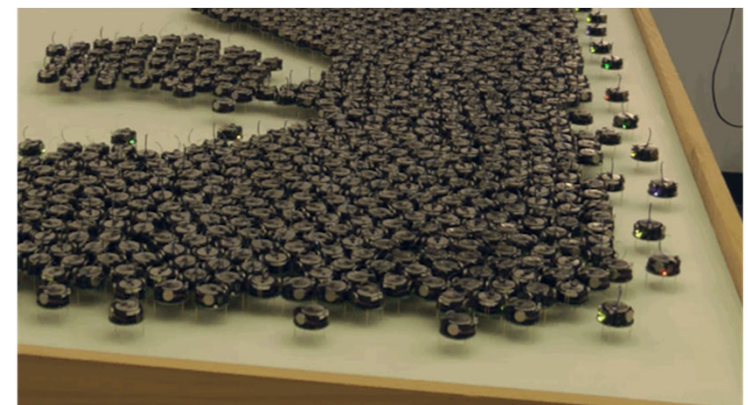
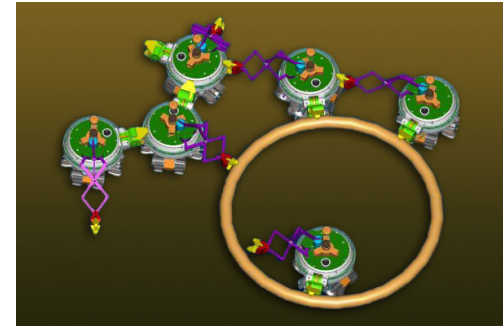
Swarm Robotics

Enfoque inspirado en el comportamiento colectivo de los animales sociales y que se centra en la interacción de múltiples robots

Hace hincapié en el control descentralizado, comunicación limitada entre agentes, la información local, el seguimiento de un comportamiento global y la robustez del grupo

Los sistemas de enjambre robóticos difieren de otros sistemas multi-robot en:

- Son robots autónomos ubicados en un determinado medio ambiente
- El enjambre tiene un gran número de robots, posiblemente agrupados en pequeños grupos de robots homogéneos
- Los robots son relativamente simples, tienen sensores locales y sus habilidades de comunicación son limitados
- La coordinación entre los robots se distribuye tal que el sistema es tolerante a fallos, debido a la redundancia de robots cada uno de los agentes en el sistema no es esencial y puede ser sustituido por otro agente
- Son fácilmente escalables, permitiendo que más agentes para ser agregados o eliminados de acuerdo a las demandas de la tarea



Científicos de Harvard crean un enjambre cooperativo de 1.000 robots

Objetivo: Aplicar técnicas de Inteligencia Colectiva

Design and implementation of self-organising and self-assembling artefacts. This novel approach finds its theoretical roots in recent studies in swarm intelligence: self-organising and self-assembling capabilities shown by social insects and other animal societies

The main tangible objective is the construction of at least one of such artefact (swarm-bot). That is, an artifact composed of a number of simpler, insect-like, robots(-bots), built out of relatively cheap components, capable of self-assembling and self-organising to adapt to its environment

Simulación de
Escenarios



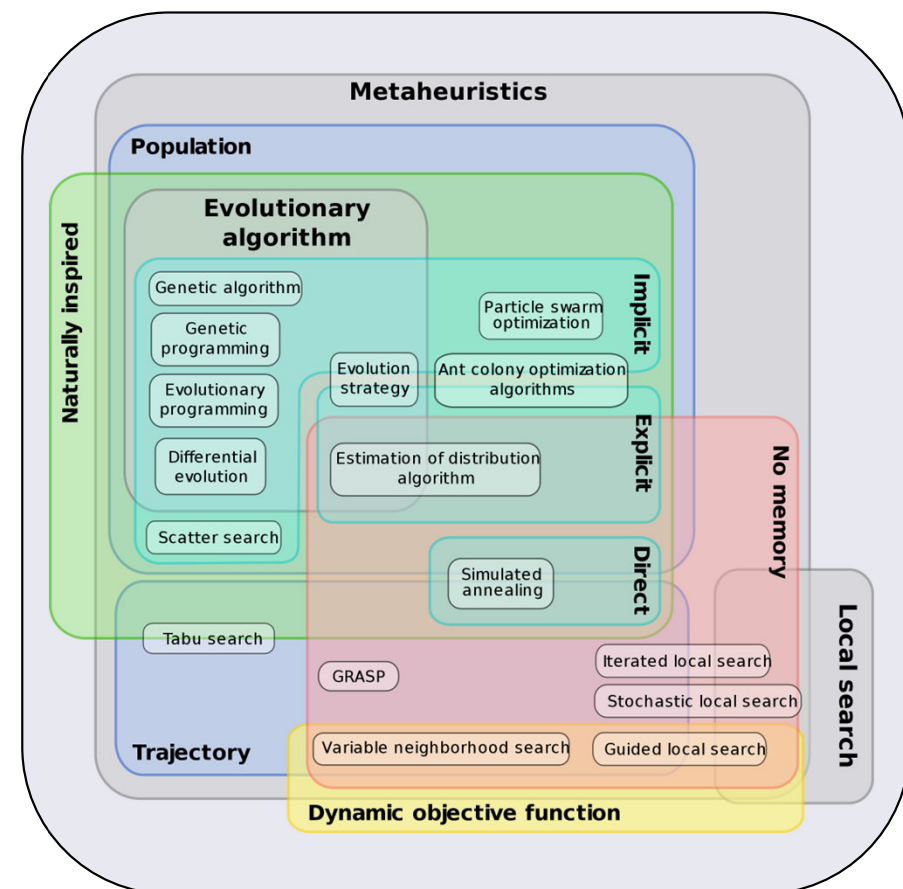
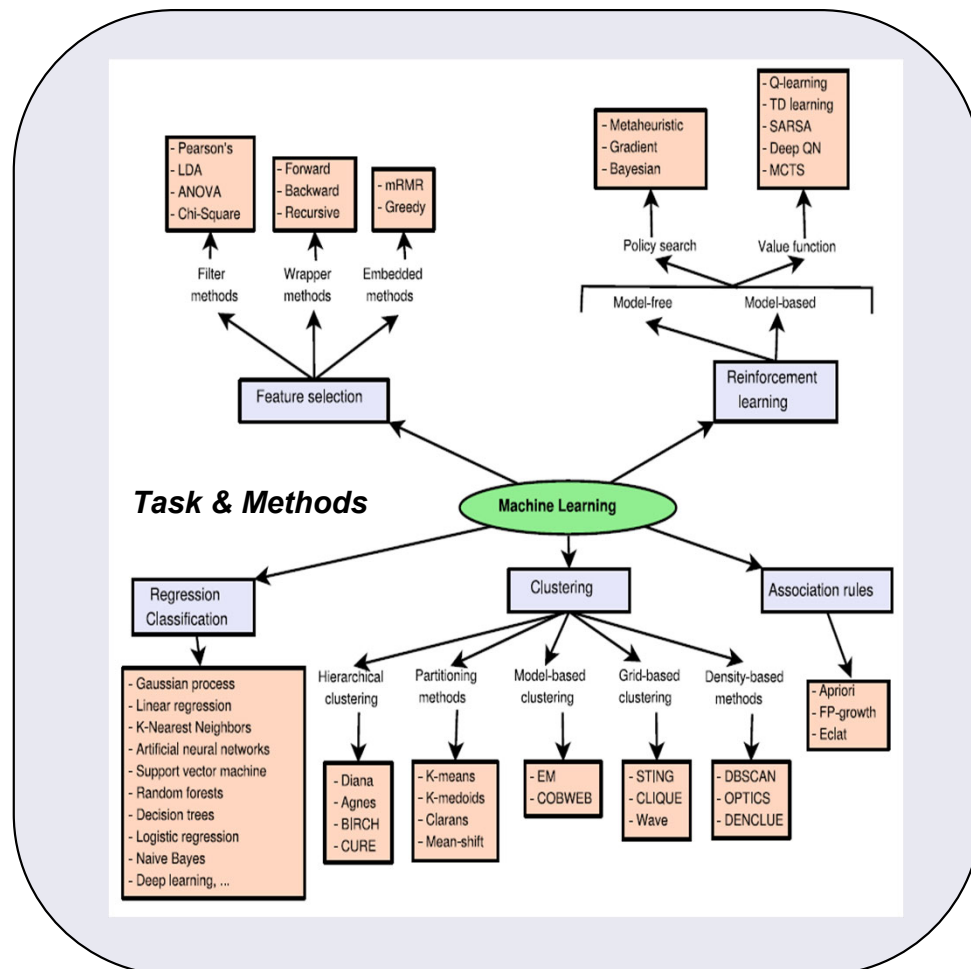
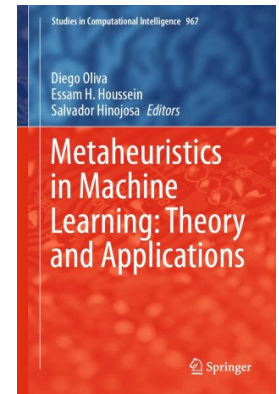
Swarm-Bots



Aplicación de Machine Learning a Metaheurísticas

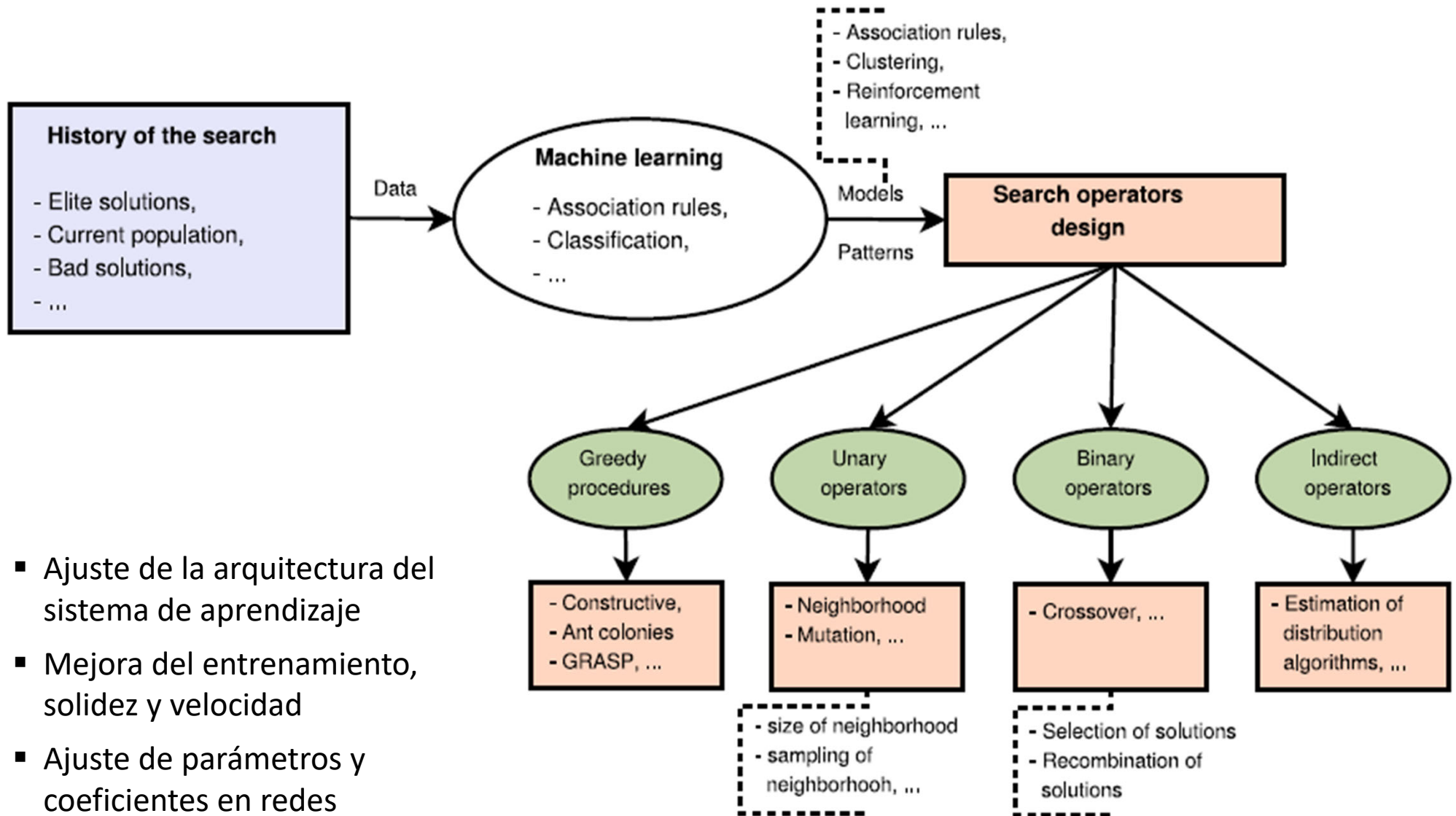
Machine Learning into Metaheuristics: A Survey and Taxonomy, ACM Computing Surveys. Vol. 54, Iss. 6 (2021)

Aplicación del aprendizaje automático (ML) para diseñar metaheurísticas más eficientes y efectivas (y viceversa)



Ayuda para *parameter tuning* (parametrización)

Extracción de conocimiento de búsquedas previas para la obtención de buenos operadores en nuevos problemas: parametrización de la metaheurística dependiente del problema (tamaño, tipología, etc.)



- Ajuste de la arquitectura del sistema de aprendizaje
- Mejora del entrenamiento, solidez y velocidad
- Ajuste de parámetros y coeficientes en redes neuronales

Ayuda para poblar las soluciones iniciales

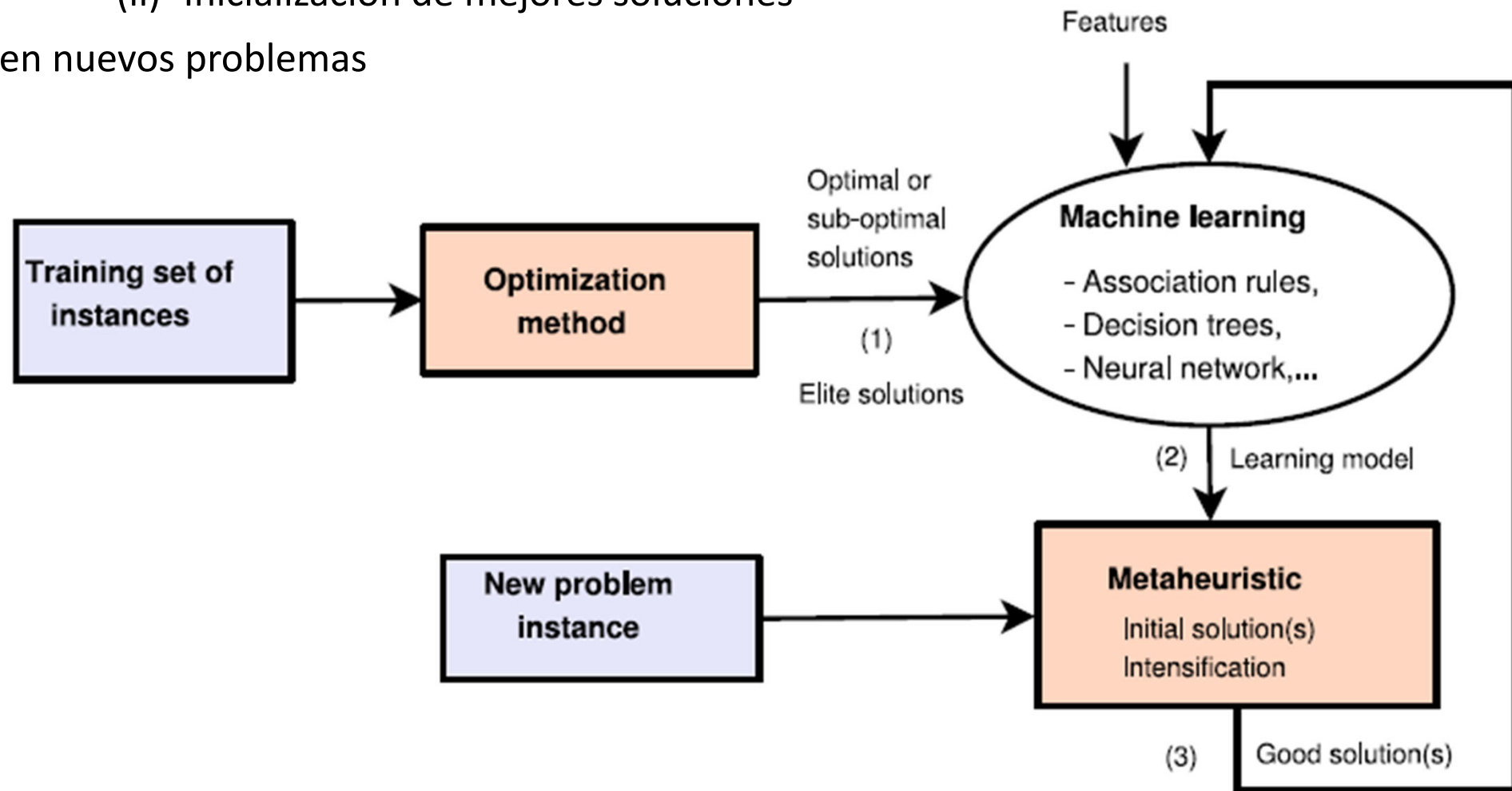
Aprendizaje de características/patrones/propiedades de buenas soluciones en un dominio de problemas,

para ayudar a la metaheurística a:

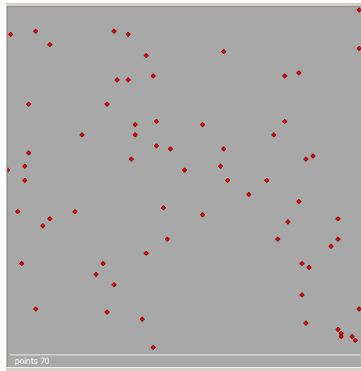
(i) buscar en regiones prometedoras del espacio de búsqueda, o

(ii) Inicialización de mejores soluciones

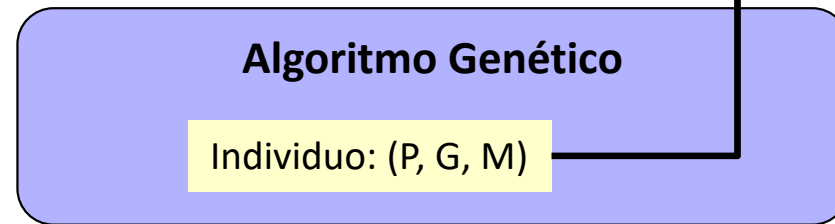
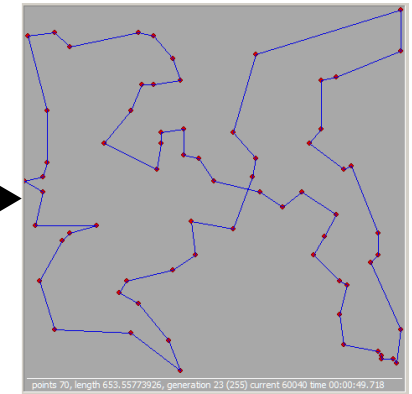
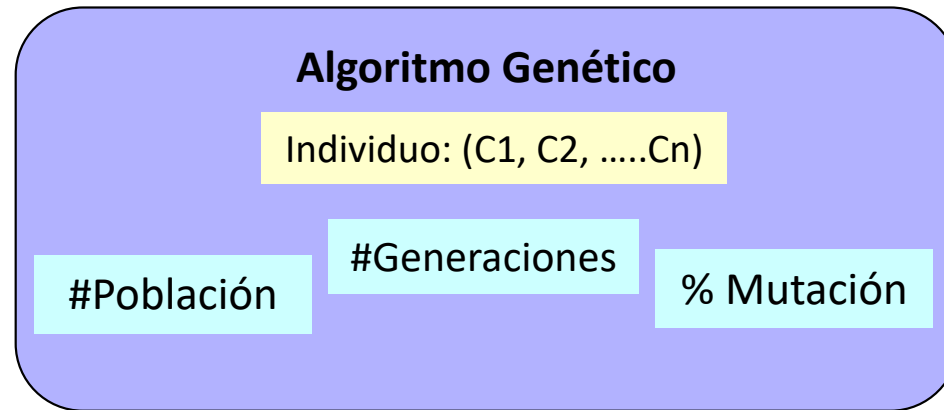
en nuevos problemas



Aplicación de Metaheurísticas a Metaheurísticas



- Un AG es un proceso estocástico (no garantía obtención de buenas soluciones).
- Ante un tipo de problemas, hay que establecer sus parámetros, en función de su tipología, tamaño, etc.
- Esto se hace mediante un análisis razonado del problema, esperando encontrar los parámetros que darán buenas soluciones (sin garantía).



Objetivo:
Mejor (P, G, M)

Se puede plantear un proceso de optimización metaheurístico de los parámetros de un AG, que obtenga los mejores parámetros ante similar clase de problemas: AG sobre AG